REEPHAM HIGH SCHOOL & COLLEGE / COMPUTER SCIENCE

H446 Paper 2

Algorithms and Programming

UNDERSTAND > MODEL > TRY > CHECK

Choose a chapter

Use the links below or the PDF bookmark panel. Each topic has separate practice and answer pages for independent revision.

2.1 Elements of computational thinking

Pages 4-43

2.2 Problem solving and programming

Pages 44-138

2.3 Algorithms

Pages 139-248

Exam practice and answers

Additional questions, worked examples and mark points

Practice questions are RHSC teaching materials, not OCR-issued examination papers.

Attempt the questions before reading the indicative answers. Use the PDF bookmarks to navigate.

CONTENTS

Choose your chapter

Select a chapter. Within each chapter, its original clickable topic contents and the PDF bookmarks navigate to individual four-page learning cycles.

2.1 **Elements of computational thinking**

Abstractly, ahead, procedurally, logically and concurrently

Pages 4-43 | 9 learning cycles

2.2 **Problem solving and programming**

Programming techniques and computational methods

Pages 44-138 | 18 core + 4 code cycles

2.3 **Algorithms**

Design, Big O, structures, traversals, searches, sorts and pathfinding

Pages 139-248 | 20 core + 6 code/trace cycles

Navigation note: the original chapter checklist pages retain chapter-local page references. Use the PDF viewer page number or the chapter bookmarks for master-page positions.

INDEPENDENT REVISION

How to use this guide

A consistent routine across all three chapters; read the model before attempting questions and keep indicative answers separate.

01 UNDERSTAND

Read the explanation, diagram and highlighted technical vocabulary.

02 MODEL

Trace the worked example and explain each step rather than just the outcome.

03 TRY

Use black pen; answer the exam-style questions without looking at the next page.

04 CHECK

Self-mark in green pen; correct one error and explain why the correction works.



GREEN PEN / IMPROVE YOUR ANSWER

1. Answer in black pen first. After CHECK, correct or improve in green.
2. If already correct, improve the hardest answer with a precise reason.

Example: Replace "RISC is fast" with a comparison of instructions and cycles.

MASTER CONTENTS

Choose a learning cycle

Select a row to jump to the four-page revision cycle. Each topic has an explanation, an original worked example, questions and a separate self-mark page.

2.1.1(a,b)	Thinking abstractly / Keep What Matters	pp. 6-9
2.1.1(c,d)	Thinking abstractly / Test The Assumptions	pp. 10-13
2.1.2(a,b)	Thinking ahead / Inputs And Preconditions	pp. 14-17
2.1.2(c)	Thinking ahead / Caching	pp. 18-21
2.1.2(d)	Thinking ahead / Reusable Components	pp. 22-25
2.1.3(a-d)	Thinking procedurally / Decompose And Order	pp. 26-29
2.1.4(a-c)	Thinking logically / Conditions And Paths	pp. 30-33
2.1.5(a)	Thinking concurrently / Dependencies	pp. 34-37
2.1.5(b)	Thinking concurrently / Trade-Offs	pp. 38-41

NAVIGATION TIP

Use the PDF bookmark panel or click a topic above. A live MCQ practice link is provided after each cycle.

INDEPENDENT REVISION

How to use each four-page cycle

01

UNDERSTAND

Read the explanation. Trace the diagram. Recall the highlighted vocabulary.

02

MODEL

Follow the complete worked example, not just the final output.

03

TRY

Work independently in black pen. Explain, trace or evaluate as asked.

04

CHECK

Self-mark in green pen. Correct one weakness and explain the fix.

STUDY TIP

Use the CHECK page after completing TRY. Correct errors and explain each change.

2.1 / ELEMENTS OF COMPUTATIONAL THINKING

Understand it. Model it. Try it. Check it.

The five official OCR thinking areas connect to practical algorithm design. Each subtopic here uses scenarios, diagrams, fully traced examples and independent practice.

2.1.1 Thinking abstractly

2.1.2 Thinking ahead

2.1.3 Thinking procedurally

2.1.4 Thinking logically

2.1.5 Thinking concurrently

WHAT REMAINS UNCHANGED

This is an independent chapter candidate.

2.1 / SPECIFICATION CHECKLIST

Coverage before page count

Nine four-page cycles cover all five numbered 2.1 topics and their 17 lettered subpoints. Tick only after you can apply the idea in an unfamiliar scenario.

SPEC	SPECIFICATION REQUIREMENT	PAGES
2.1.1(a,b)	Nature and purpose of abstraction	6-9
2.1.1(c,d)	Abstraction versus reality; devising models	10-13
2.1.2(a,b)	Inputs/outputs and preconditions	14-17
2.1.2(c)	Caching: nature, benefits and drawbacks	18-21
2.1.2(d)	Reusable program components	22-25
2.1.3(a-d)	Components, ordering, sub-procedures	26-29
2.1.4(a-c)	Decision points, logic and program paths	30-33
2.1.5(a)	Parts that can be tackled together	34-37
2.1.5(b)	Concurrent benefits and trade-offs	38-41

SPECIFICATION SOURCE

OCR H446 A Level Computer Science specification v2.7 (2024), section 2c, 2.1. Wording is summarised here; consult OCR for the official requirements.

2.1.1(a,b) / SEE IT / CONCEPT AND VISUAL

Keep the features the problem actually needs

Abstraction deliberately removes detail while retaining information that is relevant to a particular task.

MODEL

A purposeful simplified representation.

KEEP

Objects, links, costs and constraints needed to answer the question.

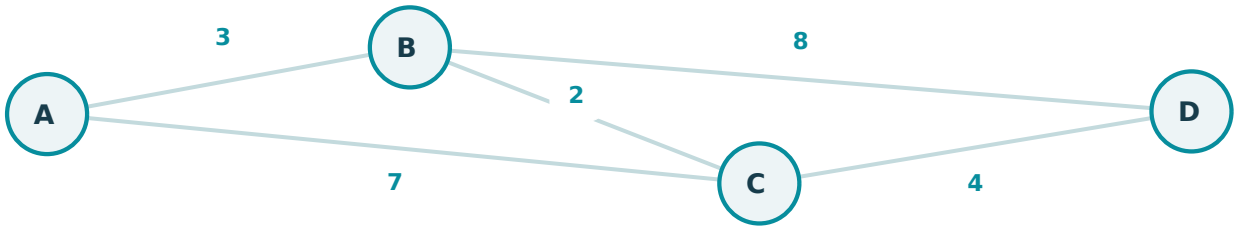
IGNORE

Appearance, decoration and background detail that do not affect the answer.

CHECK

Can the model still answer the original question?

A CITY MAP FOR FINDING THE SHORTEST ROUTE



FROM THE DIAGRAM TO EXAM LANGUAGE

A route planner can represent stations as vertices and direct journeys as weighted edges. Train colour and station-wall artwork usually do not change the shortest-route calculation.

WORDS THAT EARN MARKS

Abstraction	Relevant detail	**Model**	Representation
Vertex	Location	**Edge weight**	Journey cost

EXAM TRAP

Abstraction does not mean deleting as much as possible. Remove only what is irrelevant to the stated purpose.

2.1.1(a,b) / WORKED EXAMPLE / STEP BY STEP

From a real network to a weighted graph

A pupil needs the fastest route from A to D. Travel times: A-B=3, A-C=7, B-C=2, B-D=8, C-D=4 minutes.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Objective	**Minimise travel time**	The task defines which information is relevant.
2 / Nodes	**A, B, C, D**	A station is represented by a labelled vertex.
3 / Edges	**AB 3, AC 7, BC 2, BD 8, CD 4**	Only direct journeys become weighted edges.
4 / Compare	**A-B-C-D = 3+2+4 = 9**	This route uses an intermediate stop.
5 / Alternative	**A-C-D = 7+4 = 11**	A shorter-looking route can take longer.
6 / Decision	**Choose A-B-C-D: 9 minutes**	The model preserves the measure we optimise.

TRANSFER IT TO AN EXAM ANSWER

What we have not modelled: station colour, ticket design, seat layout or weather. Add them only if the question changes.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.1(a,b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define abstraction and state two types of detail that a shortest-route model should retain.

Q2 / 5 MARKS

Explain how a weighted graph abstracts the rail network above. Give two deliberately omitted details and justify one omission.

Q3 / 6 MARKS

A passenger now needs a wheelchair-accessible journey. Evaluate the original model and explain how to modify it to meet this changed requirement.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.1(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Abstraction is simplifying a situation to its relevant features [1]. Keep the connected stations/journeys [1] and a route cost such as travel time [1].

Q2 / 5

Stations represented as vertices [1]; direct journeys as edges [1]; travel minutes as edge weights [1]; irrelevant details might include paint colour or advertisements [1]; these are omitted because they do not change journey time [1].

Q3 / 6

Existing weights and connections support time optimisation [1], but the model omits accessibility [1]. Add a step-free attribute to stations or journeys [1]. Exclude inaccessible edges or apply an accessibility constraint [1]. Recalculate among valid routes [1]. Explain that adding only necessary features retains the value of abstraction [1].

COMMON MISCONCEPTION

Abstraction does not mean deleting as much as possible. Remove only what is irrelevant to the stated purpose.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.1(c,d) / SEE IT / CONCEPT AND VISUAL

A model is useful because it is not reality

Every abstraction introduces assumptions. A good model is accurate enough for its purpose, not a copy of the real world.

REAL WORLD

Hundreds of events and changing circumstances.

ASSUMPTION

A declared simplification, such as constant journey time.

MODEL

A smaller set of facts with clear relationships.

VALIDATE

Test the model against realistic and boundary scenarios.

DESIGN A LIBRARY LOAN MODEL

Real loan: ID / date / cover / shelf



Model: loan / due / return status

Ignored here: cover colour, shelf decoration

FROM THE DIAGRAM TO EXAM LANGUAGE

For overdue reminders, the system needs loan ID, borrower contact, book identity and due date. Cover colour and shelf decoration are irrelevant; renewals and lost books are not.

WORDS THAT EARN MARKS

- **Assumption** Simplification
- **Scope** Included cases
- **Validation** Check suitability
- **Constraint** Requirement

EXAM TRAP

A model that works for an ordinary case is not automatically suitable for every real-world case.

2.1.1(c,d) / WORKED EXAMPLE / STEP BY STEP

Model late returns without inventing unnecessary data

A school library sends an alert when a book is overdue. Today is day 20; loan L7 has due day 17. Grace period is zero.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Required data	**Loan L7; due_day=17; today=20**	Retain the dates for the overdue rule.
2 / Condition	**today > due_day**	Define what counts as late.
3 / Calculation	**20 - 17 = 3 days**	Use integer day values in this simplified example.
4 / Output	**L7: overdue by 3 days**	Expose information needed by the user.
5 / Boundary	**today = due_day => not overdue**	Test the comparison at the boundary.
6 / Limitation	**Calendar/holidays ignored**	The simplified day numbers need real-date rules in production.

TRANSFER IT TO AN EXAM ANSWER

The solution should also test returned, renewed and missing loans. A model that ignores relevant exceptions may be misleading.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.1(c,d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Explain one difference between a real-world loan and its computer model. Give two fields needed for an overdue reminder.

Q2 / 5 MARKS

Design the data and the condition needed to send reminders for an overdue item. Include a boundary test.

Q3 / 6 MARKS

The school adds a rule: no reminders during closure days. Evaluate the current abstract model and explain how you would improve and test it.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.1(c,d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

A model includes selected aspects rather than every real-world detail [1]; due date [1] and borrower identifier/contact or return status [1].

Q2 / 5

Retain the loan and borrower identifiers [1], due date [1], today/current date [1]; send only if today > due date and not returned [1]; test today = due date, expecting no reminder [1].

Q3 / 6

The original model cannot distinguish closure days [1]. Include calendar/closure information [1]. Define how the policy adjusts due dates or reminder timing [1]. Apply the rule consistently [1]. Test a due date on a closure day and a normal day [1]. Validate actual notifications against school policy [1].

COMMON MISCONCEPTION

A model that works for an ordinary case is not automatically suitable for every real-world case.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.2(a,b) / SEE IT / CONCEPT AND VISUAL

Before you code, define what must be true

Thinking ahead identifies inputs, outputs, prior conditions and likely consequences before implementation.

INPUTS

Values supplied: user, event, quantity.

PRECONDITIONS

Conditions required for the operation to succeed.

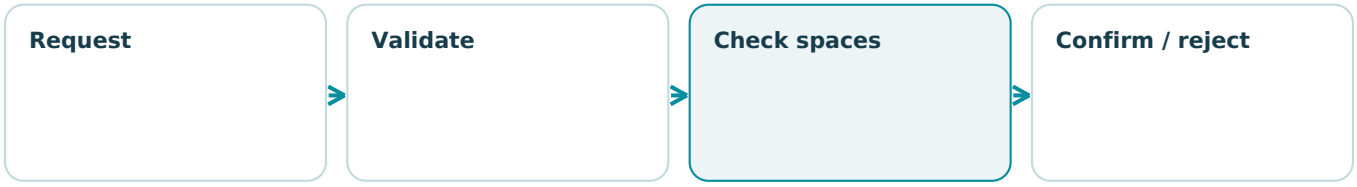
PROCESS

Validation, checking capacity and updating state.

OUTPUTS

A confirmed booking or an explained rejection.

A BOOKING FUNCTION: INPUT TO OUTPUT



FROM THE DIAGRAM TO EXAM LANGUAGE

A request for two seats is not sufficient on its own. It must have a valid event, a non-negative quantity and enough remaining capacity. Only a successful request updates seats.

WORDS THAT EARN MARKS

Input	Supplied value	**Output**	Produced result
Precondition	Must hold first	**Postcondition**	Must hold after

EXAM TRAP

A precondition describes what must hold before an operation; an output describes the result, not a prerequisite.

2.1.2(a,b) / WORKED EXAMPLE / STEP BY STEP

Specify a booking operation before writing code

A workshop has capacity 24, with 19 places already booked. One valid user requests 4 places.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Inputs	**capacity 24, booked 19, request 4**	Record the relevant state and request.
2 / Preconditions	**request > 0, user/event valid**	Do not assume valid data.
3 / Available	**24 - 19 = 5**	Compute the remaining capacity.
4 / Branch	**4 <= 5 : true**	The request fits.
5 / Update	**booked = 19 + 4 = 23**	Only update after passing all checks.
6 / Output	**Confirmation; remaining = 1**	Return a meaningful success result.

TRANSFER IT TO AN EXAM ANSWER

For a request of 6 places, reject and keep booked at 19; a failed operation must not modify the booking count.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.2(a,b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Identify two inputs to a seat-booking system and one appropriate output.

Q2 / 5 MARKS

State suitable preconditions for accepting a request. Explain the result for 19 booked places, capacity 24 and request 6.

Q3 / 6 MARKS

Design and explain a booking algorithm that never exceeds capacity, including what happens when an input fails validation.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.2(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Event ID [1]; requested number of seats [1]; output could be confirmation or a reason for rejection [1].

Q2 / 5

Event exists [1]; request is positive integer [1]; valid booking/user or suitable other precondition [1]; available is 24-19 = 5 [1]; request 6 is rejected with no state change [1].

Q3 / 6

Validate supplied fields [1]; calculate available capacity [1]; check request <= available [1]; if true update bookings and confirm [1]; otherwise return rejection with no update [1]; protect the check/update operation against competing requests [1].

COMMON MISCONCEPTION

A precondition describes what must hold before an operation; an output describes the result, not a prerequisite.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.2(c) / SEE IT / CONCEPT AND VISUAL

A cached answer can become the wrong answer

A cache stores a copy of data or a result for reuse. Reduced latency is valuable only if stale data is handled.

CACHE HIT

Reusable copy found: answer may be quicker.

CACHE MISS

Retrieve or compute the value, then store it.

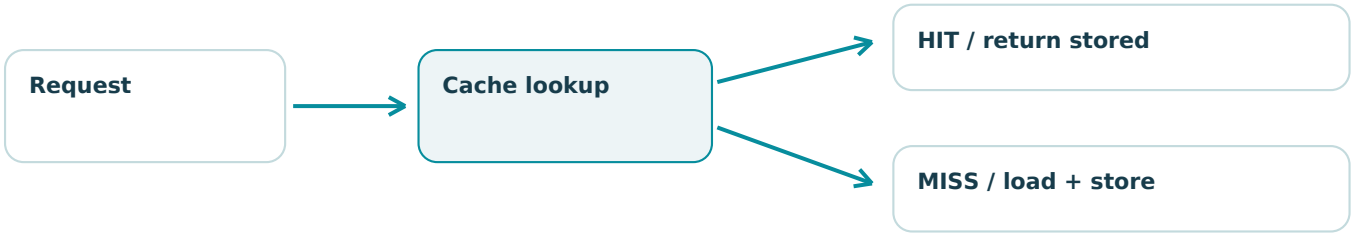
BENEFIT

Less repeated work and lower response time.

COST

Storage, invalidation, complexity and stale values.

A SIMPLE READ-THROUGH CACHE



FROM THE DIAGRAM TO EXAM LANGUAGE

Caching results of an expensive timetable search can save computation. If room allocations change, a cache containing the old timetable must expire or be invalidated.

WORDS THAT EARN MARKS

- **Cache** Stored copy
- **Hit** Value present
- **Miss** Value absent
- **Invalidation** Discard stale copy

EXAM TRAP

A cache is not always faster overall, and a cache hit does not guarantee that the answer is current.

2.1.2(c) / WORKED EXAMPLE / STEP BY STEP

Cache an expensive timetable lookup

A room timetable is requested three times. Uncached lookup takes 120 ms; a cache hit takes 8 ms. Ignore cache-management overhead for this exercise.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Request 1	**Miss => database 120 ms**	Nothing is cached yet.
2 / Save	**Cache key room:R4**	Associate a key with the returned timetable.
3 / Request 2	**Hit => 8 ms**	Return a stored copy.
4 / Request 3	**Hit => 8 ms**	Reuse the stored result again.
5 / Total	**120 + 8 + 8 = 136 ms**	Three uncached reads would total 360 ms.
6 / Update	**Room change => invalidate R4**	Old answers must no longer be reused.

TRANSFER IT TO AN EXAM ANSWER

Saving = 224 ms for these three reads under the stated timings. The real gain depends on cache hit rate and update frequency.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.2(c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define caching, cache hit and cache miss.

Q2 / 5 MARKS

Calculate the time saved by the three requests above and explain why the result is conditional on the assumptions.

Q3 / 6 MARKS

Evaluate using a cache for live exam-room changes. Discuss responsiveness, invalidation and consequences of stale data.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.2(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Caching stores a reusable copy [1]; hit means the requested value is present [1]; miss means it is not present and must be obtained/computed [1].

Q2 / 5

Uncached 3 x 120 = 360 ms [1]; cached 120 + 8 + 8 = 136 ms [1]; saving 224 ms [1]; assumes the subsequent requests hit [1]; excludes changes, overhead and other latency [1].

Q3 / 6

A hit provides faster access [1] and reduces repeat queries [1], but copies can become stale [1]. Changes require invalidation or expiration [1]. Incorrect room assignments can misdirect students [1]. Use a suitable freshness policy or fetch authoritative data when essential [1].

COMMON MISCONCEPTION

A cache is not always faster overall, and a cache hit does not guarantee that the answer is current.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.2(d) / SEE IT / CONCEPT AND VISUAL

Reuse a component, not a copied mistake

A reusable procedure or function has a defined purpose, inputs, outputs and behaviour that can be tested independently.

ONE PURPOSE

Keep the function focused and cohesive.

INTERFACE

Clear parameters, returned value and assumptions.

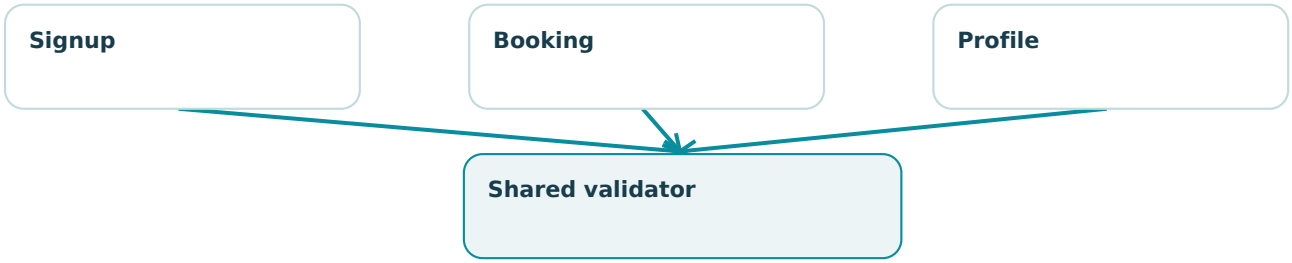
REUSE

Call the same validated component in several features.

TEST

Boundary and invalid-input tests protect every caller.

ONE VALIDATOR USED BY THREE SCREENS



FROM THE DIAGRAM TO EXAM LANGUAGE

Sign-up, booking and profile screens can all call a shared email validator. A code change then affects every caller, so its contract and tests matter.

WORDS THAT EARN MARKS

Function	Reusable unit	**Parameter**	Passed input
Return value	Produced result	**Contract**	Expected behaviour

EXAM TRAP

Reuse is not copying a block into several places; copied blocks drift apart and require multiple fixes.

2.1.2(d) / WORKED EXAMPLE / STEP BY STEP

Create an independently testable quantity validator

A function validQuantity(q, maxSeats) returns true only when q is a positive integer no greater than maxSeats.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Contract	validQuantity(q, maxSeats) -> Boolean	Specify parameters and output.
2 / Basic	q is an integer	Reject text, decimals and booleans as quantities.
3 / Lower bound	q >= 1	Reject zero and negative numbers.
4 / Upper bound	q <= maxSeats	Reject requests above the permitted maximum.
5 / Test	q=4,max=5 => true	Normal valid case.
6 / Boundaries	q=0 => false; q=5 => true	Confirm lower and upper edges.

TRANSFER IT TO AN EXAM ANSWER

The validator does not itself reserve seats or check live availability. Other components can call it before their own state updates.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.2(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State three benefits or design features of a reusable program component.

Q2 / 5 MARKS

Write pseudocode for validQuantity(q, maximum) using an integer check and inclusive range 1 to maximum.

Q3 / 6 MARKS

Explain why shared validation requires a clear interface and give a suitable set of normal, boundary and invalid tests.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.2(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Any three: one maintained implementation [1]; reduced duplication [1]; consistent behaviour [1]; defined parameters/interface [1]; independent testing [1].

Q2 / 5

Function accepts two parameters [1]; rejects non-integer q [1]; checks q >= 1 [1]; checks q <= maximum [1]; returns Boolean without modifying bookings [1].

Q3 / 6

The interface defines inputs and output [1]; callers rely on consistent rules [1]. Normal q=3,max=5 => true [1]; lower boundary q=1 => true [1]; invalid q=0 or 6 => false [1]; non-integer/text input => false [1].

COMMON MISCONCEPTION

Reuse is not copying a block into several places; copied blocks drift apart and require multiple fixes.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.3(a-d) / SEE IT / CONCEPT AND VISUAL

Break a problem into steps that can run

Procedural thinking identifies components, orders their operations and extracts useful sub-procedures.

DECOMPOSE

Identify manageable subproblems and data.

SEQUENCE

Decide which steps depend on earlier work.

SUB-PROCEDURE

Give repeated or cohesive operations a name.

INTERFACE

Pass required values between procedures.

A LIBRARY RETURN IS A SEQUENCE



FROM THE DIAGRAM TO EXAM LANGUAGE

The database lookup must happen before the update. Printing a receipt should use the confirmed updated state, not an assumed success.

WORDS THAT EARN MARKS

Decomposition	Smaller parts	**Sequence**	Ordered steps
Procedure	Named actions	**Dependency**	Needs earlier result

EXAM TRAP

Decomposition is not the same as simply listing code in order; the components must have meaningful responsibilities.

2.1.3(a-d) / WORKED EXAMPLE / STEP BY STEP

Design a book-return procedure

An assistant scans book B42; an active loan exists. The system must close the loan and provide a receipt.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Capture	**bookId <- scan()**	Obtain the identity of the book.
2 / Validate	**findActiveLoan(bookId)**	Get the current loan record.
3 / Branch	**if loan not found => explain error**	Stop safely if there is no active loan.
4 / Update	**closeLoan(loanId)**	Commit the return before confirming it.
5 / Output	**makeReceipt(loanId)**	Show user the confirmed result.
6 / Reuse	**lookup(), close(), receipt()**	Use named parts with clear responsibilities.

TRANSFER IT TO AN EXAM ANSWER

If update fails, do not issue a successful receipt. A success message belongs after the confirmed state change.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.3(a-d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define decomposition, sequencing and a sub-procedure in the context of a book-return system.

Q2 / 5 MARKS

List five suitably ordered steps to process a return, including one validation and one failure outcome.

Q3 / 6 MARKS

Outline a modular solution to this problem, explaining data passed between components and two important dependencies.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.3(a-d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Decomposition divides return processing into smaller tasks [1]; sequencing puts operations in a necessary order [1]; a sub-procedure is a named part such as findActiveLoan() [1].

Q2 / 5

Read/scanned book ID [1]; validate/find active loan [1]; reject if none [1]; update the record [1]; issue receipt only after confirmed success [1].

Q3 / 6

Identify scan or capture component [1]; loan lookup component [1]; update component [1]; receipt/output component [1]; pass book ID or loan ID between components [1]; lookup before update and successful update before receipt [1].

COMMON MISCONCEPTION

Decomposition is not the same as simply listing code in order; the components must have meaningful responsibilities.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.4(a-c) / SEE IT / CONCEPT AND VISUAL

Every decision creates a path through the program

Logical thinking identifies choice points, expresses conditions precisely and traces their consequences.

DECISION POINT

Where a choice changes what happens next.

CONDITION

An expression that evaluates to true or false.

BRANCH

A path taken when the condition matches.

TEST

Use boundary and contrasting values to test paths.

A BOOLEAN DECISION THAT CAN BE TRACED



FROM THE DIAGRAM TO EXAM LANGUAGE

For a restricted workshop, admission requires age >= 16 AND validID = true. Being 16 does not override a failed identity check; both conditions matter.

WORDS THAT EARN MARKS

- **Boolean** True or false
- **AND** Both true
- **OR** Either true
- **Branch** Selected path

EXAM TRAP

An inclusive condition >= 16 is different from > 16. Tests should include the exact boundary value.

2.1.4(a-c) / WORKED EXAMPLE / STEP BY STEP

Construct and trace a compound condition

Entry rule: age >= 16 AND validID. Trace all four cases below with an explicit result.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Condition	**eligible = (age >= 16) AND validID**	Both requirements must hold.
2 / Case A	**age 15, ID yes => false**	Age fails.
3 / Case B	**age 16, ID no => false**	ID fails at the age boundary.
4 / Case C	**age 16, ID yes => true**	Both conditions are satisfied.
5 / Case D	**age 19, ID yes => true**	A value above the boundary is valid.
6 / Action	**if eligible then admit else reject**	Only one path executes.

TRANSFER IT TO AN EXAM ANSWER

Parentheses make the intended logic clear. Change AND to OR and the rule would admit an under-age person with ID, which is incorrect.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.4(a-c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State the purpose of a condition, a Boolean operator and a branch.

Q2 / 5 MARKS

Complete the admission rule in pseudocode and trace age=16/validID=false and age=17/validID=true.

Q3 / 6 MARKS

Evaluate a flawed program that uses OR instead of AND. Show two counterexamples and explain a test strategy.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.4(a-c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Condition evaluates to true/false [1]; Boolean operator combines or negates conditions [1]; branch chooses subsequent execution path [1].

Q2 / 5

eligible <- (age >= 16) AND validID [1]; if eligible then admit [1] else reject [1]; (16,false) rejects [1]; (17,true) admits [1].

Q3 / 6

OR allows entry when only one condition holds [1]. Age 15 with validID=true is incorrectly admitted [1]. Age 18 with validID=false is also incorrectly admitted [1]. AND requires both [1]. Test at age 15/16 [1] with both ID states to cover all logical branches [1].

COMMON MISCONCEPTION

An inclusive condition >= 16 is different from > 16. Tests should include the exact boundary value.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.5(a) / SEE IT / CONCEPT AND VISUAL

Parallel work needs independent tasks

Thinking concurrently identifies tasks that may proceed together, while respecting dependencies between results.

INDEPENDENT

Can begin without needing the other result.

DEPENDENT

Must wait until prerequisite work is complete.

JOIN

Wait for all needed results before the next step.

PLAN

A dependency diagram reveals available parallelism.

A SIMPLE DEPENDENCY MAP



FROM THE DIAGRAM TO EXAM LANGUAGE

After a batch begins, file A and file B can be parsed concurrently. Comparing their results must wait for both parses to complete.

WORDS THAT EARN MARKS

- **Concurrency** — Overlapping progress
- **Dependency** — Must come first
- **Parallelism** — Simultaneous execution
- **Join** — Wait for results

EXAM TRAP

Concurrency is not a promise that every line runs simultaneously; dependent tasks still require ordering.

2.1.5(a) / WORKED EXAMPLE / STEP BY STEP

Calculate the earliest finish under two workers

Task Load=2 s. After Load, Parse A=5 s and Parse B=4 s may run together. Compare=2 s needs both parses.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Start	**Load: time 0-2 s**	All later tasks require loaded inputs.
2 / Branch A	**Parse A: time 2-7 s**	Use first worker.
3 / Branch B	**Parse B: time 2-6 s**	Use second worker concurrently.
4 / Join	**Wait until time 7 s**	Compare requires both completed parses.
5 / Finish	**Compare: time 7-9 s**	Final dependent task.
6 / Contrast	**Sequential: 2+5+4+2 = 13 s**	Concurrent elapsed time is 9 s here.

TRANSFER IT TO AN EXAM ANSWER

Speed-up in this idealised case is 13/9 approximately 1.44x, not 2x: the initial load, final comparison and unequal branch times limit gains.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.5(a) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish concurrency, parallelism and a dependency.

Q2 / 5 MARKS

Draw or describe a dependency plan for Load -> (Parse A, Parse B) -> Compare. Calculate elapsed time with two workers.

Q3 / 6 MARKS

Explain why adding more workers does not make this workflow four times faster. Refer to dependent work and communication overhead.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.5(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Concurrency means tasks make overlapping progress [1]; parallelism means simultaneous execution [1]; dependency means a task needs an earlier result [1].

Q2 / 5

Load is first [1]; A and B can start after Load [1]; comparison waits for both [1]; parallel branch takes max(5,4)=5 s [1]; total 2+5+2=9 s [1].

Q3 / 6

The 2 s load is serial [1]; both parses cannot start earlier [1]; only two parse tasks offer parallel work [1]; compare must await both [1]; synchronisation/communication adds potential overhead [1]; therefore extra workers have diminishing or no benefit for this graph [1].

COMMON MISCONCEPTION

Concurrency is not a promise that every line runs simultaneously; dependent tasks still require ordering.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.5(b) / SEE IT / CONCEPT AND VISUAL

A faster booking service can still be wrong

Concurrency can improve responsiveness, but shared mutable data creates risks such as race conditions.

BENEFIT

Handle independent requests while another waits.

RACE

The result depends on unpredictable timing.

PROTECT

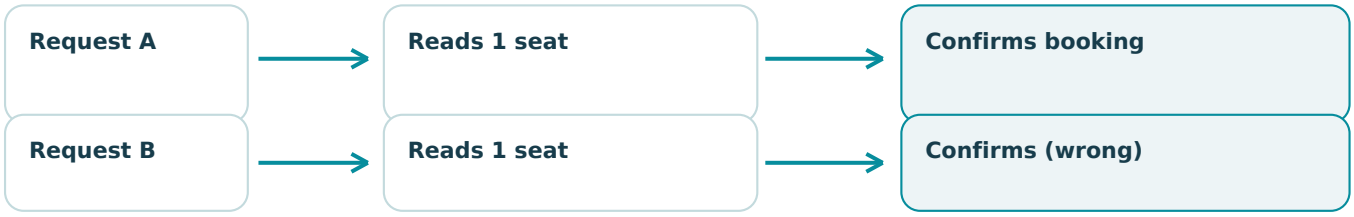
Use atomic operations, locks or transactions.

TRADE-OFF

Synchronisation adds overhead and may reduce throughput.

TWO REQUESTS READING THE SAME LAST SEAT

SHARED STATE: 1 SEAT LEFT



FROM THE DIAGRAM TO EXAM LANGUAGE

If A and B both read availability=1 before either updates it, naïve code might confirm both. The capacity check and decrement must be one protected operation.

WORDS THAT EARN MARKS

- **Race condition** Timing bug
- **Atomic** Indivisible step
- **Lock** Exclusive access
- **Throughput** Work per time

EXAM TRAP

A precondition check is not enough if shared state can change between checking and updating.

2.1.5(b) / WORKED EXAMPLE / STEP BY STEP

Find and repair an overselling race

One seat remains. Requests A and B execute check-then-decrement without a lock.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Shared state	**available = 1**	Both requests share the same count.
2 / A reads	**A observes available = 1**	A decides one place can be sold.
3 / B reads	**B also observes available = 1**	B reaches same decision before A updates.
4 / A commits	**A confirms; available becomes 0**	The last seat is sold.
5 / B commits	**B also confirms; logical result oversold**	Checking earlier was insufficient.
6 / Repair	**Atomic check + decrement**	Second request rechecks protected state and fails.

TRANSFER IT TO AN EXAM ANSWER

A database transaction or suitable lock can keep the test and update together. The serialised critical section can reduce contention-related throughput.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.5(b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Identify a race condition and state one benefit and one drawback of concurrent request handling.

Q2 / 5 MARKS

Explain, step by step, how two simultaneous requests can both book the last seat and describe one prevention method.

Q3 / 6 MARKS

Evaluate concurrency for an online booking system, balancing responsiveness, consistency, locking and testing.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.5(b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Race condition: result depends on interleaving of shared-state operations [1]; benefit: responsiveness/throughput [1]; drawback: race complexity or synchronisation overhead [1].

Q2 / 5

Seat count initially 1 [1]; A reads 1 [1]; B reads 1 before update [1]; both confirm, causing oversell [1]; use atomic check/update or lock/transaction [1].

Q3 / 6

Concurrent requests improve responsiveness [1] and use server resources [1], but shared availability may be inconsistent [1]. Protect the critical section [1]. Locking/transactions incur contention or overhead [1]. Test overlapping requests and check invariant confirmed <= capacity [1].

COMMON MISCONCEPTION

A precondition check is not enough if shared state can change between checking and updating.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

MASTER CONTENTS

Choose a learning cycle

Each topic has one explanation page, a worked trace, original exam-style practice and a separate self-marking page. Click any row to jump there.

2.2.1(a)	Sequence And Selection	pp. 8-11
2.2.1(a)	Iteration And Loop Bounds	pp. 12-15
2.2.1(b)	Recursion And The Call Stack	pp. 16-19
2.2.1(c)	Local And Global Scope	pp. 20-23
2.2.1(d)	Modularity And Subroutines	pp. 24-27
2.2.1(d)	Value Vs Reference	pp. 28-31
2.2.1(e)	Ide And Debugging	pp. 32-35
2.2.1(f)	Object-Oriented Design	pp. 36-39
2.2.2(a,b)	Recognise A Solvable Problem	pp. 40-43

NAVIGATION

Use the bookmark panel or linked rows. Separate questions and answers support independent study. Section 2.2.1 precedes 2.2.2.

MASTER CONTENTS

Choose a learning cycle

Each topic has one explanation page, a worked trace, original exam-style practice and a separate self-marking page. Click any row to jump there.

2.2.2(c)	**Problem Decomposition**	pp. 44-47
2.2.2(d)	**Divide And Conquer**	pp. 48-51
2.2.2(e)	**Abstraction In Design**	pp. 52-55
2.2.2(f)	**Backtracking**	pp. 56-59
2.2.2(f)	**Data Mining**	pp. 60-63
2.2.2(f)	**Heuristics**	pp. 64-67
2.2.2(f)	**Performance Modelling**	pp. 68-71
2.2.2(f)	**Pipelining**	pp. 72-75
2.2.2(f)	**Visualisation**	pp. 76-79

NAVIGATION

Use the bookmark panel or linked rows. Separate questions and answers support independent study. Section 2.2.1 precedes 2.2.2.

INDEPENDENT REVISION

How to use each four-page cycle

01

UNDERSTAND

Read the explanation, inspect the original diagram, and recall the highlighted key terms.

02

MODEL

Follow each change of value, branch or state; explain why the step is necessary.

03

TRY

Attempt all questions independently in black pen before consulting answers.

04

CHECK

Self-mark using indicative points; correct a weakness in green pen and explain the fix.

These are original RHSC practice questions, not OCR past-paper questions.

2.2 / PROBLEM SOLVING AND PROGRAMMING

Understand it. Model it. Try it. Check it.

Programming techniques and computational methods work together. Choose a clear model, trace its state, justify each algorithmic decision and evaluate constraints.

2.2.1 Programming techniques

Sequence, selection, iteration; recursion; scope; modularity, parameter passing; IDE; OOP

2.2.2 Computational methods

Computability, recognition, decomposition, divide-and-conquer, abstraction, six problem-solving methods

18 learning cycles: eight in 2.2.1 and ten in 2.2.2. Use the topic checklists to plan revision.

2.2 / SPECIFICATION CHECKLIST

Check every OCR requirement

Check each cycle only when you can apply it to an unfamiliar scenario. The OCR requirements are summarised, not reproduced verbatim.

SPEC		PAGES
2.2.1(a)	Sequence, iteration and branching; trace loops	8-11
2.2.1(a)	Guard and termination; FOR, WHILE, REPEAT	12-15
2.2.1(b)	Recursion, base cases and iterative comparisons	16-19
2.2.1(c)	Local and global variables; scope	20-23
2.2.1(d)	Modularity, functions and procedures	24-27
2.2.1(d)	Passing parameters by value and reference	28-31
2.2.1(e)	Use an IDE to develop and debug	32-35
2.2.1(f)	Object-oriented techniques	36-39
2.2.2(a,b)	Solvable features; problem recognition	40-43

OFFICIAL SOURCE

Cambridge OCR H446 A Level Computer Science, specification v3.0 (April 2026), section 2.2.1 and 2.2.2.
Use these checklists to choose a topic, then attempt its exam-style questions.

2.2 / SPECIFICATION CHECKLIST

Check every OCR requirement

Check each cycle only when you can apply it to an unfamiliar scenario. The OCR requirements are summarised, not reproduced verbatim.

SPEC		PAGES
2.2.2(c)	Problem decomposition	44-47
2.2.2(d)	Divide and conquer	48-51
2.2.2(e)	Abstraction applied to problems	52-55
2.2.2(f)	Backtracking	56-59
2.2.2(f)	Data mining	60-63
2.2.2(f)	Heuristics	64-67
2.2.2(f)	Performance modelling	68-71
2.2.2(f)	Pipelining	72-75
2.2.2(f)	Visualisation	76-79

OFFICIAL SOURCE

Cambridge OCR H446 A Level Computer Science, specification v3.0 (April 2026), section 2.2.1 and 2.2.2.
Use these checklists to choose a topic, then attempt its exam-style questions.

2.2.1(a) / SEE IT / CONCEPT AND VISUAL

Choose an action from a testable condition

Sequence executes instructions in order. Branching selects a path after a Boolean test; both are fundamental program constructs.

SEQUENCE

One statement follows another.

IF

Execute only when a condition is true.

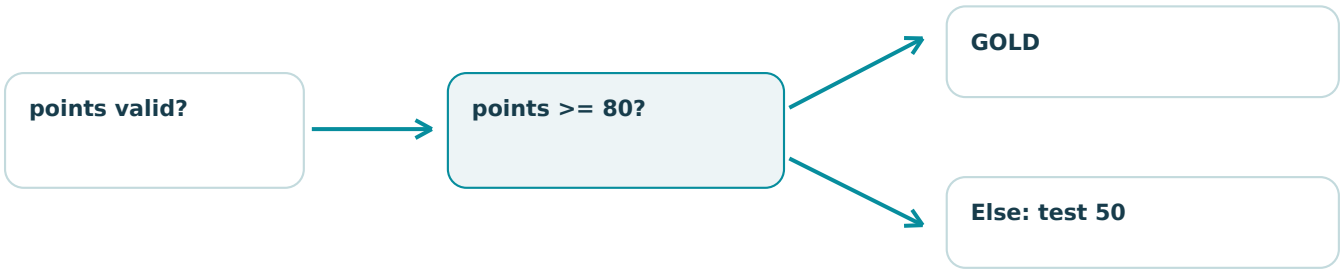
ELSE

Alternative path for a false condition.

TRACE

State exactly which statements execute.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Guard validation before classification. ELSE is tied to the preceding IF, and comparison uses a Boolean result, not an assignment.

WORDS THAT EARN MARKS

Sequence	Ordered steps	**Selection**	Choose a branch
Boolean	True or false	**Condition**	Logical test

EXAM TRAP

ELSE IF conditions must be ordered carefully; changing the first threshold can change all later outcomes.

2.2.1(a) / WORKED EXAMPLE / STEP BY STEP

Trace a three-way classification with boundaries

A booking scheme awards GOLD for points >= 80, SILVER for points >= 50, otherwise BRONZE. Points must be between 0 and 100.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Read	**points = 80**	Start with the value to classify.
2 / Validate	**0 <= 80 <= 100: true**	Only classify an allowed value.
3 / First test	**80 >= 80: true**	Enter GOLD branch.
4 / Skip	**ELSE IF points >= 50 skipped**	Only one branch of this chain is taken.
5 / Assign	**award = GOLD**	Store selected result.
6 / Output	**Display GOLD**	The boundary 80 belongs to GOLD.

TRANSFER IT TO AN EXAM ANSWER

For points = 49 the outcome is BRONZE; for 50 it is SILVER. If points = 120, reject as invalid before allocating an award.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(a) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define sequence, selection and Boolean condition.

Q2 / 5 MARKS

Trace the award for points 49, 50, 79 and 80. Explain why the tests must be ordered.

Q3 / 6 MARKS

Write pseudocode for this scheme, including rejection outside 0-100 and no award for invalid input.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Sequence means ordered execution [1]; selection chooses a path [1]; a condition evaluates TRUE/FALSE [1].

Q2 / 5

49 BRONZE [1], 50 SILVER [1], 79 SILVER [1], 80 GOLD [1]; test GOLD first so high values are not taken by the broad SILVER threshold [1].

Q3 / 6

Read points [1]; IF outside 0..100 reject [1]; ELSE IF >=80 choose GOLD [1]; ELSE IF >=50 choose SILVER [1]; ELSE choose BRONZE [1]; display outcome only for valid input [1].

COMMON MISCONCEPTION

ELSE IF conditions must be ordered carefully; changing the first threshold can change all later outcomes.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(a) / SEE IT / CONCEPT AND VISUAL

Use the right loop and protect termination

Iteration repeats a block. Count-controlled loops have known repeat counts; condition-controlled loops need a changing state and a termination condition.

FOR

Count-controlled repetition.

WHILE

Pre-condition: may run zero times.

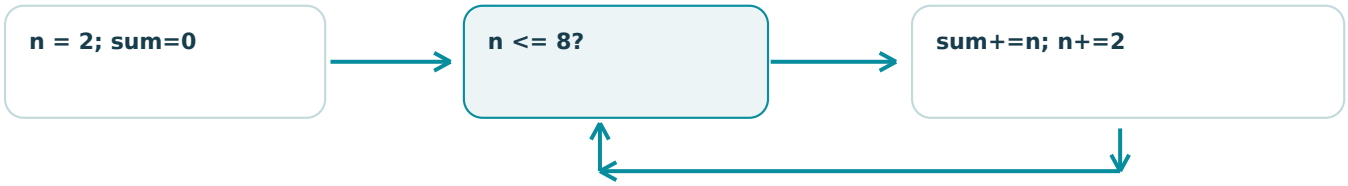
REPEAT

Post-condition: executes at least once.

INVARIANT

A fact that stays true each pass.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

In a WHILE loop, test before executing. A counter or other changing value must eventually make the loop condition false.

WORDS THAT EARN MARKS

- **Iteration** Repetition
- **Termination** Eventually stops
- **Loop guard** Continue condition
- **Accumulator** Running total

EXAM TRAP

A loop is not correct just because its body is correct: the guard, update and initial state must all be valid.

2.2.1(a) / WORKED EXAMPLE / STEP BY STEP

Trace an accumulator, including the final failed test

Sum 2 + 4 + 6 + 8 with n initially 2, total initially 0, and WHILE n <= 8. Update total then add 2 to n.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Setup	**n=2; total=0**	No values accumulated yet.
2 / Pass 1	**total=2; n=4**	Add the first even integer.
3 / Pass 2	**total=6; n=6**	Add 4 to the accumulator.
4 / Pass 3	**total=12; n=8**	Add 6 to the accumulator.
5 / Pass 4	**total=20; n=10**	Add 8 then advance beyond the bound.
6 / Exit	**10 <= 8 is false; output 20**	The next guard test stops iteration.

TRANSFER IT TO AN EXAM ANSWER

For upper bound 1, the WHILE body executes zero times and total remains 0. A REPEAT ... UNTIL loop would run once before its condition is checked.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(a) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish FOR, WHILE and REPEAT loops.

Q2 / 5 MARKS

Show n and total after each pass for n=2,4,6,8. Explain the exit condition.

Q3 / 6 MARKS

Design a condition-controlled algorithm that sums positive inputs until a sentinel -1; reject other negative numbers and do not include the sentinel.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

FOR has a counted range [1]; WHILE tests first [1]; REPEAT tests after and runs at least once [1].

Q2 / 5

After passes: (4,2) [1], (6,6) [1], (8,12) [1], (10,20) [1]; condition n <= 8 then becomes false [1].

Q3 / 6

Start total=0 [1]; read input [1]; loop while input != -1 [1]; add only when input >= 0 [1]; reject any negative other than -1 [1]; read again and output final total [1].

COMMON MISCONCEPTION

A loop is not correct just because its body is correct: the guard, update and initial state must all be valid.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(b) / SEE IT / CONCEPT AND VISUAL

Base case first; shrinking problem second

Recursion is a procedure or function calling itself on a smaller instance. A valid base case prevents indefinite self-calls.

BASE CASE

An answer without another call.

RECURSIVE CASE

Invoke the same function on a smaller problem.

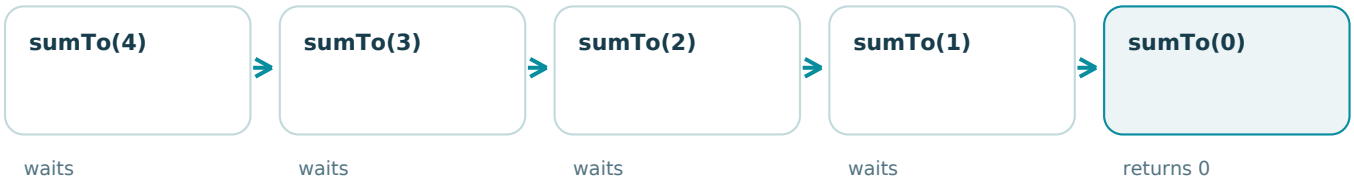
STACK FRAME

Keeps each call's local state.

RETURN

Results unwind to the caller.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Each call waits for its inner call to return. Show the downward calls and then the upward returns; do not add numbers until the return stage.

WORDS THAT EARN MARKS

Recursion	Self-call	**Base case**	Stops calls
Call stack	Pending returns	**Stack overflow**	Too much depth

EXAM TRAP

Recursive calls do not automatically return in the same order they are made. Show both the call and return sequence.

2.2.1(b) / WORKED EXAMPLE / STEP BY STEP

Trace sumTo(4) with explicit return values

Function sumTo(n) returns 0 if n=0; otherwise it returns n + sumTo(n-1). All inputs in this model are non-negative integers.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Call	**sumTo(4) = 4 + sumTo(3)**	First return waits for next call.
2 / Call	**sumTo(3) = 3 + sumTo(2)**	Depth increases.
3 / Call	**sumTo(2) = 2 + sumTo(1)**	Another pending addition.
4 / Call	**sumTo(1) = 1 + sumTo(0)**	Approach the base case.
5 / Base	**sumTo(0) returns 0**	No further recursive call.
6 / Unwind	**1 -> 3 -> 6 -> 10**	Returns of sumTo(1..4) in order.

TRANSFER IT TO AN EXAM ANSWER

Iterative comparison: total=0; FOR k=1 TO 4, total=total+k. Both return 10; recursion keeps pending stack frames, while this loop uses constant extra space. Guard negative inputs.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State the role of a base case and a stack frame. Give one recursion risk.

Q2 / 5 MARKS

Trace the call order and return values for sumTo(3), including sumTo(0).

Q3 / 6 MARKS

Write recursive pseudocode for factorial(n), n >= 0; explain its base case and termination, and compare its extra memory use with an iterative factorial.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Base case returns directly [1]; frame stores the pending caller state [1]; excessive recursion can overflow the stack [1].

Q2 / 5

Call sumTo(3), (2), (1), (0) [2]; returns 0, 1, 3, 6 [2]; every call decreases n [1].

Q3 / 6

Reject n<0 [1]; return 1 when n=0 [1]; otherwise return n * factorial(n-1) [1]; recursive input decreases [1] until the base case [1]; recursive stack depth grows with n, unlike an iterative accumulator with constant extra space [1].

COMMON MISCONCEPTION

Recursive calls do not automatically return in the same order they are made. Show both the call and return sequence.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(c) / SEE IT / CONCEPT AND VISUAL

Know which variable changes and which survives

Local variables belong to their procedure or function invocation. Global variables are available in a wider scope and can create unwanted coupling.

LOCAL

Declared inside a routine or block.

GLOBAL

Declared in wider program scope.

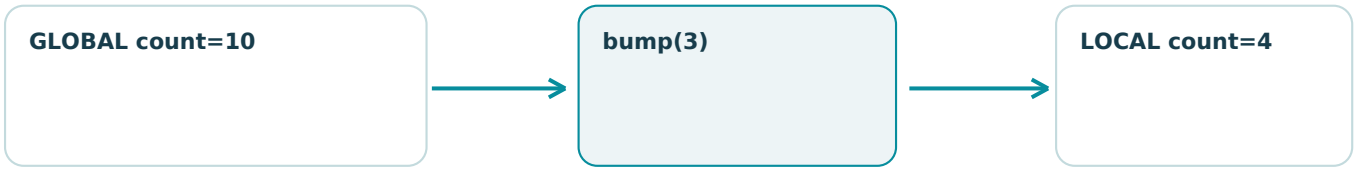
SHADOW

A local name can hide a global name.

SIDE EFFECT

A call may change shared state.

HOW THE PARTS FIT TOGETHER



Return 4 does not change the global count unless caller assigns it.

FROM THE DIAGRAM TO EXAM LANGUAGE

A local variable with the same name is not automatically the global. Separate each variable's lifetime and scope in a trace.

WORDS THAT EARN MARKS

- **Scope** Where name is valid
- **Local** Routine-owned value
- **Global** Shared program value
- **Shadowing** Name hides another

EXAM TRAP

Names that are spelled identically do not imply the variables share the same storage location.

2.2.1(c) / WORKED EXAMPLE / STEP BY STEP

Track two different variables called count

Global count starts at 10. Function bump(count) receives value 3 in its local parameter, increases it by 1, then returns it. The caller does not assign the result to global count.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Global	**count = 10**	Program-level value exists.
2 / Call	**bump(3)**	Create a new local parameter.
3 / Local	**count = 3**	The parameter hides the global name in function body.
4 / Update	**local count = 4**	Only the local parameter is incremented.
5 / Return	**function returns 4**	No global assignment occurs.
6 / End	**global count still 10**	Local parameter ceases to exist.

TRANSFER IT TO AN EXAM ANSWER

If the caller then executes count = bump(3), the global becomes 4 because the result is explicitly assigned.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define local scope, global scope and shadowing.

Q2 / 5 MARKS

Trace the local return and global value after calling bump(3). Explain why they differ.

Q3 / 6 MARKS

Evaluate two possible designs for a score-tracking application: routines modify a global score, or functions return a new score. Justify which is easier to test.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Local: visible within a routine [1]; global: wider program visibility [1]; shadowing: local name masks wider one [1].

Q2 / 5

Parameter count starts 3 [1]; becomes 4 [1]; function returns 4 [1]; global remains 10 [1]; no global write takes place [1].

Q3 / 6

Global sharing can reduce parameter passing [1] but causes hidden dependencies [1] and unpredictable side effects [1]. Returning a new score makes inputs/outputs explicit [1], simplifies isolated tests [1], though values must be passed and assigned [1].

COMMON MISCONCEPTION

Names that are spelled identically do not imply the variables share the same storage location.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(d) / SEE IT / CONCEPT AND VISUAL

Break a program into reusable, testable routines

Modularity separates a solution into coherent components. Functions return values; procedures perform an action, possibly changing program state.

FUNCTION

Returns a computed value.

PROCEDURE

Performs an action without needing a return value.

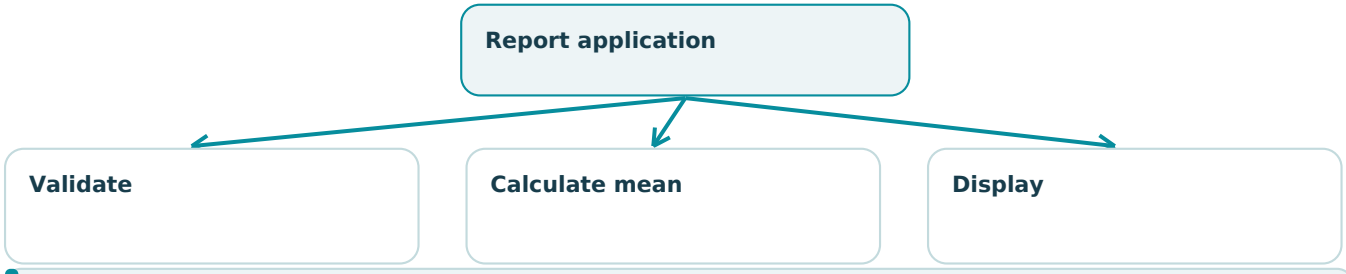
PARAMETER

Named input to a routine.

COHESION

Each routine has a clear job.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Use distinct routine contracts to make testing precise. A validation function can be reused whenever a mark enters the system.

WORDS THAT EARN MARKS

Modularity	Split into routines	**Function**	Returns a value
Procedure	Performs an action	**Parameter**	Passed information

EXAM TRAP

A procedure is not defined by having no parameters; both procedures and functions may accept parameters.

2.2.1(d) / WORKED EXAMPLE / STEP BY STEP

Decompose a mark-reporting program into contracts

A mark sheet contains 60, 70, 80. A report must reject out-of-range marks, calculate the mean and display it.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Receive	**marks = [60,70,80]**	Main program obtains input.
2 / Validate	**validMarks(marks) => true**	Each mark is checked in 0..100.
3 / Sum	**sumMarks = 210**	Calculation is in one routine.
4 / Average	**average = 210 / 3 = 70**	Return a numeric result.
5 / Present	**showAverage(70)**	Procedure displays output.
6 / Test	**[60,120,80] => invalid**	Do not calculate from invalid data.

TRANSFER IT TO AN EXAM ANSWER

Be explicit about empty-list handling before division. A function returning a status or error can make failure visible to its caller.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish a function, a procedure and a parameter.

Q2 / 5 MARKS

Identify three routines for the report and give an input and output/result for each.

Q3 / 6 MARKS

Design modular pseudocode for validating, calculating and presenting a mean. Explain two testing advantages of this design.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Function returns a value [1]; procedure performs an action [1]; parameter supplies routine data [1].

Q2 / 5

Validation receives marks, returns Boolean [2]; average receives valid marks, returns mean [2]; display receives mean and outputs report [1].

Q3 / 6

Use separate validation routine [1], mean calculation [1], display routine [1]; refuse invalid/empty inputs [1]; test subroutines independently [1]; reuse validation or isolate a bug more quickly [1].

COMMON MISCONCEPTION

A procedure is not defined by having no parameters; both procedures and functions may accept parameters.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(d) / SEE IT / CONCEPT AND VISUAL

Trace copies and shared changes accurately

Pass-by-value gives a routine a separate value; pass-by-reference lets a routine operate on the caller's variable or storage.

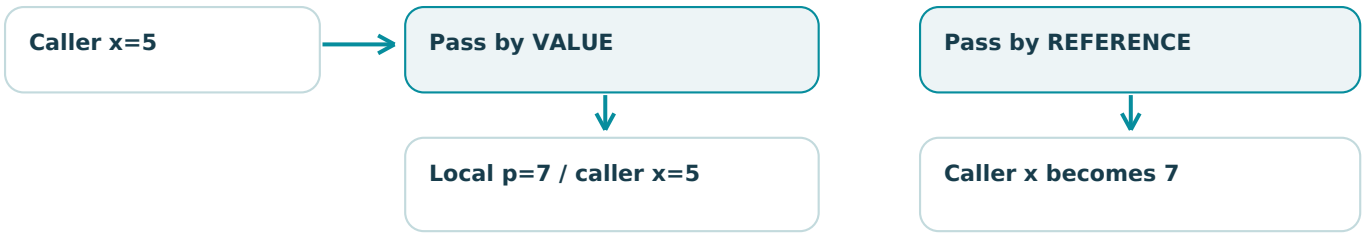
BY VALUE
Local changes do not alter caller scalar.

BY REFERENCE
Changes affect caller's original value.

CALLER
Owns the original variable.

SIDE EFFECT
A routine changes something outside itself.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Different languages and data types have different semantics: the example uses explicit conceptual value/reference parameter modes, not Python list assignment rules.

WORDS THAT EARN MARKS

Pass by value	Use a copy	**Pass by reference**	Use original
Parameter	Routine input	**Side effect**	External change

EXAM TRAP

Passing a mutable object in one language does not automatically mean its parameter mechanism is pass-by-reference in the OCR sense.

2.2.1(d) / WORKED EXAMPLE / STEP BY STEP

Run the same routine with two passing modes

Procedure addTwo(p) sets p = p + 2. Caller variable x starts at 5. Compare calling it with parameter p BY VALUE versus p BY REFERENCE.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Caller	**x = 5**	Original variable.
2 / Value call	**p starts at 5**	Local copy is made.
3 / Value update	**p = 7**	Caller x remains 5.
4 / Reference call	**p aliases x**	Routine refers to original location.
5 / Reference update	**x becomes 7**	Change is visible to caller.
6 / Observe	**value mode x=5; reference x=7**	Same body, different parameter semantics.

TRANSFER IT TO AN EXAM ANSWER

A pure function returning x+2 could make the update explicit: x = addTwo(x). That differs from mutating the caller by reference.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define pass-by-value, pass-by-reference and a side effect.

Q2 / 5 MARKS

Complete a trace of x=5 after addTwo(x) under each passing mode and explain the difference.

Q3 / 6 MARKS

Discuss when to use a by-reference parameter for a routine that updates both a balance and a transaction count. State testing risks.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Value passes a separate copy [1]; reference lets routine update original variable [1]; side effect changes state outside routine [1].

Q2 / 5

By-value caller x remains 5 [1]; local p becomes 7 [1]; by-reference caller x becomes 7 [1]; p refers to caller storage [1]; a later read sees changed state [1].

Q3 / 6

Reference can update balance [1] and transaction count [1] directly, but should validate transaction first [1]; unexpected side effects complicate tests [1]; two outputs can alternatively be returned explicitly [1]; document parameters and test no-change on failed operations [1].

COMMON MISCONCEPTION

Passing a mutable object in one language does not automatically mean its parameter mechanism is pass-by-reference in the OCR sense.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(e) / SEE IT / CONCEPT AND VISUAL

Use editor, breakpoints and watch variables deliberately

An integrated development environment combines authoring, executing, inspecting and debugging tools; an error can be located by observing state at a breakpoint.

EDITOR

Write and navigate source.

BREAKPOINT

Pause at a chosen location.

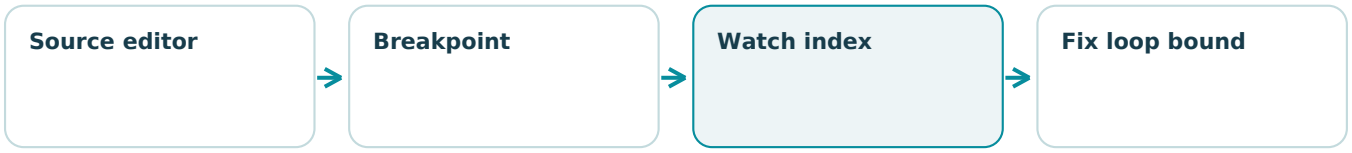
WATCH

Inspect values as execution proceeds.

STEPPING

Execute one statement at a time.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

A successful run with one input does not establish correctness. Retest boundary and erroneous inputs after changing the code.

WORDS THAT EARN MARKS

IDE	Development tools	**Breakpoint**	Pause execution
Watch window	Inspect state	**Step over**	Next statement

EXAM TRAP

A syntax-error-free program may still have logical and runtime errors.

2.2.1(e) / WORKED EXAMPLE / STEP BY STEP

Locate an off-by-one error with a watch window

An average-mark routine loops through indices 0 to LENGTH(marks) inclusive. For marks=[8,10,12], the array has three elements.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Expected	**Valid indices are 0,1,2**	Length 3 does not include index 3.
2 / Pause	**Breakpoint inside loop**	Stop just before array access.
3 / Watch	**index=0,1,2: valid**	Inspect counter and current element.
4 / Failure	**index=3: out of range**	Inclusive upper bound is wrong.
5 / Correct	**Use index < LENGTH(marks)**	Last visited index is 2.
6 / Retest	**Try empty, one-item, three-item arrays**	Check other boundaries and empty handling.

TRANSFER IT TO AN EXAM ANSWER

Use diagnostic messages or a call stack when the fault occurs inside nested routines. A debugger helps find faults; it does not replace deliberate tests.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(e) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Name three useful IDE debugging facilities.

Q2 / 5 MARKS

Explain how a breakpoint, step-through and watch window identify the indexing error.

Q3 / 6 MARKS

Propose a debugging and test plan for a function failing only on empty or one-item arrays.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Breakpoint [1]; watch variable window [1]; stepping, call stack or run tools [1].

Q2 / 5

Breakpoint just before array access [1]; step through [1]; inspect index [1]; find index=3 [1]; replace inclusive upper bound with < length [1].

Q3 / 6

Reproduce with failing input [1]; set breakpoints at loop/mean calculation [1]; inspect length/index/total [1]; guard against empty division [1]; test one-item, boundary and ordinary cases [1]; rerun whole relevant test suite after fix [1].

COMMON MISCONCEPTION

A syntax-error-free program may still have logical and runtime errors.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(f) / SEE IT / CONCEPT AND VISUAL

Connect a class, an object and its behaviour

Object-oriented programming encapsulates data and methods together. Classes define objects; inheritance and polymorphism support reusable behaviour.

CLASS

Blueprint for objects.

OBJECT

Instance with its own state.

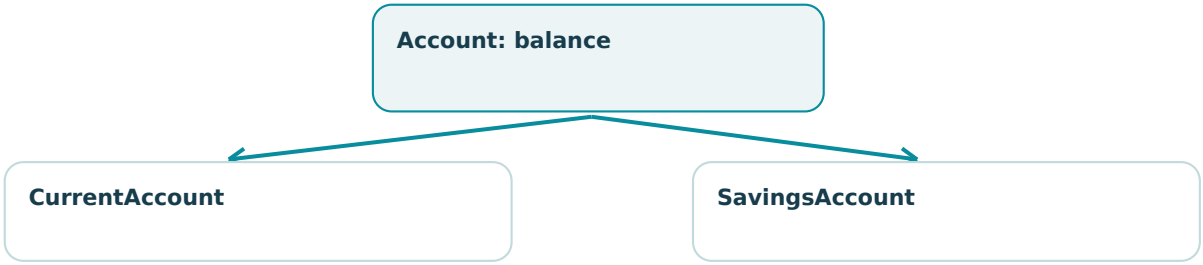
ENCAPSULATION

Protect state through controlled access.

INHERITANCE

Specialise shared features.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Both current and savings accounts can share a valid deposit method. A savings account may override calculateInterest while preserving the shared interface.

WORDS THAT EARN MARKS

Class	Blueprint	**Object**	Instance
Method	Behaviour	**Polymorphism**	One interface; forms

EXAM TRAP

Inheritance represents an is-a relationship, not every has-a relationship. Do not confuse overloading and overriding.

2.2.1(f) / WORKED EXAMPLE / STEP BY STEP

Trace one account object with a protected balance

An Account object starts with balance 100. deposit(30) validates a positive amount; withdraw(50) requires enough funds. A later withdraw(90) is attempted.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Instantiate	a = Account(100)	Object owns its balance.
2 / Deposit	a.deposit(30)	Public method checks amount > 0.
3 / State	balance = 130	Method updates encapsulated state.
4 / Withdraw	a.withdraw(50)	Balance sufficient.
5 / State	balance = 80	Valid withdrawal changes state.
6 / Reject	withdraw(90) => rejected	Balance stays 80: invariant retained.

TRANSFER IT TO AN EXAM ANSWER

A child SavingsAccount class could reuse deposit and withdraw, and override interest calculation. Calling calculateInterest on different account types demonstrates polymorphism.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish a class, an object and encapsulation.

Q2 / 5 MARKS

Trace the balance and explain why the last withdrawal is rejected.

Q3 / 6 MARKS

Design a class hierarchy for Account, CurrentAccount and SavingsAccount, explaining inheritance, polymorphism and one limitation.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Class is blueprint [1]; object is instantiated state [1]; encapsulation controls access to state via methods [1].

Q2 / 5

After deposit 130 [1], after withdrawal 80 [1], insufficient funds for 90 [1], reject operation [1], balance remains 80 [1].

Q3 / 6

Account base class holds shared data/behaviour [1]; subclasses inherit common methods [1]; override appropriate method e.g. interest [1]; same call works with account subtypes (polymorphism) [1]; protect balance with validation [1]; deeper hierarchy may be harder to maintain [1].

COMMON MISCONCEPTION

Inheritance represents an is-a relationship, not every has-a relationship. Do not confuse overloading and overriding.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.2(a,b) / SEE IT / CONCEPT AND VISUAL

Turn a vague request into a computational problem

A computationally solvable problem needs defined inputs, a specified output, workable rules and finite executable steps within practical resource limits.

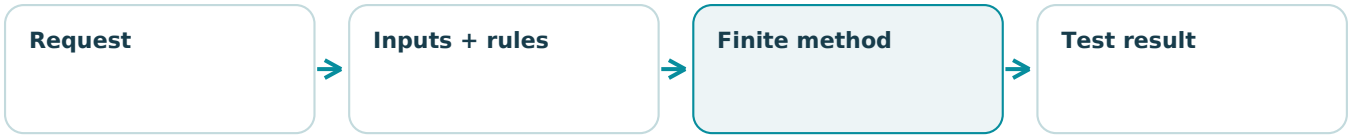
RECOGNISE
State the actual problem.

INPUT
Specify valid data and constraints.

OUTPUT
Define an observable result.

FEASIBILITY
Can a procedure meet resource limits?

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

"Make traffic better" is not a full computational specification. "Find a route with minimum recorded journey time on a fixed road graph" is testable and constrained.

WORDS THAT EARN MARKS

Recognition	Identify problem	**Constraint**	Required limit
Feasible	Practical to solve	**Computable**	Algorithmic solution

EXAM TRAP

Not every human judgement is a well-defined algorithmic problem until goals, rules and acceptable trade-offs are stated.

2.2.2(a,b) / WORKED EXAMPLE / STEP BY STEP

Frame a bus-routing request before choosing an algorithm

A school wants a route from Depot D to School S on a road network. Each road has a non-negative estimated travel time.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Goal	**Minimise predicted travel time**	Convert vague wish into a target.
2 / Inputs	**Road graph, D, S, edge times**	Identify needed data.
3 / Constraints	**Only allowed roads; times >= 0**	Specify valid solution space.
4 / Output	**A route plus predicted minutes**	Define verifiable result.
5 / Method	**Shortest-path search**	Problem has graph structure.
6 / Test	**A small known network with expected route**	Check solution before real deployment.

TRANSFER IT TO AN EXAM ANSWER

Dynamic traffic conditions require updated weights or a stated model limitation. Practical feasibility considers memory, execution time and quality of data.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(a,b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Give three features of a well-specified computational problem.

Q2 / 5 MARKS

Explain how to turn "find a good route" into an algorithmic problem, specifying input, output and success criterion.

Q3 / 6 MARKS

Evaluate whether a computer can choose a "fair" timetable for all pupils without further information; describe what must be defined first.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Explicit input [1]; precise output or success criterion [1]; usable rules/constraints and finite processing [1].

Q2 / 5

Define the desired criterion, e.g. minimum minutes [1]; road graph [1]; start and destination [1]; legal-route constraints [1]; output route and total minutes [1].

Q3 / 6

"Fair" is ambiguous [1]; identify affected groups [1]; define measurable priorities [1]; specify hard constraints [1]; choose objective/trade-offs [1]; evaluate with stakeholders and real-world cases [1].

COMMON MISCONCEPTION

Not every human judgement is a well-defined algorithmic problem until goals, rules and acceptable trade-offs are stated.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(c) / SEE IT / CONCEPT AND VISUAL

Name genuine sub-problems and their interfaces

Decomposition divides a larger problem into smaller sub-problems that can be specified, solved and tested before integration.

TOP LEVEL

One clear problem.

SUB-PROBLEM

A manageable independent responsibility.

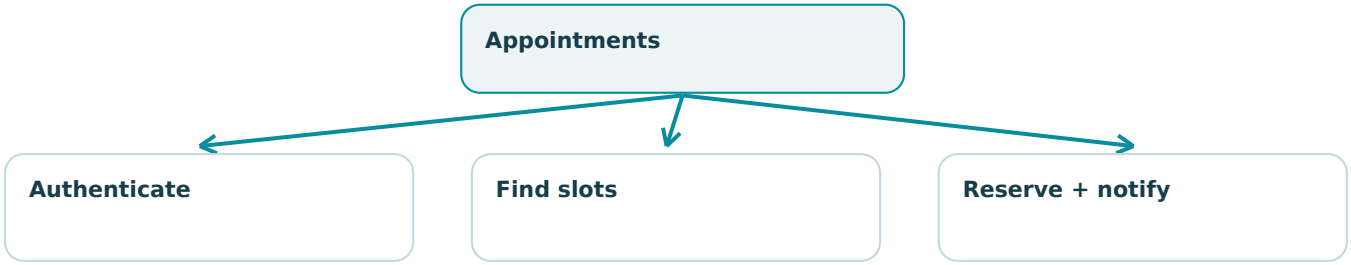
INTERFACE

What data enters and leaves.

INTEGRATION

Combine tested parts in the right order.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

A decomposition diagram should show meaningful tasks, not vague phrases like "do first part". Keep data flow visible between sub-problems.

WORDS THAT EARN MARKS

- **Decomposition** — Smaller sub-problems
- **Module** — Coherent component
- **Interface** — Input/output contract
- **Integration** — Join components

EXAM TRAP

Decomposition is not the same as abstraction: decomposition separates responsibilities; abstraction removes irrelevant detail.

2.2.2(c) / WORKED EXAMPLE / STEP BY STEP

Break down a parents-evening appointment system

A system receives pupil/parent requests, offers available teacher slots, saves a booking and sends a confirmation.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Identify	**Manage appointments**	Top-level problem.
2 / Subproblem	**Authenticate parent**	Check eligibility.
3 / Subproblem	**Find available slots**	Read teacher and slot data.
4 / Subproblem	**Reserve chosen slot**	Make a consistent state update.
5 / Subproblem	**Send confirmation**	Use booking ID as input.
6 / Integrate	**Authenticate -> choose -> reserve -> notify**	Define dependency between outputs.

TRANSFER IT TO AN EXAM ANSWER

Each sub-problem can have a separate test: for example, an already-booked slot must not be offered. Integration tests then check complete workflows.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define decomposition and give two benefits.

Q2 / 5 MARKS

Construct four sub-problems for the appointment system, with one input/output pair.

Q3 / 6 MARKS

Explain how interfaces and tests would prevent a double-booked teacher slot when integrating modules.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Break into smaller sub-problems [1]; easier isolated testing [1]; easier teamwork/reuse/maintenance [1].

Q2 / 5

Authenticate parent [1]; list availability [1]; reserve slot [1]; confirm booking [1]; interface example: teacher and date in, free slots out [1].

Q3 / 6

Specify one reservation API [1]; validate desired slot [1]; check availability at commit [1]; atomically reserve [1]; unit test clashes [1]; integrate two simultaneous booking requests [1].

COMMON MISCONCEPTION

Decomposition is not the same as abstraction: decomposition separates responsibilities; abstraction removes irrelevant detail.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(d) / SEE IT / CONCEPT AND VISUAL

Split, solve recursively and combine

Divide and conquer recursively splits a problem into smaller related instances, solves them, then combines their results.

DIVIDE

Split input into parts.

CONQUER

Solve each part; recurse if needed.

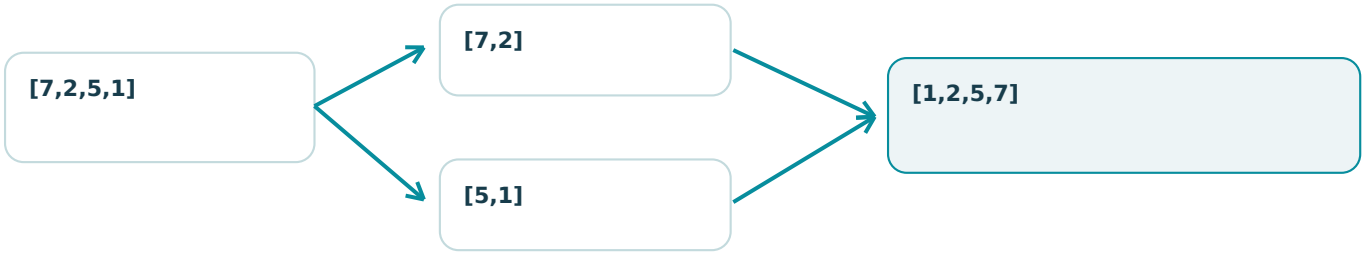
COMBINE

Join partial results.

BASE CASE

Stop when small enough.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Merge sort is a concrete divide-and-conquer example: the final result depends on combining two already-sorted halves.

WORDS THAT EARN MARKS

- **Divide** Split problem
- **Conquer** Solve parts
- **Combine** Merge answers
- **Recurrence** Repeated subproblems

EXAM TRAP

Divide and conquer does not mean "split once and stop"; explain what is done to each sub-problem and how answers combine.

2.2.2(d) / WORKED EXAMPLE / STEP BY STEP

Follow one merge-sort split and merge completely

Input array is [7, 2, 5, 1]. The merge operation repeatedly selects the smallest front item of two sorted lists.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Divide	**[7,2] and [5,1]**	Split array in halves.
2 / Divide	**[7],[2],[5],[1]**	Single values are sorted base cases.
3 / Merge left	**[2,7]**	Compare 7 and 2.
4 / Merge right	**[1,5]**	Compare 5 and 1.
5 / Merge	**1, then 2, then 5, then 7**	Select lower available front each time.
6 / Result	**[1,2,5,7]**	All elements retained and ordered.

TRANSFER IT TO AN EXAM ANSWER

Divide and conquer is an algorithm design method, not just dividing work between programmers. Merge sort typically uses O(n log n) time and O(n) extra space for arrays.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State divide, conquer and combine.

Q2 / 5 MARKS

Show every split and merge for [7,2,5,1].

Q3 / 6 MARKS

Compare using divide and conquer in merge sort with a simple repeated adjacent-swap approach, commenting on time and space.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Divide the problem [1]; solve parts recursively/base cases [1]; combine results [1].

Q2 / 5

Split [7,2] / [5,1] [1]; four singleton lists [1]; merge [2,7] [1]; merge [1,5] [1]; final [1,2,5,7] [1].

Q3 / 6

Merge sort splits recursively [1], combines sorted lists [1], worst-case time O(n log n) [1]; bubble sort has O(n^2) worst-case [1]; merge sort requires auxiliary storage in usual array version [1]; choice depends on input size and memory [1].

COMMON MISCONCEPTION

Divide and conquer does not mean "split once and stop"; explain what is done to each sub-problem and how answers combine.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(e) / SEE IT / CONCEPT AND VISUAL

Simplify a real situation without losing correctness

In computational methods, abstraction identifies the features needed to solve the stated problem and deliberately ignores unrelated detail.

PURPOSE

Specify the exact question.

KEEP

Variables that affect the decision.

IGNORE

Details irrelevant to the question.

VALIDATE

Check the assumptions and exceptions.

HOW THE PARTS FIT TOGETHER



Ignore cover colour; retain due date and return state.

FROM THE DIAGRAM TO EXAM LANGUAGE

A model for overdue notices needs due dates, return state and borrower identity; it does not need book-cover artwork.

WORDS THAT EARN MARKS

Abstraction	Remove irrelevant	**Model**	Simplified system
Assumption	Rule treated as true	**Validation**	Check model

EXAM TRAP

A model is not automatically valid forever; new requirements can make an omitted detail relevant.

2.2.2(e) / WORKED EXAMPLE / STEP BY STEP

Design the smallest useful overdue-loan model

Library policy: notify a borrower if a book is unreturned and today is later than its due date. Today is day 15; loan due day 12 is unreturned.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Purpose	**Decide whether to notify**	One specified outcome.
2 / Keep	**borrowerID, dueDay, returned**	All can affect output.
3 / Ignore	**coverColour, shelfDecoration**	Do not affect notification.
4 / Rule	**NOT returned AND today>dueDay**	Boolean predicate.
5 / Evaluate	**TRUE AND 15>12 => TRUE**	This loan is overdue.
6 / Output	**Notify borrower**	Relevant result from the model.

TRANSFER IT TO AN EXAM ANSWER

If loans can be renewed or due dates shifted on closure days, those real-world features may need to be restored to the model.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(e) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define abstraction and name two relevant loan fields.

Q2 / 5 MARKS

Identify two excluded details and explain why the sample loan triggers a notice.

Q3 / 6 MARKS

Evaluate how the model should change if pupils may renew loans online or return them during closure periods.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Abstraction omits irrelevant detail [1]; due date [1]; return status or borrower identifier [1].

Q2 / 5

Book-cover colour and shelf decoration irrelevant [2]; not returned [1]; day 15 later than day 12 [1]; therefore notify [1].

Q3 / 6

Keep renewal status or revised due date [1]; include actual return timestamp [1]; define closure-day policy [1]; update notification predicate [1]; test same-day/renewed/closure boundaries [1]; validate against library policy [1].

COMMON MISCONCEPTION

A model is not automatically valid forever; new requirements can make an omitted detail relevant.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Try a choice; undo it if it blocks the goal

Backtracking explores candidate choices systematically. When a partial solution violates constraints or reaches a dead end, undo the choice and try another.

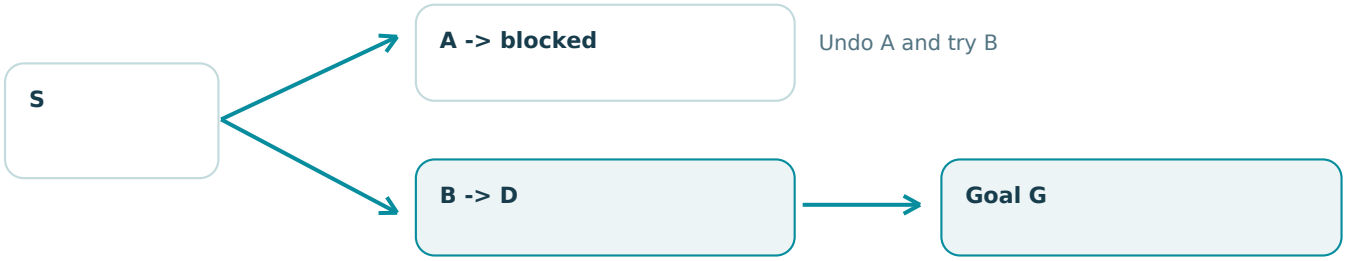
CHOOSE
Add one possible step.

TEST
Does it respect constraints?

BACKTRACK
Undo a failed choice.

STOP
Goal found, or choices exhausted.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

The search must record the current path or state. Undoing an invalid candidate is not randomly restarting from zero.

WORDS THAT EARN MARKS

Backtracking	Undo decisions	**Search tree**	Possible choices
Constraint	Rule to satisfy	**Dead end**	No valid next step

EXAM TRAP

Backtracking is an exhaustive-search strategy with pruning, not a guarantee of efficient performance on large inputs.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Find a legal path through a tiny maze

From Start S choose A or B. A leads to C, which is blocked. B leads to D and then Goal G. Depth-first order tries A first.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Start	**Path [S]**	No decision made yet.
2 / Choose A	**[S,A]**	First branch is explored.
3 / Try C	**[S,A,C] => blocked**	Constraint violation discovered.
4 / Undo	**Return to [S]**	Backtrack past failed A branch.
5 / Choose B	**[S,B,D]**	Second branch remains available.
6 / Goal	**[S,B,D,G]**	Valid path found.

TRANSFER IT TO AN EXAM ANSWER

If no branch leads to a goal, the algorithm must report failure rather than pretending that its last partial path is complete.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define backtracking, a dead end and a search tree.

Q2 / 5 MARKS

Trace the first failed choice and successful path in the maze.

Q3 / 6 MARKS

Describe a backtracking algorithm for assigning pupils to limited places in clubs; give pruning conditions and one drawback.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Backtracking undoes choices [1]; dead end violates requirements/has no options [1]; search tree represents alternative choices [1].

Q2 / 5

Try S-A [1], then C [1], find it blocked [1], undo A [1], choose S-B-D-G [1].

Q3 / 6

Assign next pupil [1]; try club with capacity and preference rules [1]; recurse on partial assignment [1]; undo when no valid extension [1]; prune capacity/eligibility violations early [1]; worst-case combinations may grow exponentially [1].

COMMON MISCONCEPTION

Backtracking is an exhaustive-search strategy with pruning, not a guarantee of efficient performance on large inputs.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Find patterns, then test whether they are useful

Data mining searches large data sets for trends, associations and anomalies. A discovered correlation does not by itself establish a cause.

COLLECT
Gather relevant, lawful data.

CLEAN
Handle errors and missing records.

DISCOVER
Identify repeated patterns.

VALIDATE
Check patterns on separate data.

HOW THE PARTS FIT TOGETHER



60% and 20% are observed group rates, not a causal statement.

FROM THE DIAGRAM TO EXAM LANGUAGE

A group with lower attendance and lower scores shows an association, but unseen factors may influence both. A model should be tested, not simply trusted.

WORDS THAT EARN MARKS

Data mining	Pattern discovery	**Correlation**	Association
Outlier	Unusual case	**Bias**	Systematic skew

EXAM TRAP

An observed correlation is not a causal explanation or permission to make automated high-stakes decisions.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Interpret a small aggregated pattern responsibly

Among 100 pupils, 20 have attendance under 90%; 12 of those 20 score below a target. Of 80 pupils at 90% or above, 16 score below target.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Group A	**12 / 20 = 60%**	Low-attendance pupils below target.
2 / Group B	**16 / 80 = 20%**	Other pupils below target.
3 / Compare	**Difference = 40 percentage points**	An association appears in these data.
4 / Avoid	**Do NOT claim low attendance caused result**	Other factors may matter.
5 / Validate	**Compare a new cohort / held-out data**	Check whether pattern generalises.
6 / Safeguard	**Limit access and inspect bias**	Avoid unfair decisions about individuals.

TRANSFER IT TO AN EXAM ANSWER

The percentages are group observations; they do not predict an individual pupil with certainty. Any support decision needs appropriate safeguards.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define data mining, correlation and an outlier.

Q2 / 5 MARKS

Calculate both rates and explain why the pattern is not proof of causation.

Q3 / 6 MARKS

Outline a defensible data-mining workflow to study attendance and outcomes, with privacy, representativeness and evaluation.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Data mining seeks patterns [1]; correlation means association [1]; outlier is unusual observation [1].

Q2 / 5

Low group 12/20=60% [1]; high group 16/80=20% [1]; difference 40 percentage points [1]; association detected [1]; other causes and confounders exist [1].

Q3 / 6

Specify legitimate question [1]; use minimised, appropriately protected data [1]; clean missing/outlier errors [1]; explore patterns [1]; validate on independent sample [1]; check representation and consequences before acting [1].

COMMON MISCONCEPTION

An observed correlation is not a causal explanation or permission to make automated high-stakes decisions.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Find a useful answer without exhaustive search

A heuristic is a rule-of-thumb that directs search or produces a quick approximate solution. It may improve speed without guaranteeing the best result.

RULE

A fast decision strategy.

ADVANTAGE

Reduce search effort.

COST

Can miss optimum.

EVALUATE

Compare quality and runtime.

HOW THE PARTS FIT TOGETHER

A -> B -> D

2+10 = 12

A -> C -> D

3+4 = 7

Greedy is worse

FROM THE DIAGRAM TO EXAM LANGUAGE

Choosing the next nearest stop is a heuristic: it uses local information. The globally best route can require a locally worse choice.

WORDS THAT EARN MARKS

- **Heuristic** Rule of thumb
- **Greedy** Local choice
- **Optimal** Best possible
- **Approximate** May not be best

EXAM TRAP

"Heuristic" is not another name for an exact algorithm. State the trade-off between speed and guaranteed optimality.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Expose a greedy-choice trap with a tiny graph

The goal is travel from A to D. Direct edges: A-B=2, A-C=3, B-D=10, C-D=4. A greedy nearest-first choice prefers B.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Options	**A-B 2 versus A-C 3**	Greedy rule prefers smaller immediate cost.
2 / Choose B	**Cost so far 2**	Local choice committed.
3 / Continue	**B-D 10**	Only listed onward option.
4 / Greedy total	**2 + 10 = 12**	Candidate route A-B-D.
5 / Alternative	**A-C-D = 3 + 4 = 7**	Locally longer first edge is better.
6 / Conclusion	**Heuristic result is not optimal**	Compare complete path costs.

TRANSFER IT TO AN EXAM ANSWER

Use a provably suitable method such as Dijkstra on non-negative weighted graphs when guaranteed shortest path is required; do not confuse "nearest first" with Dijkstra.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define heuristic, greedy and optimal.

Q2 / 5 MARKS

Compare routes A-B-D and A-C-D and explain the poor greedy choice.

Q3 / 6 MARKS

Evaluate a heuristic for scheduling homework tasks under a tight processing-time limit, including how to judge solution quality.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Heuristic guides search quickly [1]; greedy takes a locally preferred option [1]; optimal is best according to objective [1].

Q2 / 5

A-B-D=12 [1]; A-C-D=7 [1]; heuristic selects B because 2<3 [1]; total 12 is worse than 7 [1]; local decision not globally best [1].

Q3 / 6

Specify objective [1]; choose priority rule [1]; improve runtime compared with searching all schedules [1]; test on known optimal small cases [1]; measure quality gap [1]; accept only if result is adequate for context [1].

COMMON MISCONCEPTION

"Heuristic" is not another name for an exact algorithm. State the trade-off between speed and guaranteed optimality.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Predict work, then test the prediction

Performance modelling uses simplified mathematical or simulated representations to estimate resource use, response time or throughput before deployment.

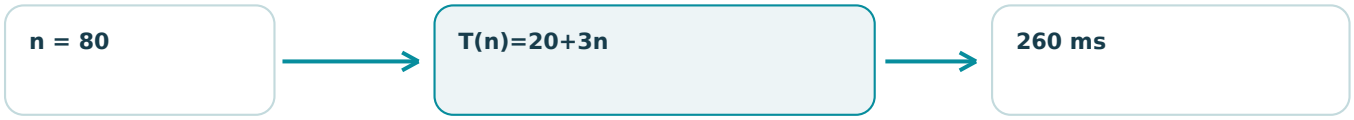
ASSUMPTION
Declare what is held constant.

MODEL
Predict time or capacity.

MEASURE
Collect observed results.

REVISE
Explain discrepancies.

HOW THE PARTS FIT TOGETHER



Measured runtime may differ because assumptions are simplified.

FROM THE DIAGRAM TO EXAM LANGUAGE

Use an explicitly assumed per-item cost to make a prediction; real systems add startup costs, caching, contention and I/O latency.

WORDS THAT EARN MARKS

Performance Time or resources

Throughput Completed per second

Latency Time per request

Bottleneck Limiting stage

EXAM TRAP

An algebraic model is a prediction based on assumptions; measurements are still required.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Build a small cost model for a serial batch

A program processes 80 identical records, taking an assumed 3 ms per record plus a fixed 20 ms setup cost. Assume no overlap.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Define	**T(n) = 20 + 3n ms**	Linear model with startup cost.
2 / Substitute	**n = 80**	Batch size is known.
3 / Compute	**T(80) = 20 + 240 = 260 ms**	Predicted completion time.
4 / Scale	**T(160) = 20 + 480 = 500 ms**	Doubling input does not double setup.
5 / Measure	**Observe actual timings**	Compare against prediction.
6 / Refine	**Account for cache or I/O variance**	Update assumptions if needed.

TRANSFER IT TO AN EXAM ANSWER

The model cannot justify a real-world guarantee without collecting evidence and checking whether per-item cost remains stable.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define latency, throughput and bottleneck.

Q2 / 5 MARKS

Use T(n)=20+3n to predict 80 and 160 records; explain one model assumption.

Q3 / 6 MARKS

Design a fair performance experiment comparing two solutions to the same problem, including repetition and data-set effects.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Latency is elapsed response time [1]; throughput is number completed per unit time [1]; bottleneck limits overall performance [1].

Q2 / 5

T(80)=260 ms [2]; T(160)=500 ms [2]; each record assumed to cost 3 ms regardless of size or order [1].

Q3 / 6

Use identical hardware/environment [1]; same representative inputs [1]; measure wall time and relevant memory [1]; repeat trials [1]; inspect mean/distribution and anomalies [1]; compare against model with caveats [1].

COMMON MISCONCEPTION

An algebraic model is a prediction based on assumptions; measurements are still required.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Overlap stages instead of waiting for each task

Pipelining divides work into stages so different tasks can occupy different stages at the same time. Throughput may improve after fill-up.

STAGES

Sequential operations for one item.

OVERLAP

Different items in different stages.

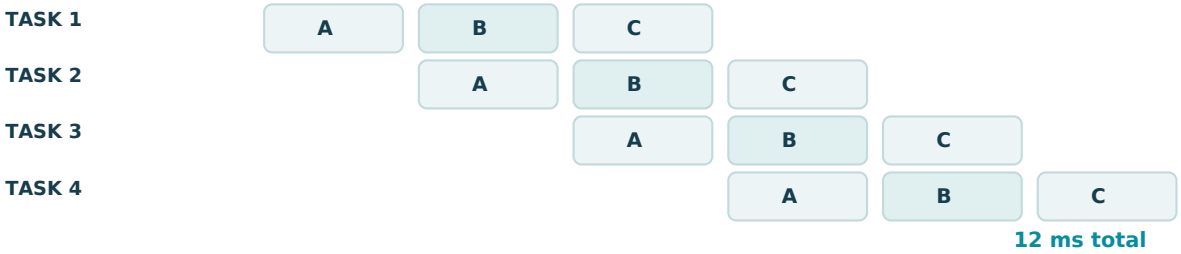
LATENCY

One item still traverses all stages.

THROUGHPUT

More items finish per time unit.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

An idealised three-stage pipeline makes four tasks complete faster as a batch, but does not make the latency of the first task one stage long.

WORDS THAT EARN MARKS

- **Pipeline** Overlapping stages
- **Fill** Start-up cycles
- **Latency** Time for one
- **Hazard** Prevents overlap

EXAM TRAP

Pipeline speed-up is a batch throughput improvement, not a guarantee that each single operation becomes that many times faster.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Compare serial and pipelined work with equal stages

Each task requires three 2 ms stages (A,B,C). Four independent tasks pass through the stages; assume no overhead or stalls.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Serial item	**2+2+2 = 6 ms**	One task traverses all stages.
2 / Serial batch	**4 x 6 = 24 ms**	No overlap.
3 / Pipeline fill	**First completion at 6 ms**	Latency of first task unchanged.
4 / Steady state	**Next finishes every 2 ms**	One completion per stage period.
5 / Four tasks	**(3+4-1) x 2 = 12 ms**	Ideal batch completion time.
6 / Compare	**24 ms versus 12 ms**	Batch throughput improves twofold here.

TRANSFER IT TO AN EXAM ANSWER

If one stage takes 5 ms, a simple fixed-stage pipeline cannot finish items faster than that bottleneck stage. Dependencies and hazards can cause stalls.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish pipeline latency and throughput; state one hazard.

Q2 / 5 MARKS

Calculate serial and pipelined completion times for four tasks with three 2 ms stages.

Q3 / 6 MARKS

Explain why a real pipeline might not achieve the ideal speed-up; apply dependency and bottleneck arguments.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Latency means one task completion time [1]; throughput means completions over time [1]; a hazard blocks ideal overlap [1].

Q2 / 5

Serial: 4 x 6=24 ms [2]; pipeline (3+4-1)x2=12 ms [2]; speed-up 24/12=2 [1].

Q3 / 6

Stage imbalance limits clock/period [1]; data dependency may stall [1]; branch or control hazard may flush work [1]; filling/draining adds cost [1]; communication/synchronisation has overhead [1]; so ideal throughput may not be achieved [1].

COMMON MISCONCEPTION

Pipeline speed-up is a batch throughput improvement, not a guarantee that each single operation becomes that many times faster.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

2.2.2(f) / SEE IT / CONCEPT AND VISUAL

Choose a picture that reveals the computation

Visualisation turns data or process state into diagrams or charts that reveal trends, bottlenecks or relationships. Choosing the wrong scale or chart misleads.

TABLE

Exact values are visible.

BAR CHART

Compare categories.

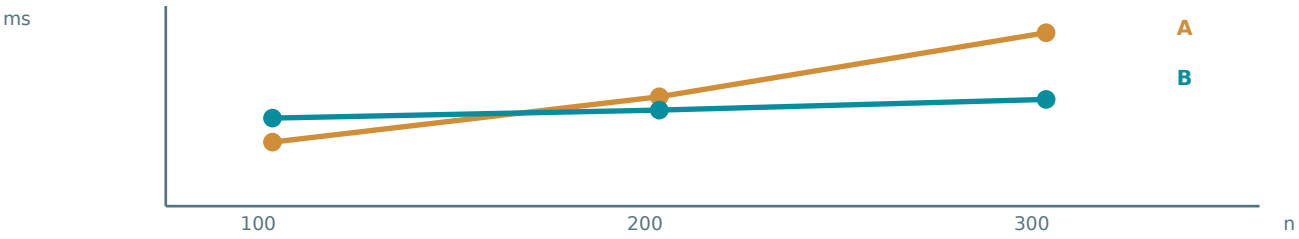
LINE GRAPH

Show change over time.

GRAPH NETWORK

Show connections or routes.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Choose an appropriate encoding for the question: a bar chart compares algorithms at a fixed n; a line chart shows how execution time changes as n increases.

WORDS THAT EARN MARKS

Visualisation	Graphical display	**Axis**	Scaled dimension
Trend	Overall pattern	**Misleading scale**	Distorted reading

EXAM TRAP

A neat graphic is not necessarily an accurate representation; axes, units, sample size and context affect validity.

2.2.2(f) / WORKED EXAMPLE / STEP BY STEP

Visualise a performance comparison without hiding labels

Algorithm A times (ms) for n=100,200,300: [8,16,24]. Algorithm B: [12,13,14]. Draw n on the horizontal axis and runtime on vertical axis.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Data	**n=100,200,300**	Input size is independent variable.
2 / A	**8,16,24 ms**	A grows by 8 ms per +100 records.
3 / B	**12,13,14 ms**	B grows by 1 ms per +100 records.
4 / Plot	**Two labelled series**	Use same axes for fair comparison.
5 / Compare	**At n=100, A faster (8<12)**	A starts with smaller observed time.
6 / Compare	**At n=300, B faster (14<24)**	Performance ranking changes with n.

TRANSFER IT TO AN EXAM ANSWER

Plot measured values only and explain what the chart does not prove: three measurements do not establish a full asymptotic complexity class.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.2(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Name a suitable visual for time series, categories and graph connections.

Q2 / 5 MARKS

Interpret the A/B timings at n=100 and n=300. Identify the axes and units.

Q3 / 6 MARKS

Critique a graph comparing algorithm times that truncates the y-axis, lacks units and mixes different hardware results.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Line chart for time series [1]; bar for categories [1]; network diagram for connections [1].

Q2 / 5

Horizontal input size n [1]; vertical time in ms [1]; n=100 A faster 8<12 [1]; n=300 B faster 14<24 [1]; ranking changes as input grows [1].

Q3 / 6

Truncated scale can exaggerate relative change [1]; missing units prevent interpretation [1]; different hardware confounds comparison [1]; use consistent apparatus [1]; label both axes and legend [1]; show honest scale/data plus methodological notes [1].

COMMON MISCONCEPTION

A neat graphic is not necessarily an accurate representation; axes, units, sample size and context affect validity.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-2/

OCR APPENDIX 5d / APPLIED PROGRAMMING

Read it. Trace it. Write it.

Four additional four-page learning cycles complement the 18 existing Section 2.2 cycles. The examples use OCR H446 Appendix 5d conventions but are original practice, not OCR examination questions or prescribed code to memorise.

01 Passing parameters by value and reference
2.2.1(c,d) / pp. 2-5

02 Zero-based 2D arrays and loop bounds
2.2.1(a) / 1.4.2 / pp. 6-9

03 File read/write and end-of-file loops
Appendix 5d / synoptic / pp. 10-13

04 Constructors, inheritance and methods
2.2.1(f) / pp. 14-17

What OCR expects

Recognise the OCR reference notation, explain each state change, write a logically correct solution and justify boundary decisions. Alternative readable pseudocode or suitable programming-language solutions may also be acceptable.

What this adds

The chapter already covers concepts. This bridge adds complete code models, intermediate traces, original exam-style writing tasks, and separate Check pages.

Source

Cambridge OCR H446 specification v3.0 (2026), section 2.2 and Appendix 5d, including file and object-oriented conventions.

UNDERSTAND / 1 OF 4

Parameter passing: byVal and byRef

OCR assumes parameters are passed by value unless otherwise stated. When relevant it labels each parameter :byVal or :byRef. Trace the caller as well as the local subroutine state.

KEY IDEAS AND APPENDIX CONVENTIONS

byVal / copy

An update inside the procedure changes the local parameter, not the caller variable.

byRef / shared original

An update inside the procedure changes the caller variable immediately.

Scope

Local variables belong to the subroutine. A global value can persist and be changed elsewhere; explicit parameters reduce hidden dependencies.

EXAM-STYLE PSEUDOCODE

```
procedure adjust(a:byVal, b:byRef)
    a = a + 4
    b = b + a
endprocedure

x = 6
y = 6
adjust(x, y)
print(x)  // 6
print(y)  // 16
```

MODEL / 2 OF 4

Trace the caller and the parameter separately

A complete worked model: follow the code, then track each intermediate state before the final output.

```
procedure adjust(a:byVal, b:byRef)
    a = a + 4
    b = b + a
endprocedure

x = 6; y = 6
adjust(x,y)
print(x)
print(y)
```

WORKED TRACE / THINK ALOUD

1. Caller starts x=6 and y=6.

2. Inside adjust, a starts as a copy: 6; b refers to caller y.

3. a becomes 10. Updating b to 6+10 changes y to 16.

4. After return, the caller prints 6, 16. x did not change.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

State the difference between :byVal and :byRef and identify the default when it is not specified.

QUESTION 2 / 5 MARKS

Trace adjust(x,y) when x=3 and y=5. Show local a, final x and final y using the model code.

QUESTION 3 / 6 MARKS

Write procedure addCharge(bill:byRef, extra:byVal) that rejects a negative extra and otherwise adds extra to bill. Show a call and explain which variable changes.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

Value passes a copy [1]; reference can modify the caller variable [1]; the default is by value unless otherwise stated [1].

Q2 / 5 MARKS

Start a=3 and b refers to y=5 [1]; a becomes 7 [1]; b/y becomes 12 [1]; x stays 3 [1]; print 3,12 [1].

Q3 / 6 MARKS

Correct procedure name/parameters [1]; bill marked byRef [1]; extra byVal [1]; reject negative extra [1]; add extra to bill only otherwise [1]; correct call and explanation that the caller bill changes [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
procedure addCharge(bill:byRef, extra:byVal)
    if extra >= 0 then
        bill = bill + extra
    endif
endprocedure
amount = 20; addCharge(amount,5)  // 25
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.

UNDERSTAND / 1 OF 4

2D arrays: indices and boundaries

Appendix 5d uses zero-based arrays, including [row,column] 2D notation. The loop limits must match the declared dimensions. Trace a concrete grid rather than assuming index 1 is the first cell.

KEY IDEAS AND APPENDIX CONVENTIONS

Dimensions

array grid[3,4] has three rows 0..2 and four columns 0..3.

2D indexing

grid[r,c] selects one value. In OCR-style examples, do not silently change to the Python form grid[r][c].

Nested loops

Outer row 0..2; inner column 0..3. This visits 12 elements. Check the final legal index.

EXAM-STYLE PSEUDOCODE

```
array grid[3,4]  // 3 rows, 4 columns
grid[0,0] = 7
grid[2,3] = 9
for r = 0 to 2
    for col = 0 to 3
        print(grid[r,col])
    next col
next r
```

MODEL / 2 OF 4

Count values in an entire grid

A complete worked model: follow the code, then track each intermediate state before the final output.

```
function countMatches(grid, target)
    count = 0
    for r = 0 to 2
        for col = 0 to 3
            if grid[r,col] == target then
                count = count + 1
            endif
        next col
    next r
    return count
endfunction
```

WORKED TRACE / THINK ALOUD

1. A 3x4 grid contains 12 indexed positions.

2. The first cell is [0,0], not [1,1].

3. After [0,3] the next cell visited is [1,0].

4. The last legal cell is [2,3]; [3,0] is out of bounds.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

Give the first and last valid row and column indexes for array grid[3,4].

QUESTION 2 / 5 MARKS

A 3 by 4 array has target values in positions [0,1], [1,3] and [2,0]. Trace countMatches and give its return value.

QUESTION 3 / 6 MARKS

Write OCR-style nested loops to set every cell of array board[2,5] to 0. Include both correct bounds and a brief explanation of why every cell is reached.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

Rows 0..2 [1]; columns 0..3 [1]; last valid position is [2,3] and first [0,0] [1].

Q2 / 5 MARKS

Initial count 0 [1]; visit rows 0,1,2 [1]; count increases at [0,1] [1], [1,3] [1], [2,0] and final return 3 [1].

Q3 / 6 MARKS

Outer row loop 0..1 [1]; inner column loop 0..4 [1]; write board[r,col]=0 [1]; correct nesting [1]; 2x5=10 cells [1]; no negative/out-of-range indexes [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
array board[2,5]
for r = 0 to 1
    for col = 0 to 4
        board[r,col] = 0
    next col
next r
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.

UNDERSTAND / 1 OF 4

File handling: read, write and close

Appendix 5d shows file operations as methods on a file object: openRead, readLine, endOfFile, openWrite, writeLine and close. These conventions are useful in applied Paper 2 scenarios.

KEY IDEAS AND APPENDIX CONVENTIONS

Read a file

myFile=openRead("data.txt") then myFile.readLine() to retrieve text. Use an end-of-file test for a loop.

Write a file

myFile=openWrite("result.txt") then myFile.writeLine(text). Opening for writing replaces existing contents in the OCR model.

Finish safely

Close every opened file. Do not read after endOfFile() is reached.

EXAM-STYLE PSEUDOCODE

```
source = openRead("scores.txt")
while NOT source.endOfFile()
    line = source.readLine()
    print(line)
endwhile
source.close()

out = openWrite("status.txt")
out.writeLine("complete")
out.close()
```

MODEL / 2 OF 4

Copy each line without losing the final line

A complete worked model: follow the code, then track each intermediate state before the final output.

```
source = openRead("input.txt")
dest = openWrite("output.txt")
while NOT source.endOfFile()
    line = source.readLine()
    dest.writeLine(line)
endwhile
source.close()
dest.close()
```

WORKED TRACE / THINK ALOUD

1. For source lines "A" then "B", the first loop writes "A".
2. The second loop writes "B".
3. The following test finds endOfFile() true: loop ends.
4. Both files are closed; output has the two lines in order.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

Name the OCR-style operations used to open for reading, read one line and close a file.

QUESTION 2 / 5 MARKS

A file contains the lines Oak, Elm, Ash. Trace the model loop: how many times does it run and what appears in output.txt? Include the final end-of-file decision.

QUESTION 3 / 6 MARKS

Write OCR-style pseudocode that copies every line from notes.txt to backup.txt, converting none of the text, and closes both files. Explain one consequence of openWrite.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

openRead [1]; readLine [1]; close [1].

Q2 / 5 MARKS

Three loop iterations [1]; Oak copied [1]; Elm copied [1]; Ash copied [1]; endOfFile becomes true then loop stops, without a fourth read [1].

Q3 / 6 MARKS

Open notes with openRead [1]; backup with openWrite [1]; loop until endOfFile [1]; read and write each line [1]; close both files [1]; openWrite overwrites existing destination contents [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
f = openRead("notes.txt")
out = openWrite("backup.txt")
while NOT f.endOfFile()
    out.writeLine(f.readLine())
endwhile
f.close(); out.close()
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.

UNDERSTAND / 1 OF 4

OOP: constructors and inheritance

OCR Appendix 5d models constructors as new, subclasses with inherits and calls to the parent constructor with super.new(...). Practise writing code that initialises both inherited and new attributes.

KEY IDEAS AND APPENDIX CONVENTIONS

Class and attributes

Use class and endclass, with private / public where the question needs access control.

Constructor

A public procedure named new initialises the object when created.

Inheritance

Subclass header uses inherits; super.new(...) calls the parent constructor to initialise inherited state.

EXAM-STYLE PSEUDOCODE

```
class Device
    private label
    public procedure new(givenLabel)
        label = givenLabel
    endprocedure
endclass

class Sensor inherits Device
    private limit
    public procedure new(labelIn,limitIn)
        super.new(labelIn)
        limit = limitIn
    endprocedure
endclass
```

MODEL / 2 OF 4

Construct a Sensor with inherited state

A complete worked model: follow the code, then track each intermediate state before the final output.

```
class Sensor inherits Device
    private limit
    public procedure new(labelIn,limitIn)
        super.new(labelIn)
        limit = limitIn
    endprocedure
    public function tooHigh(value)
        return value > limit
    endfunction
endclass
s = new Sensor("S2",50)
```

WORKED TRACE / THINK ALOUD

1. new Sensor("S2",50) calls Sensor.new.
2. super.new("S2") initialises parent Device label="S2".
3. Sensor stores limit=50 as its own private attribute.
4. s.tooHigh(60) returns true; 50 or lower returns false.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

Name the three OCR-style terms used to declare a subclass, a constructor and a call to the parent constructor.

QUESTION 2 / 5 MARKS

Trace creation of Sensor("S2",50): state the two stored attributes and the results of tooHigh(60), tooHigh(50), and tooHigh(12).

QUESTION 3 / 6 MARKS

Write an OCR-style subclass AlertSensor that inherits Sensor. Add a private message, pass label and limit to its parent constructor, store message, and provide a public getMessage() function.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

inherits [1]; new [1]; super.new(...) [1].

Q2 / 5 MARKS

Label S2 is set by parent [1]; limit 50 set by subclass [1]; tooHigh(60) true [1]; tooHigh(50) false [1]; tooHigh(12) false [1].

Q3 / 6 MARKS

Correct class name/inherits Sensor [1]; private message [1]; constructor new has three arguments [1]; super.new(labelIn,limitIn) [1]; assign message from supplied argument [1]; public getter correctly returns message [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
class AlertSensor inherits Sensor
    private message
    public procedure new(limLabel, lim, msg)
        super.new(limLabel, lim)
        message = msg
    endprocedure
    public function getMessage()
        return message
    endfunction
endclass
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.

2.3 / MASTER CONTENTS

Choose a learning cycle

Each topic has one concept page, one worked model, exam-style practice and a separate answer page.

2.3.1(a)	Design an algorithm from a brief	7-10
2.3.1(a,b)	Choose an algorithm for the situation	11-14
2.3.1(b,c)	Analyse time and space costs	15-18
2.3.1(c,d)	Recognise constant, log and linear	19-22
2.3.1(c,d)	Compare polynomial and exponential	23-26
2.3.1(e)	Trace a stack: LIFO	27-30
2.3.1(e)	Trace a queue: FIFO	31-34
2.3.1(e)	Follow a linked list	35-38
2.3.1(e)	Search a binary search tree	39-42
2.3.1(e)	Depth-first post-order traversal	43-46

NAVIGATION

2.3 / MASTER CONTENTS

Choose a learning cycle

Each topic has one concept page, one worked model, exam-style practice and a separate answer page.

2.3.1(e)	**Breadth-first traversal of a tree**	47-50
2.3.1(f)	**Linear search**	51-54
2.3.1(f)	**Binary search**	55-58
2.3.1(f)	**Bubble sort**	59-62
2.3.1(f)	**Insertion sort**	63-66
2.3.1(f)	**Merge sort**	67-70
2.3.1(f)	**Quick sort**	71-74
2.3.1(f)	**Dijkstra: shortest known distances**	75-78
2.3.1(f)	**A*: use a lower-bound heuristic**	79-82
2.3.1(a-f)	**Synoptic algorithm decisions**	83-86

NAVIGATION

2.3 / INDEPENDENT REVISION

How to use each four-page cycle

A predictable routine for learners reading independently, revisiting lessons or preparing for assessments.

01 UNDERSTAND

Read the explanation, follow the visual and recall highlighted vocabulary.

02 MODEL

Trace the worked example one change at a time; explain why each step happens.

03 TRY

Answer in black pen before viewing the next page. Show intermediate workings.

04 CHECK

Self-mark in green; correct one misconception and explain the improvement.

school house style before publication.

2.3 / ALGORITHMS

Understand it. Model it. Try it. Check it.

This chapter spans OCR H446 2.3.1(a-f) and connects algorithm design, data structures, complexity and standard algorithms.

2.3.1(a,b)

Design and suitability

Requirements, constraints, correct choices.

2.3.1(c,d)

Efficiency

Constant, logarithmic, polynomial and exponential growth.

2.3.1(e)

Data structures

Stacks, queues, linked lists, trees, DFS and BFS.

2.3.1(f)

Standard algorithms

Linear/binary search; four sorts; Dijkstra and A*.

Try each exam question before using the separate indicative answers.

2.3 / SPECIFICATION CHECKLIST

Check every OCR requirement

The OCR items are summarised here. Page ranges are for this chapter PDF; tick only after applying each idea.

2.3.1(a)	Analyse/design algorithms	7-14
2.3.1(b)	Choose for data, time and space	11-18
2.3.1(c)	Efficiency, Big O and measurement	15-26
2.3.1(d)	Compare complexity classes	19-26
2.3.1(e)	Stacks and queues	27-34
2.3.1(e)	Linked lists and trees	35-42
2.3.1(e)	Post-order DFS and BFS traversal	43-50
2.3.1(f)	Linear and binary search	51-58
2.3.1(f)	Bubble and insertion sort	59-66
2.3.1(f)	Merge and quick sort	67-74
2.3.1(f)	Dijkstra and A*	75-82
2.3.1(a-f)	Synoptic selection/justification	83-86

OFFICIAL SOURCE

Use the examples to practise applying each specification requirement.

2.3.1(A) / SEE IT / CONCEPT AND VISUAL

Design an algorithm from a brief

An algorithm is a finite, unambiguous sequence of steps. Begin by deciding what enters, what leaves and which cases can break the proposed solution.

INPUT

Data supplied, its type and permitted range.

OUTPUT

The required result and its format.

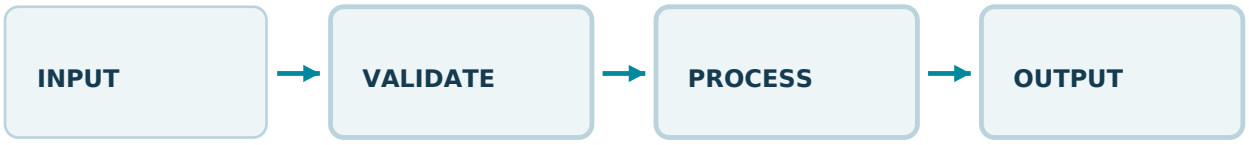
RULE

Precisely which transformation is needed.

TEST

Normal, boundary and invalid cases.

HOW THE PARTS FIT TOGETHER



INPUT -> VALIDATE -> PROCESS -> OUTPUT

VISUAL MEMORY BRIDGE

Think of a recipe: ingredients are inputs, the method is the algorithm, and the finished dish is the output. A good recipe also says what to do when an ingredient is missing.

WORDS THAT EARN MARKS

Algorithm	Finite steps	**Precondition**	Required before
Postcondition	Required after	**Boundary**	Edge value

EXAM TRAP

Do not jump from a scenario straight to code; unspecified inputs and failure cases become hidden bugs.

2.3.1(A) / WORKED EXAMPLE / STEP BY STEP

Design a delivery-charge algorithm

Orders of at least GBP 40 have free delivery; smaller orders pay GBP 4.50. Negative prices are invalid.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Input	Read subtotal (number)	Identify the single needed input.
2 Validate	If subtotal < 0, reject	An invalid value must not be charged.
3 Branch	If subtotal >= 40 then delivery = 0	The boundary is inclusive.
4 Else	Otherwise delivery = 4.50	Handle valid orders below the limit.
5 Calculate	total = subtotal + delivery	Return the complete bill.
6 Test	39.99, 40.00, -1	Check both sides and invalid input.

EXPLAIN THE METHOD

State the inclusive threshold explicitly and make error handling part of the design.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(A) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Identify the input, output and one precondition for the delivery-charge algorithm.

QUESTION 2 / 5 MARKS

Trace the charge and final total for subtotals 39.99 and 40.00. Explain the difference.

QUESTION 3 / 6 MARKS

Write pseudocode that validates subtotal, chooses the delivery charge and outputs either INVALID or the final total.

2.3.1(A) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Input: order subtotal [1].
- Output: total payable or an error indication [1].
- Precondition: subtotal is a numeric value, or otherwise is validated [1].

Q2 / 5 MARKS

- At 39.99, delivery = 4.50 [1], total = 44.49 [1].
- At 40.00, delivery = 0 [1], total = 40.00 [1].
- The condition uses >=, including exactly 40.00 [1].

Q3 / 6 MARKS

- INPUT subtotal [1].
- IF subtotal < 0 THEN [1].
- OUTPUT "INVALID" [1].
- ELSE determine whether subtotal >= 40 [1].
- Select delivery 0 or 4.50 as appropriate [1].
- OUTPUT subtotal + delivery [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Check that the output and boundary test are both explicit.

2.3.1(A,B) / SEE IT / CONCEPT AND VISUAL

Choose an algorithm for the situation

The best algorithm depends on the task, input properties and resource limits. Correctness comes first; time and memory trade-offs then determine suitability.

CORRECT?

Produces the required answer for all allowed cases.

INPUT

Size, ordering and update frequency.

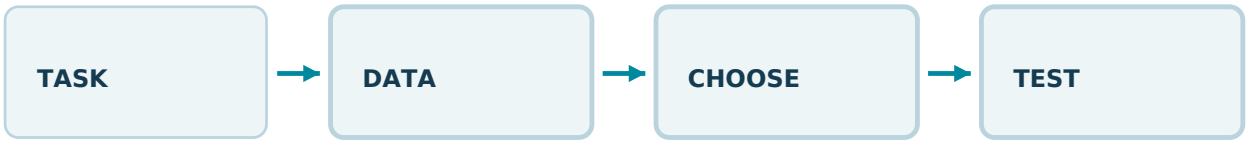
TIME

Work done as input size increases.

SPACE

Extra storage required.

HOW THE PARTS FIT TOGETHER



SORTED DATA? YES -> BINARY SEARCH NO -> LINEAR SEARCH

VISUAL MEMORY BRIDGE

Choose a route based on road conditions: a motorway is fast only when it is available. Likewise binary search is fast only when the data is already ordered.

WORDS THAT EARN MARKS

Constraint	Limiting condition	**Sorted**	Ordered data
Trade-off	One cost vs another	**Correctness**	Always right

EXAM TRAP

Do not call binary search the best search for an unsorted list unless sorting cost is justified.

2.3.1(A,B) / WORKED EXAMPLE / STEP BY STEP

Pick a search for a one-off lookup

A class list has 30 unsorted names and a teacher wants to check one name once.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Inspect	30 names; not sorted	Record the actual data condition.
2 Option A	Linear search: scan until found	Works without preprocessing.
3 Option B	Sort then binary search	Adds a sort for only one query.
4 Compare	One small search vs sort + search	Include setup work.
5 Select	Linear search	Simpler and appropriate here.
6 Reconsider	For many future searches, maintain sorted order	Changed constraints can change the choice.

EXPLAIN THE METHOD

Suitability is a justified decision, not a memorised ranking.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(A,B) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State three factors affecting the choice between searching algorithms.

QUESTION 2 / 5 MARKS

Explain why linear search may be preferable for a single search of 30 unsorted names.

QUESTION 3 / 6 MARKS

A million records are kept sorted and searched thousands of times each day. Recommend a search and justify it against an alternative, including a limitation.

2.3.1(A,B) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Whether the data is sorted [1].
- Number of items or data size [1].
- Number of searches / time or memory constraints [1].

Q2 / 5 MARKS

- Linear search can run on unsorted data [1].
- It needs no sorting step [1].
- The list is small [1].
- There is only one search [1].
- Sorting first would incur extra time / complexity [1].

Q3 / 6 MARKS

- Binary search is suitable because records are sorted [1].
- It repeatedly halves the remaining range [1].
- Time is O(log n) per lookup [1].
- Thousands of repeated lookups make this valuable [1].
- Linear search may require O(n) comparisons [1].
- Maintaining sort order on insertions can be costly / binary search requires random access [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Separate the cost of preparation from the cost of a single search.

2.3.1(B,C) / SEE IT / CONCEPT AND VISUAL

Analyse time and space costs

Time complexity counts how work grows with input size n; auxiliary space measures additional memory used beyond the input. Execution timings can help but depend on hardware.

INPUT SIZE

Define n before counting operations.

TIME

Count dominant operations or passes.

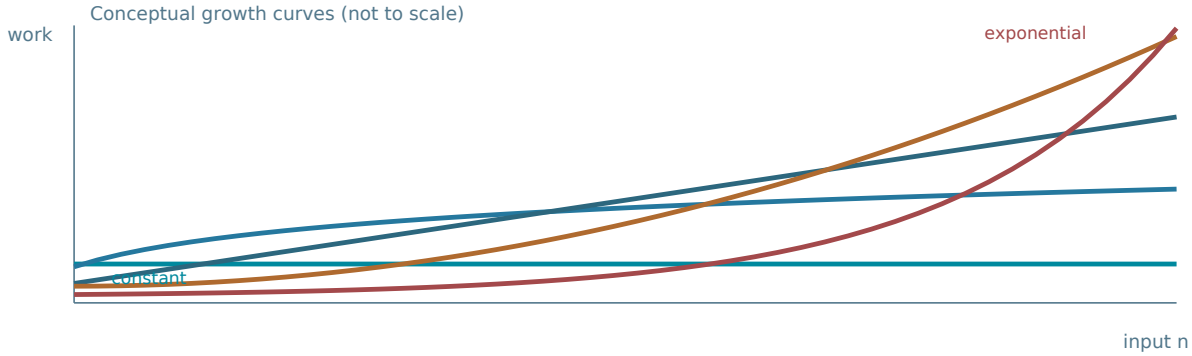
SPACE

Count extra arrays, stack frames or buffers.

EMPIRICAL

Measure repeated runs with fair conditions.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

Packing a suitcase: time is how many items you inspect; extra space is how many spare bags you need. Counting them separately prevents misleading comparisons.

WORDS THAT EARN MARKS

n	Input size	**Time**	Growth of work
Auxiliary	Extra storage	**Worst case**	Maximum work

EXAM TRAP

O(1) space does not mean zero memory; it means extra storage does not grow with n.

2.3.1(B,C) / WORKED EXAMPLE / STEP BY STEP

Count a nested loop

FOR i = 1 TO n: FOR j = 1 TO n: comparisons = comparisons + 1.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Define	n = loop bound	The same n controls both loops.
2 Inner	n iterations per outer pass	For each i, j runs n times.
3 Outer	n complete passes	There are n values of i.
4 Count	n x n = n^2 increments	Multiply, do not add.
5 Time	O(n^2)	The dominant term is quadratic.
6 Space	O(1) auxiliary	Only the loop variables and counter are stored.

EXPLAIN THE METHOD

Use a trace to explain the count; then express its growth using Big O.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(B,C) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define input size, time complexity and auxiliary space.

QUESTION 2 / 5 MARKS

For n = 4, count the inner-loop increments in the given nested loop and explain the general expression.

QUESTION 3 / 6 MARKS

An algorithm copies all n items to a new array, then scans the array once. Determine its time and auxiliary space complexities with reasons.

2.3.1(B,C) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- n is a measurable amount of input [1].
- Time complexity describes growth of required operations [1].
- Auxiliary space describes additional memory used as input grows [1].

Q2 / 5 MARKS

- Outer loop runs four times [1].
- Inner loop runs four times each pass [1].
- There are 16 increments [1].
- In general n x n = n^2 [1].
- Time complexity is O(n^2) [1].

Q3 / 6 MARKS

- Copy loop processes each item once, O(n) [1].
- Scan processes each item once, O(n) [1].
- Together 2n grows as O(n) [1].
- New array holds n values [1].
- Auxiliary space is O(n) [1].
- Sequential loops add rather than multiply [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. When counting loops, distinguish nested loops from consecutive loops.

2.3.1(C,D) / SEE IT / CONCEPT AND VISUAL

Recognise constant, log and linear

Big O describes an upper bound on asymptotic growth. Constant time does not grow with n; logarithmic growth rises slowly; linear work grows proportionally with n.

O(1)

A fixed number of operations.

O(log n)

Repeatedly shrink the problem.

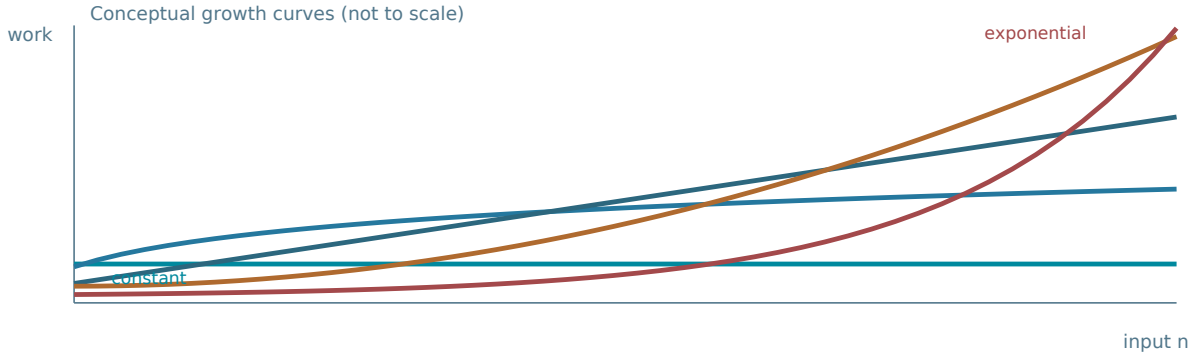
O(n)

Visit a number of items proportional to n.

ASSUMPTION

Know whether data is indexed or sorted.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

A library: take a labelled book from a known shelf slot (constant); use a sorted catalogue by halving ranges (logarithmic); check every spine (linear).

WORDS THAT EARN MARKS

O(1)	Constant	**O(log n)**	Logarithmic
O(n)	Linear	**Asymptotic**	As n grows

EXAM TRAP

Do not infer O(log n) from a short run: the algorithm must actually reduce the candidate set each step.

2.3.1(C,D) / WORKED EXAMPLE / STEP BY STEP

Compare work for 1,024 items

Assume a list provides direct indexed access and a binary-search list is sorted.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Constant	Access index 37: one lookup	Position is already known.
2 Linear	Worst-case scan: 1,024 checks	Last item may be the target.
3 Binary	Halve 1,024 -> 512 -> ... -> 1	Ten halvings reduce 1,024 to one.
4 Classify	O(1), O(n), O(log n)	Classify growth, not milliseconds.
5 Preconditions	Binary search requires sorted data	Otherwise the discard decision fails.
6 Distinguish	Worst-case count depends on algorithm	All three can be correct for different tasks.

EXPLAIN THE METHOD

Being faster for one measured n is not the definition of a better Big O class.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(C,D) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Match indexed access, linear search and binary search (sorted list) to their Big O time classes.

QUESTION 2 / 5 MARKS

Explain why binary search uses roughly 10 halvings on 1,024 ordered items and why this is logarithmic.

QUESTION 3 / 6 MARKS

Compare linear and binary search for a 1,024-item list when data is unsorted and used once; include the sorting prerequisite and overall work.

2.3.1(C,D) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Indexed access is O(1) [1].
- Linear search worst case is O(n) [1].
- Binary search worst case is O(log n) [1].

Q2 / 5 MARKS

- 1,024 = 2^10 [1].
- Each comparison removes about half the candidates [1].
- Ten halvings leave one candidate [1].
- Doubling n adds only about one extra halving [1].
- This growth is O(log n) [1].

Q3 / 6 MARKS

- Binary search requires the list to be sorted [1].
- An unsorted list must first be sorted or maintained sorted [1].
- This setup has a cost [1].
- Linear search works immediately on unsorted data [1].
- Worst-case linear work is O(n) [1].
- For one lookup, sorting plus binary search may not be worth it [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Explain why halving gives logarithmic growth, not just the label.

2.3.1(C,D) / SEE IT / CONCEPT AND VISUAL

Compare polynomial and exponential

Quadratic work is polynomial; doubling choices at every step may create exponential work. For large n an exponential algorithm can become infeasible.

O(n^2)

A common nested-loop growth rate.

O(n^3)

Polynomial, but steeper than quadratic.

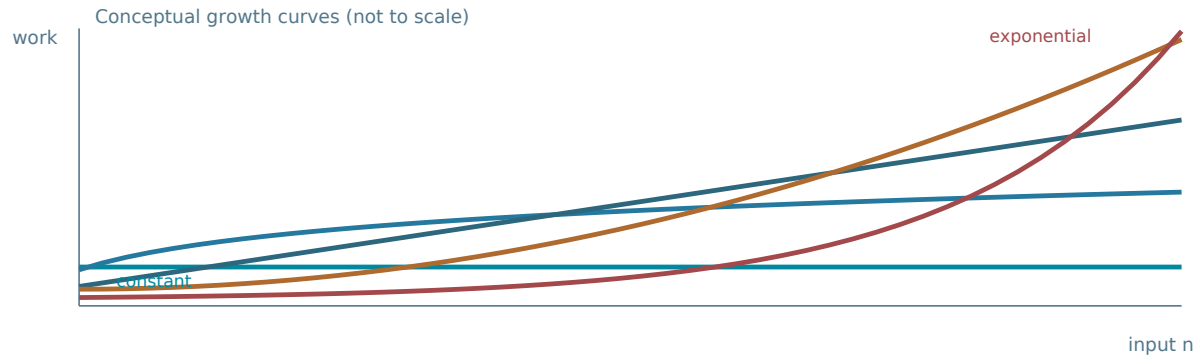
O(2^n)

Exponential doubling with each extra input.

COMPARE

Ignore constants for asymptotic class.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

A seating chart: comparing every person with every person grows roughly as a square; considering every yes/no invitation pattern doubles the possibilities with each guest.

WORDS THAT EARN MARKS

Quadratic	n squared	**Polynomial**	n to fixed power
Exponential	Base to n	**Scalability**	Larger n

EXAM TRAP

O(2^n) is not the same as O(n^2); compare what is the exponent and what is the base.

2.3.1(C,D) / WORKED EXAMPLE / STEP BY STEP

Estimate growth before implementation

Two approaches require n^2 operations and 2^n operations respectively.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Input	n = 8	Select a comparison size.
2 Quadratic	8^2 = 64	A polynomial number of operations.
3 Exponential	2^8 = 256	All binary choices across eight inputs.
4 Increase	n = 12	Four more input items.
5 Quadratic	12^2 = 144	A moderate increase.
6 Exponential	2^12 = 4,096	A 16-fold increase for four more items.

EXPLAIN THE METHOD

Complexity classes describe scaling; real timing also depends on constants and implementation.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(C,D) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Classify O(n^2), O(n^3) and O(2^n) as polynomial or exponential.

QUESTION 2 / 5 MARKS

Calculate n^2 and 2^n for n = 5 and n = 10, and state which grows faster.

QUESTION 3 / 6 MARKS

A brute-force algorithm tests every subset of n choices. Explain why its worst-case work is exponential and suggest one strategy for reducing practical search.

2.3.1(C,D) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- O(n^2) is polynomial [1].
- O(n^3) is polynomial [1].
- O(2^n) is exponential [1].

Q2 / 5 MARKS

- At n=5: n^2 = 25 [1], 2^n = 32 [1].
- At n=10: n^2 = 100 [1], 2^n = 1,024 [1].
- Exponential grows faster for large n [1].

Q3 / 6 MARKS

- Each input can be included or excluded [1].
- There are 2^n subsets [1].
- Naively trying each subset costs at least O(2^n) [1].
- This scales quickly as n increases [1].
- Pruning/backtracking may discard impossible branches [1].
- Memoisation or another problem-specific algorithm can avoid repeated work [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. State that polynomial and exponential refer to functions of n.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Trace a stack: LIFO

A stack inserts and removes items at one end called the top. PUSH and POP are normally O(1) using a linked stack or dynamic array with amortised append.

PUSH(x)

Place x on top.

POP()

Remove and return top.

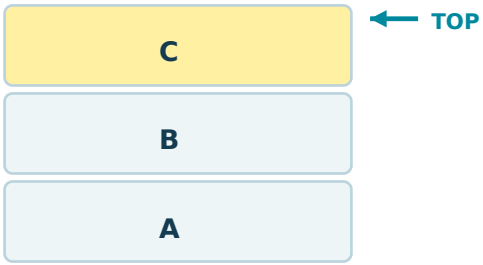
PEEK()

Read top without removing.

EMPTY

Guard against stack underflow.

HOW THE PARTS FIT TOGETHER



PUSH / POP at the TOP • Last in, first out

VISUAL MEMORY BRIDGE

Like a stack of trays: take the latest tray placed at the top before those underneath.

WORDS THAT EARN MARKS

Stack	LIFO	**Push**	Insert top
Pop	Remove top	**Underflow**	Remove from empty

EXAM TRAP

POP returns the most recently pushed item still on the stack, not the earliest.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Simulate six stack actions

Start with an empty stack; represent contents bottom-to-top.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 PUSH A	[A]	A is the first item and the bottom.
2 PUSH B	[A, B]	B becomes the top.
3 PUSH C	[A, B, C]	C becomes the top.
4 POP	returns C; [A, B]	LIFO removes newest.
5 PUSH D	[A, B, D]	D is now at top.
6 PEEK	returns D; unchanged	Peek does not remove the item.

EXPLAIN THE METHOD

Draw the stack after every operation; record both returned values and remaining contents.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define LIFO and the operations PUSH and POP.

QUESTION 2 / 5 MARKS

Starting empty, trace PUSH 4, PUSH 7, POP, PUSH 9, POP. Give returned values and final contents.

QUESTION 3 / 6 MARKS

Write pseudocode for safe POP on an array-backed stack with topIndex = -1 when empty; explain what happens if it is empty.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- LIFO means last in, first out [1].
- PUSH inserts at the top [1].
- POP removes/returns the top [1].

Q2 / 5 MARKS

- PUSH 4 -> stack contains 4 [1].
- PUSH 7 -> [4,7] [1].
- First POP returns 7 [1].
- PUSH 9 -> [4,9] [1].
- Second POP returns 9; only 4 remains [1].

Q3 / 6 MARKS

- IF topIndex = -1 THEN [1].
- Signal EMPTY/UNDERFLOW without removing [1].
- ELSE read stack[topIndex] [1].
- Decrease topIndex by 1 [1].
- Return stored item [1].
- Explain that attempting POP when empty must not access an invalid index [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep TOP and returned items separate in the trace.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Trace a queue: FIFO

A queue inserts at the rear and removes at the front. ENQUEUE and DEQUEUE support first-in-first-out service; a circular array avoids shifting every item.

ENQUEUE(x)

Insert at rear.

DEQUEUE()

Remove from front.

FRONT

Item waiting the longest.

CIRCULAR

Wrap array indices when needed.

HOW THE PARTS FIT TOGETHER



DEQUEUE at FRONT • ENQUEUE at REAR

VISUAL MEMORY BRIDGE

Like a ticket queue: the person who joined first is served first, unless a separate priority rule is introduced.

WORDS THAT EARN MARKS

Queue	FIFO	**Enqueue**	Add at rear
Dequeue	Remove front	**Overflow**	Queue full

EXAM TRAP

Do not implement a fixed-array dequeue by always shifting elements if O(1) operations are required; use head/tail pointers.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Simulate a circular queue of capacity four

Empty queue, front=0, rear=0; rear is the next free slot.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 ENQ A	[A]; rear=1	Write at index 0, then advance.
2 ENQ B	[A,B]; rear=2	B goes behind A.
3 DEQ	returns A; front=1	Remove oldest item.
4 ENQ C	[B,C]; rear=3	C follows B in logical order.
5 ENQ D	[B,C,D]; rear=0	Index wraps modulo four.
6 DEQ	returns B; front=2	Only logical front moves.

EXPLAIN THE METHOD

Track front, rear and number of items; wrapping an index does not rearrange logical order.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define FIFO and distinguish ENQUEUE from DEQUEUE.

QUESTION 2 / 5 MARKS

Starting empty, trace ENQUEUE 3, ENQUEUE 6, DEQUEUE, ENQUEUE 8, DEQUEUE. Give outputs and remaining data.

QUESTION 3 / 6 MARKS

Explain how a circular queue with array capacity 5 can provide O(1) enqueue/dequeue and identify two error conditions.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- FIFO means first in, first out [1].
- ENQUEUE inserts at the rear [1].
- DEQUEUE removes from the front [1].

Q2 / 5 MARKS

- After first two adds [3,6] [1].
- First DEQUEUE returns 3 [1].
- ENQUEUE 8 leaves [6,8] [1].
- Second DEQUEUE returns 6 [1].
- Only 8 remains in the queue [1].

Q3 / 6 MARKS

- Maintain front pointer [1] and rear pointer [1].
- Update indices modulo capacity [1].
- No O(n) shifting is required, so each operation can be O(1) [1].
- Underflow if queue empty [1].
- Overflow if array queue full [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Follow the logical order independently of physical array indices.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Follow a linked list

A linked-list node stores data and a reference to the next node. Traversal follows links; inserting at a known head can be O(1), while finding a value is O(n).

NODE

Data and next pointer.

HEAD

Reference to first node.

TRAVERSE

Follow next until null.

INSERT

Update links without losing nodes.

HOW THE PARTS FIT TOGETHER



Each arrow stores the location of the next node, not a fixed index.

VISUAL MEMORY BRIDGE

A treasure hunt: each clue gives the next location. You cannot jump to clue 12 without following the chain.

WORDS THAT EARN MARKS

Node	Value plus link	**Head**	First node
Null	No successor	**Pointer**	Reference

EXAM TRAP

When inserting between nodes, preserve the old successor before changing the links.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Insert 7 between linked nodes 4 and 9

Initially HEAD -> 4 -> 9 -> 12 -> NULL; current points at the node containing 4.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Allocate	new.data = 7	Create the new node.
2 Preserve	new.next = current.next	The new node points to 9.
3 Relink	current.next = new	The node 4 now points to 7.
4 Trace	4 -> 7 -> 9 -> 12	No old node is lost.
5 Find 12	Check 4,7,9,12	Following links is linear.
6 Complexity	Known-position insert O(1)	Searching for the position can be O(n).

EXPLAIN THE METHOD

A linked list supports cheap pointer updates but lacks O(1) indexing by position.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State the components of a singly linked-list node and the purpose of HEAD.

QUESTION 2 / 5 MARKS

Show link changes needed to insert a node 7 after node 4 in 4 -> 9 -> NULL.

QUESTION 3 / 6 MARKS

Compare inserting at the head of a linked list with accessing the kth item of an array or linked list in time-complexity terms.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Each node contains data [1].
- Each node contains a next reference [1].
- HEAD identifies the first node [1].

Q2 / 5 MARKS

- Create a new node containing 7 [1].
- Set new.next to node 9 [1].
- Set node 4 next to new [1].
- Final chain 4 -> 7 -> 9 -> NULL [1].
- Old successor must not be lost [1].

Q3 / 6 MARKS

- Head insertion: set new.next=head [1], head=new [1].
- Known-head insertion is O(1) [1].
- Array random indexed access is O(1) [1].
- Linked-list kth-item access follows k links, O(n) worst case [1].
- Trade-off: pointers use extra storage / arrays may require shifting on insert [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Show where each next reference points after the operation.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Search a binary search tree

In a binary search tree (BST), values smaller than a node go left and larger values go right (with a declared duplicate policy). Height determines search performance.

ROOT

Top of the tree.

LEFT

Smaller subtree for a BST.

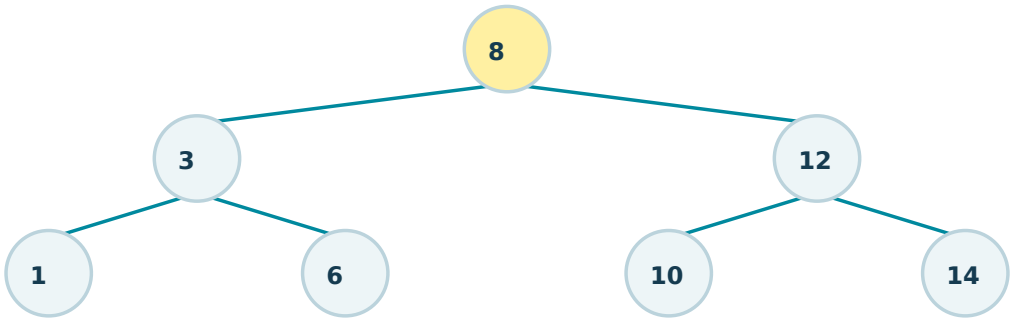
RIGHT

Larger subtree for a BST.

HEIGHT

Longest root-to-leaf distance.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

Like a choose-your-own-path signpost: every comparison tells you which branch cannot hold the value.

WORDS THAT EARN MARKS

Root	Top node	**Leaf**	No children
BST	Ordered branches	**Balanced**	Shallow height

EXAM TRAP

BST search is O(log n) only when the tree height is logarithmic; an unbalanced chain can give O(n).

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Locate 10 in a binary search tree

Tree root 8, left child 3, right child 12; 12 has children 10 and 14.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Root	Compare 10 with 8	10 > 8, so choose right.
2 Right	Visit 12	10 < 12, so choose left.
3 Left	Visit 10	Value matches.
4 Path	8 -> 12 -> 10	Three node visits.
5 Balanced	Height grows slowly	Expected O(log n) on balanced BST.
6 Skewed	Chain-like tree	Worst-case search may be O(n).

EXPLAIN THE METHOD

Traversal order and BST search are different algorithms: one explores, the other uses ordering to prune.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define root, leaf and binary search tree.

QUESTION 2 / 5 MARKS

Trace the nodes visited when searching for 1 and then 14 in the tree described above.

QUESTION 3 / 6 MARKS

Explain why inserting sorted values into a plain BST can cause poor search performance, and give one way to avoid it.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Root is the entry/top node [1].
- Leaf has no children [1].
- BST places smaller values left and larger right (according to a duplicate rule) [1].

Q2 / 5 MARKS

- Search 1: 8 -> 3 -> 1 [2].
- Search 14: 8 -> 12 -> 14 [2].
- At each step comparison chooses one branch [1].

Q3 / 6 MARKS

- Sorted insertion may create a one-sided chain [1].
- Height can become O(n) [1].
- Search then visits up to n nodes [1].
- No halving is guaranteed [1].
- Use a self-balancing tree or rebuild to balance [1].
- A balanced tree keeps search roughly O(log n) [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Label the direction chosen at each comparison.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Depth-first post-order traversal

Post-order depth-first traversal visits every left subtree, then every right subtree, then the parent: LEFT, RIGHT, ROOT. It is often used when children must be processed first.

DEPTH FIRST

Explore a branch before siblings.

POST-ORDER

Left, right, root.

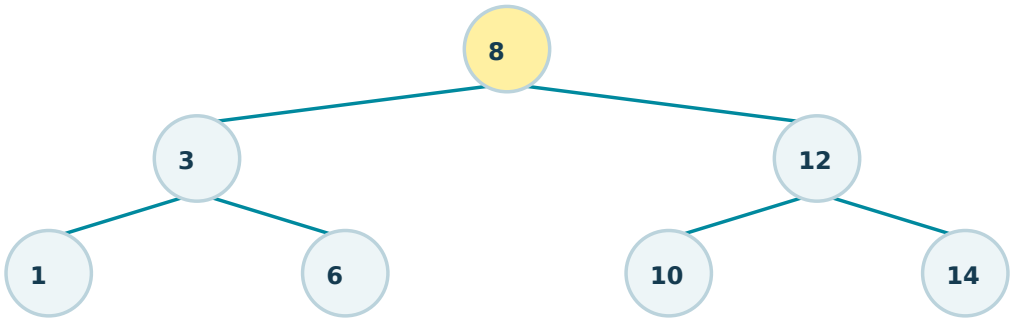
STACK

Call stack remembers unfinished nodes.

TREE

Visit each node once.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

Pack up a treehouse: clear each child room before you dismantle its parent platform.

WORDS THAT EARN MARKS

DFS	Depth-first search	**Post-order**	L-R-Root
Recursion	Self-calling	**Stack**	Pending work

EXAM TRAP

In post-order the root is last; preorder (root first) and inorder (root between) are different.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Produce the post-order list

Tree: root 8; children 3 and 12; 3 has children 1 and 6; 12 has children 10 and 14.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Left-left	Visit 1	Leaf has no subtrees.
2 Left-right	Visit 6	Second leaf under 3.
3 Left-root	Visit 3	Both children of 3 complete.
4 Right-left	Visit 10	First leaf under 12.
5 Right-right/root	Visit 14 then 12	Finish right subtree.
6 Root	Visit 8	Output: 1,6,3,10,14,12,8.

EXPLAIN THE METHOD

Write the root only after both of its subtrees have been fully traversed.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State the order of operations in recursive post-order traversal.

QUESTION 2 / 5 MARKS

Give the full post-order traversal for root A, with B and C as children; B has D,E and C has F.

QUESTION 3 / 6 MARKS

Write recursive pseudocode for post-order traversal and explain its time complexity for n nodes.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Traverse left subtree first [1].
- Traverse right subtree second [1].
- Process root last [1].

Q2 / 5 MARKS

- D then E [1].
- B follows its children [1].
- F then C [1].
- A is last [1].
- Full sequence D,E,B,F,C,A [1].

Q3 / 6 MARKS

- PROCEDURE post(node) [1].
- IF node != NULL THEN [1].
- post(node.left) [1].
- post(node.right) [1].
- OUTPUT node.value [1].
- Every node is visited once, so O(n) time [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Use the L-R-Root rule locally at each subtree, not only at the main root.

2.3.1(E) / SEE IT / CONCEPT AND VISUAL

Breadth-first traversal of a tree

Breadth-first traversal explores a tree level by level and uses a FIFO queue to remember discovered nodes. Do not confuse it with post-order DFS.

BFS

Explore current depth before next.

QUEUE

Hold discovered nodes.

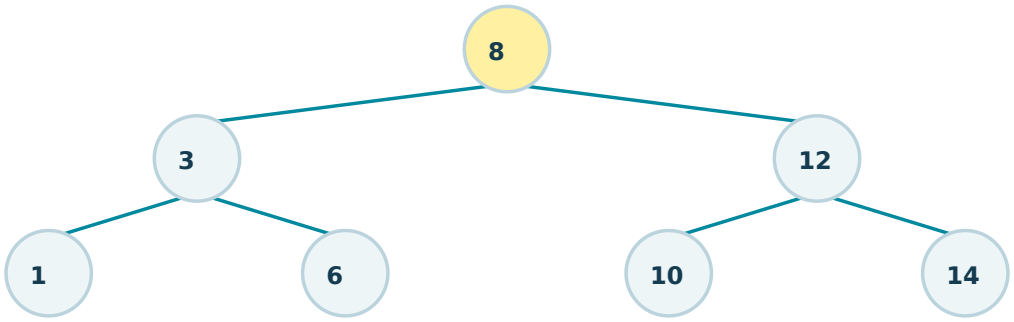
ENQUEUE

Add each child left-to-right.

VISIT

Process a node when removed.

HOW THE PARTS FIT TOGETHER



VISUAL MEMORY BRIDGE

A theatre fills one row at a time rather than following one aisle all the way to the back before returning.

WORDS THAT EARN MARKS

Breadth first	Level order	**FIFO**	Queue policy
Enqueue	Add rear	**Dequeue**	Remove front

EXAM TRAP

Using a stack instead of a queue usually changes the visit order into depth-first behaviour.

2.3.1(E) / WORKED EXAMPLE / STEP BY STEP

Trace BFS with the queue visible

Tree: root 8; children 3,12; 3 has 1,6; 12 has 10,14. Queue is shown front-to-rear.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Start	Queue [8]	Add root.
2 Visit 8	Queue [3,12]	Enqueue its children.
3 Visit 3	Queue [12,1,6]	Add 1 and 6 after 12.
4 Visit 12	Queue [1,6,10,14]	Add 10 and 14.
5 Visit leaves	1,6,10,14	No children to add.
6 Output	8,3,12,1,6,10,14	Nodes appear in depth order.

EXPLAIN THE METHOD

When tracing, record both the output sequence and the queue contents after each removal.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(E) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Name the supporting data structure and the visitation pattern used in breadth-first tree traversal.

QUESTION 2 / 5 MARKS

For tree root A, children B,C, and B has D,E, write the BFS visit order and the queue after visiting A.

QUESTION 3 / 6 MARKS

Write an outline of breadth-first tree traversal with a queue; justify O(n) time for n nodes.

2.3.1(E) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- BFS uses a queue [1].
- Queue is first in, first out [1].
- BFS processes one level before the next [1].

Q2 / 5 MARKS

- Visit order A,B,C,D,E [3].
- Queue after visiting A is [B,C] [2].

Q3 / 6 MARKS

- Add root to queue [1].
- WHILE queue not empty [1].
- Remove front node [1].
- Process/output that node [1].
- Enqueue its children [1].
- Each node is enqueued and removed once, O(n) time [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. After adding children, make sure earlier queued nodes stay at the front.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Linear search

Linear search checks elements one by one until a match is found or the list ends. It works for unsorted data and has O(n) worst-case time.

COMPARE

Test current item against target.

MATCH

Return position or found state.

NOT FOUND

Terminate after final item.

COMPLEXITY

Worst case O(n), extra space O(1).

HOW THE PARTS FIT TOGETHER



ARRAY [8, 3, 12, 5, 7] ; target 5 ; check 8,3,12,5

VISUAL MEMORY BRIDGE

Look for a lost pen by checking every desk, stopping as soon as you find it.

WORDS THAT EARN MARKS

Target	Search key	**Index**	Position
Sentinel	Not-found value	**Worst case**	Last or absent

EXAM TRAP

Avoid off-by-one errors: position 0 is the first element when using zero-based indexing.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Trace a zero-based linear search

items = [8,3,12,5,7] and target = 5.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 i=0	items[0] = 8: no	Continue.
2 i=1	items[1] = 3: no	Continue.
3 i=2	items[2] = 12: no	Continue.
4 i=3	items[3] = 5: yes	Match found.
5 Return	index 3	Do not return 4 for zero-based data.
6 Bound	Worst-case 5 comparisons	Missing target or last item.

EXPLAIN THE METHOD

State whether your result is an index or a human-friendly ordinal position.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State a benefit, worst-case time complexity and extra-space complexity of linear search.

QUESTION 2 / 5 MARKS

Trace comparisons when finding 5 in [8,3,12,5,7] and state the zero-based index returned.

QUESTION 3 / 6 MARKS

Write pseudocode to search an unsorted array of length n and return -1 when no match exists.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Works with unsorted data [1].
- Worst-case O(n) [1].
- Auxiliary space O(1) [1].

Q2 / 5 MARKS

- Checks 8 [1], 3 [1], 12 [1], 5 [1].
- Returns zero-based index 3 [1].

Q3 / 6 MARKS

- FOR i = 0 TO n-1 [1].
- IF arr[i] = target [1].
- RETURN i [1].
- End IF / loop correctly [1].
- RETURN -1 after loop [1].
- Explain -1 means not found, not a valid index [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. When not found, scan all valid indices before returning -1.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Binary search

Binary search repeatedly compares with the middle of an ordered range, discarding half. For a sorted indexable array, its worst-case time is O(log n).

SORTED

Required input property.

LOW / HIGH

Inclusive candidate bounds.

MID

(low + high) DIV 2.

HALVE

Discard the impossible half.

HOW THE PARTS FIT TOGETHER



SORTED [2,5,8,11,14,17,20]; target 17

VISUAL MEMORY BRIDGE

Find a word in a dictionary by opening near the middle and deciding which half cannot contain it.

WORDS THAT EARN MARKS

Sorted	Ordered	**Midpoint**	Middle index
DIV	Integer quotient	**Halving**	Logarithmic

EXAM TRAP

Do not apply binary search to unsorted data; also update low = mid+1 or high = mid-1 to avoid repeating a midpoint.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Trace a successful binary search

arr = [2,5,8,11,14,17,20], target = 17, indices 0..6.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Start	low=0, high=6	All seven items are candidates.
2 Mid	mid=(0+6) DIV 2=3	arr[3]=11.
3 Discard	17>11, low=4	Discard indices 0..3.
4 Mid	mid=(4+6) DIV 2=5	arr[5]=17.
5 Found	Return index 5	Two comparisons in this trace.
6 Absent	Terminate when low>high	There is no candidate left.

EXPLAIN THE METHOD

After each comparison, cross out the impossible half and update the inclusive bounds.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State a prerequisite for binary search, its worst-case Big O, and what causes it to terminate unsuccessfully.

QUESTION 2 / 5 MARKS

Trace binary search for 5 in [2,5,8,11,14,17,20]. Show mid indices and bound updates.

QUESTION 3 / 6 MARKS

Write iterative pseudocode for binary search using low, high and mid, returning -1 for absent values.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Array must be sorted [1].
- Worst-case O(log n) [1].
- Unsuccessful when low > high [1].

Q2 / 5 MARKS

- Start low=0, high=6, mid=3 with value 11 [1].
- 5 < 11 so high=2 [1].
- Next mid=1, value 5 [1].
- Match at index 1 [1].
- The discarded half cannot contain the value in sorted order [1].

Q3 / 6 MARKS

- Set low=0 and high=n-1 [1].
- WHILE low <= high [1].
- mid = (low+high) DIV 2 [1].
- If equal return mid [1].
- If target > arr[mid], low=mid+1; else high=mid-1 [1].
- RETURN -1 after loop [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Record low, high and mid on every iteration.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Bubble sort

Bubble sort compares neighbours, swapping them when out of order. Each completed pass moves a maximum to the right in ascending order.

COMPARE

Adjacent elements.

SWAP

Only if left > right for ascending.

PASS

Largest unsorted value moves right.

STOP

After no swaps, or all passes.

HOW THE PARTS FIT TOGETHER



PASS 1: [5,1,4,2] -> [1,4,2,5]

VISUAL MEMORY BRIDGE

Like bubbles rising: the largest value drifts to its final position at the end of each pass.

WORDS THAT EARN MARKS

Adjacent	Neighbours	**Swap**	Exchange
Pass	Sweep	**Stable**	Equal relative order

EXAM TRAP

Do not claim a full sort after one pass; only the largest remaining element is guaranteed to be placed.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Trace the first two bubble passes

Sort [5,1,4,2] in ascending order, swapping only on >.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Compare 5,1	[1,5,4,2]	Swap because 5>1.
2 Compare 5,4	[1,4,5,2]	Swap because 5>4.
3 Compare 5,2	[1,4,2,5]	5 reaches last position.
4 Compare 1,4	unchanged	Already ordered.
5 Compare 4,2	[1,2,4,5]	Swap; sorted result.
6 Bounds	Worst O(n^2), best O(n) with early exit	Nested comparisons; no-swap pass stops.

EXPLAIN THE METHOD

Show the array state after swaps; count passes separately from individual comparisons.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Describe an ascending bubble-sort comparison, swap rule and outer-pass purpose.

QUESTION 2 / 5 MARKS

Show the array [4,3,2,1] after the first complete bubble-sort pass and state how many swaps occur.

QUESTION 3 / 6 MARKS

Compare bubble sort and insertion sort for nearly sorted data, considering early-exit optimisations and worst-case complexity.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Compare adjacent items [1].
- Swap if left item is greater [1].
- Each pass positions a largest unsorted value at right [1].

Q2 / 5 MARKS

- 4,3 swaps -> [3,4,2,1] [1].
- 4,2 swaps -> [3,2,4,1] [1].
- 4,1 swaps -> [3,2,1,4] [1].
- Three swaps [1].
- Largest item 4 is final [1].

Q3 / 6 MARKS

- Both are O(n^2) in worst case [1].
- Bubble with no-swap early exit can be O(n) best case [1].
- Insertion sort often uses fewer shifts on nearly sorted data [1].
- Insertion best case O(n) [1].
- Both can be stable in standard versions [1].
- State that actual choice depends on input order/implementation [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Write each swap, then check the rightmost fixed portion.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Insertion sort

Insertion sort grows a sorted prefix. Take the next key and shift larger prefix values one position right until the correct slot is free.

PREFIX

Items before key are sorted.

KEY

Next unsorted value.

SHIFT

Move larger values right.

INSERT

Place key in its correct slot.

HOW THE PARTS FIT TOGETHER



SORTED [2,5,8] | KEY 4 -> [2,4,5,8]

VISUAL MEMORY BRIDGE

Organise playing cards in your hand by inserting each new card into the already sorted set.

WORDS THAT EARN MARKS

Key	Current item	**Prefix**	Sorted left part
Shift	Move one place	**In-place**	Low extra space

EXAM TRAP

A shift is not necessarily a swap; retain the key while moving larger items.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Insert keys into a sorted prefix

Sort [5,2,4,1] ascending, showing the state after each key.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Start	[5 | 2,4,1]	First item forms a sorted prefix.
2 Key 2	[2,5 | 4,1]	Shift 5 then insert 2.
3 Key 4	[2,4,5 | 1]	Shift 5; place 4.
4 Key 1	[1,2,4,5]	Shift 5,4,2; insert 1.
5 Best case	Already sorted -> O(n)	Each key needs one comparison.
6 Worst case	Reverse order -> O(n^2)	Many shifts for each key.

EXPLAIN THE METHOD

Draw the boundary between sorted prefix and unsorted suffix after every outer iteration.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Explain the purpose of the key, sorted prefix and shift in insertion sort.

QUESTION 2 / 5 MARKS

Trace insertion sort on [3,1,2,4], giving the array after inserting each key.

QUESTION 3 / 6 MARKS

Explain why insertion sort can be efficient for nearly sorted arrays but O(n^2) for reverse-sorted arrays.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Key is the next item to insert [1].
- Prefix contains already sorted items [1].
- Shift creates space by moving larger prefix items [1].

Q2 / 5 MARKS

- Key 1 -> [1,3,2,4] [2].
- Key 2 -> [1,2,3,4] [2].
- Key 4 leaves list unchanged [1].

Q3 / 6 MARKS

- Nearly sorted: few shifts needed [1].
- Outer loop visits n-1 keys [1].
- Best-case comparisons grow O(n) [1].
- Reverse sorted: each key moves past many values [1].
- Sum 1+2+...+(n-1) is quadratic [1].
- Worst-case time O(n^2) [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep the key safe while shifting values; do not lose an element.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Merge sort

Merge sort splits the sequence into halves, recursively sorts each half and merges them into order. Standard array versions run in O(n log n) time with O(n) auxiliary storage.

DIVIDE

Split array into smaller halves.

RECURSE

Sort the two halves.

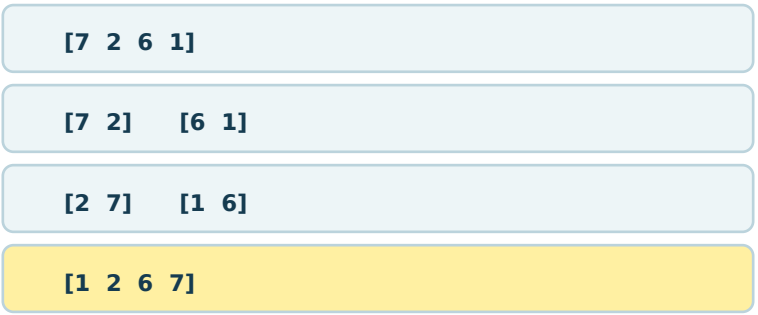
MERGE

Take the smaller next value.

SPACE

Temporary array for merging.

HOW THE PARTS FIT TOGETHER



The arrows show conceptual stages, not implementation-specific swaps.

VISUAL MEMORY BRIDGE

Two already sorted piles of exam papers can be combined by repeatedly taking the smallest paper from the two fronts.

WORDS THAT EARN MARKS

Divide and conque	Split and combine	**Merge**	Combine ordered
Stable	Preserve ties	**Auxiliary**	Extra array

EXAM TRAP

Merging works by comparing the fronts of sorted halves, not by concatenating the halves.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Divide and merge four values

Sort [7,2,6,1] in ascending order.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Split	[7,2] and [6,1]	Divide into equal halves.
2 Split again	[7],[2],[6],[1]	Single-element arrays are sorted.
3 Merge left	[2,7]	Compare 7 and 2.
4 Merge right	[1,6]	Compare 6 and 1.
5 Merge final	[1,2,6,7]	Repeatedly take smallest head.
6 Cost	O(n log n) time; O(n) array space	Log-depth splitting, linear merging per level.

EXPLAIN THE METHOD

At each merge, show both input halves already sorted.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State how merge sort splits, how it merges sorted halves, and its typical time complexity.

QUESTION 2 / 5 MARKS

Show all split levels and the merge result for [8,3,5,2].

QUESTION 3 / 6 MARKS

Explain why standard array merge sort takes O(n log n) time and uses O(n) extra space, and contrast this with an in-place sorting approach.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Divide/split [1].
- Recursively sort and merge/combine [1].
- Time O(n log n) [1].

Q2 / 5 MARKS

- Split [8,3] / [5,2] [1].
- Split to singletons (8), (3), (5), (2) [1].
- Merge to [3,8] [1] and [2,5] [1].
- Final [2,3,5,8] [1].

Q3 / 6 MARKS

- There are O(log n) split levels [1].
- Each level performs O(n) merging [1].
- Total O(n log n) [1].
- Temporary merge buffer holds O(n) values [1].
- An in-place method may need O(1) auxiliary storage [1].
- Trade-off: stable predictable time vs buffer space [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Never compare unsorted halves as if they were already merged.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Quick sort

Quick sort chooses a pivot and partitions values so smaller ones lie to one side, larger ones to the other. It recursively sorts each partition.

PIVOT

Chosen reference value.

PARTITION

Rearrange values around pivot.

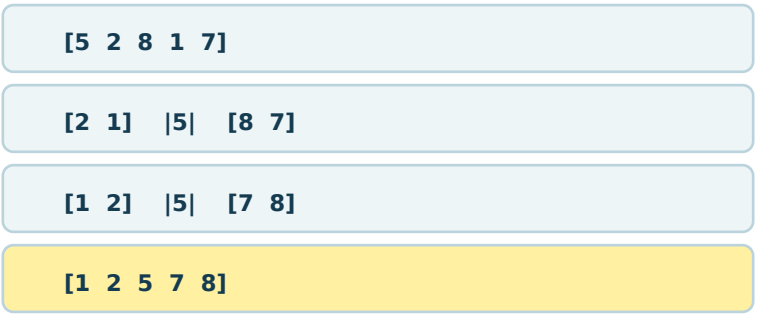
RECURSE

Sort both partitions.

PERFORMANCE

Average O(n log n); worst O(n^2).

HOW THE PARTS FIT TOGETHER



The arrows show conceptual stages, not implementation-specific swaps.

VISUAL MEMORY BRIDGE

Choose a team captain as a reference height; place shorter pupils left and taller pupils right, then organise each side.

WORDS THAT EARN MARKS

Pivot	Reference item	**Partition**	Split by pivot
Worst case	Unbalanced	**Recursion**	Repeated subproblem

EXAM TRAP

The exact array after one partition depends on the chosen pivot and partition scheme; do not assume a single trace for all implementations.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Partition around pivot 5

Illustrative partition (not a specific in-place swap scheme): input [5,2,8,1,7].

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Pivot	Choose 5	State your pivot rule.
2 Compare 2	2 < 5 -> left	Collect lower values.
3 Compare 8	8 > 5 -> right	Collect higher values.
4 Compare 1,7	1 left; 7 right	All items assigned.
5 Recur	Sort [2,1] and [8,7]	Solve independent partitions.
6 Combine	[1,2,5,7,8]	Concatenate sorted left, pivot, right.

EXPLAIN THE METHOD

The pivot must finish between values that satisfy the partition predicate.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define pivot and partition, and state quick sort average-case time complexity.

QUESTION 2 / 5 MARKS

Using first item as pivot, show the conceptual partitions of [6,9,3,8,2] and final sorted array.

QUESTION 3 / 6 MARKS

Explain why quick sort can be O(n log n) on balanced partitions but O(n^2) when partitions are consistently highly unbalanced.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Pivot is the chosen comparison value [1].
- Partition groups values by relation to pivot [1].
- Average time O(n log n) [1].

Q2 / 5 MARKS

- Pivot 6 [1].
- Lower partition [3,2] [1].
- Higher partition [9,8] [1].
- Sort left [2,3] and right [8,9] [1].
- Final [2,3,6,8,9] [1].

Q3 / 6 MARKS

- Balanced partitions give about log n levels [1].
- Each level examines approximately n items [1].
- So O(n log n) [1].
- Unbalanced partitions can give n levels [1].
- Work across levels is n+(n-1)+... [1].
- Worst case O(n^2) [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Explicitly identify pivot choice and partitioning convention.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

Dijkstra: shortest known distances

Dijkstra finds shortest paths from one start node when edge weights are non-negative. Choose the unvisited node with the smallest tentative distance and relax its outgoing edges.

DISTANCE

Best known cost so far.

RELAX

Test a route via current node.

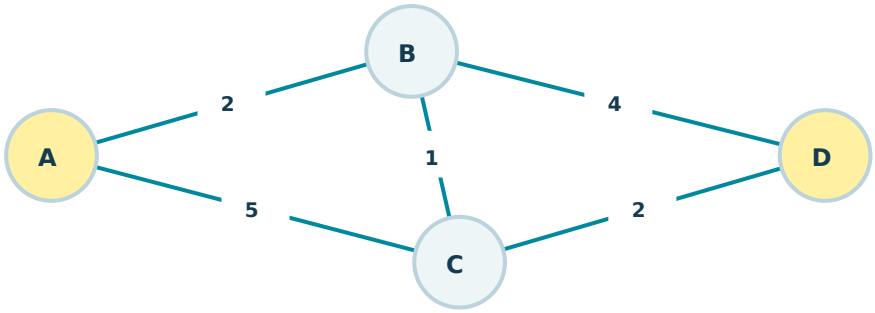
SETTLE

Finalise minimum tentative node.

RULE

No negative edge weights.

HOW THE PARTS FIT TOGETHER



Undirected, non-negative edge costs • diagram is illustrative

VISUAL MEMORY BRIDGE

Imagine spreading a ripple through a road network: the next confirmed location is whichever can be reached at the lowest current cost.

WORDS THAT EARN MARKS

Tentative	Not yet final	**Relaxation**	Improve a bound
Visited	Settled	**Non-negative**	Weight >= 0

EXAM TRAP

Do not mark an unvisited node final just because it was discovered; settle it only when it has the lowest tentative distance.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Find the shortest route from A to D

Undirected positive edges A-B=2, A-C=5, B-C=1, B-D=4, C-D=2.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Initialise	A=0, B=C=D=infinity	Start node has zero cost.
2 Settle A	B=2 via A; C=5 via A	Relax edges leaving A.
3 Settle B	C=min(5,2+1)=3; D=6	B is current cheapest.
4 Settle C	D=min(6,3+2)=5	Improve D via C.
5 Settle D	D=5, predecessor C	Cheapest remaining node is final.
6 Reconstruct	A -> B -> C -> D; cost 5	Follow predecessors backwards.

EXPLAIN THE METHOD

A tentative distance may decrease several times before its vertex is settled.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

State the weight restriction and two central actions in Dijkstra's algorithm.

QUESTION 2 / 5 MARKS

In the given graph, show how tentative distance to C and D change after settling B and then C.

QUESTION 3 / 6 MARKS

Trace shortest distance and predecessor chain from A to D; explain why a direct A-C route is not optimal.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- All edge weights are non-negative [1].
- Select lowest tentative unvisited vertex [1].
- Relax outgoing edges / settle selected vertex [1].

Q2 / 5 MARKS

- After A, C=5 [1].
- After B, C becomes 3 via B [1].
- After B, D becomes 6 via B [1].
- After C, D improves to 5 [1].
- Predecessor for D becomes C [1].

Q3 / 6 MARKS

- A=0 [1], B=2 [1], C=3 via B [1], D=5 via C [1].
- Path A-B-C-D [1].
- Direct A-C costs 5 versus A-B-C costs 3, so it is not cheapest for reaching C [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep tentative distances and finalised vertices in separate columns.

2.3.1(F) / SEE IT / CONCEPT AND VISUAL

A*: use a lower-bound heuristic

A* combines known cost g(n) with an estimated remaining cost h(n): f(n)=g(n)+h(n). With an admissible heuristic and appropriate search implementation it can find an optimal route.

g(n)

Cost from start to current node.

h(n)

Estimate of cost to goal.

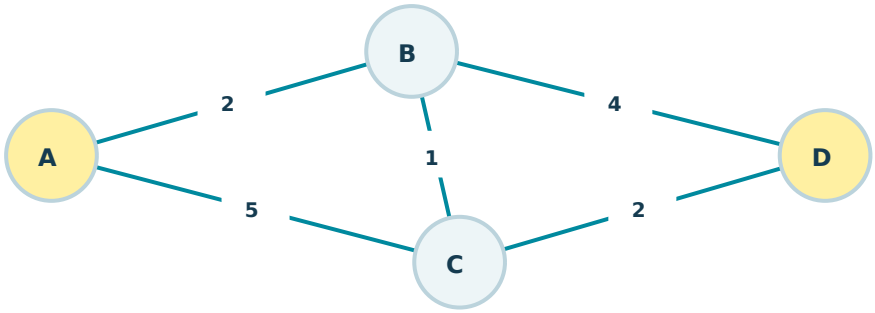
f(n)

Priority g+h.

ADMISSIBLE

Heuristic never overestimates remaining cost.

HOW THE PARTS FIT TOGETHER



Undirected, non-negative edge costs • diagram is illustrative

VISUAL MEMORY BRIDGE

Driving with a map: actual distance driven is g; a safe lower-bound estimate to the destination is h; f helps decide which route to examine next.

WORDS THAT EARN MARKS

Heuristic	Estimate	**Admissible**	No overestimate
g-score	Known cost	**f-score**	g + h

EXAM TRAP

A* chooses the smallest f, not necessarily the smallest g; careless overestimates can lose the optimal-path guarantee.

2.3.1(F) / WORKED EXAMPLE / STEP BY STEP

Compare A* candidates

At a search step, B has g=2,h=2; C has g=3,h=1; D has g=6,h=0. Lower f is preferred.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 B	f=2+2=4	B has optimistic total cost 4.
2 C	f=3+1=4	C has equal priority.
3 D	f=6+0=6	Goal is discovered but not yet best.
4 Select	Choose B or C (tie rule declared)	Priority 4 is lowest.
5 Improve	Via C, D may drop to g=5	Update goal candidate.
6 Explain	f estimates full route cost	g measured, h estimated.

EXPLAIN THE METHOD

Record all three values for each candidate; a discovered goal may not yet be the best solution.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Define g, h and f in A* and give the formula relating them.

QUESTION 2 / 5 MARKS

Calculate f for B (g=2,h=2), C (g=3,h=1) and D (g=6,h=0). Which candidate(s) have first priority?

QUESTION 3 / 6 MARKS

Explain admissibility and compare A* with Dijkstra in terms of use of an estimate, ordering and optimality assumptions.

2.3.1(F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- g is accumulated known path cost [1].
- h is estimated remaining cost [1].
- f = g+h [1].

Q2 / 5 MARKS

- B: f=4 [1].
- C: f=4 [1].
- D: f=6 [1].
- B and C tie for smallest f [1].
- Tie-break should be declared / either candidate may be selected [1].

Q3 / 6 MARKS

- Admissible h never overestimates true remaining cost [1].
- A* orders by g+h [1].
- Dijkstra effectively uses g alone (h=0) [1].
- Dijkstra requires non-negative edges [1].
- With appropriate admissible/consistent heuristic handling A* can be optimal [1].
- A* can explore fewer nodes when heuristic gives useful guidance [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Keep g and h distinct, and never label a heuristic as guaranteed accurate.

2.3.1(A-F) / SEE IT / CONCEPT AND VISUAL

Synoptic algorithm decisions

A-level solutions require more than naming an algorithm: state its input requirements, trace its behaviour, justify time/space, and check boundary cases.

SEARCH

Sorted? How many lookups?

SORT

Data size, order, memory?

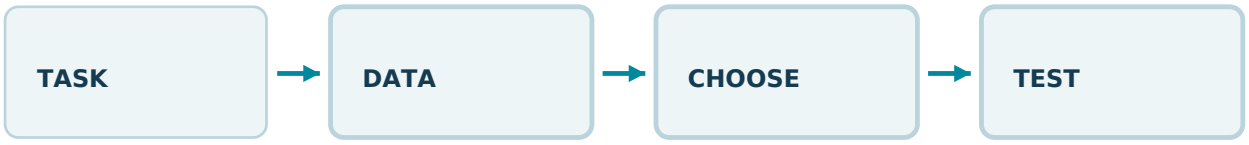
TRAVERSE

Stack/post-order or queue/BFS?

ROUTE

Non-negative weights; heuristic?

HOW THE PARTS FIT TOGETHER



TASK -> DATA PROPERTY -> ALGORITHM -> TRACE -> COMPLEXITY

VISUAL MEMORY BRIDGE

Like choosing tools for a repair: identify the job, inspect the materials, choose a suitable tool, then test whether it actually works.

WORDS THAT EARN MARKS

Justify	Reason with evidence	**Trace**	Track state
Big O	Scalability	**Invariant**	Stays true

EXAM TRAP

Naming a fast algorithm without its prerequisite is an incomplete justification.

2.3.1(A-F) / WORKED EXAMPLE / STEP BY STEP

Choose algorithms for three scenarios

Each scenario changes the data property, required output or resources.

STEP	VALUE / ACTION	WHY THIS MATTERS
1 Lookup	1 search in 18 unsorted IDs	Linear search; no setup cost.
2 Lookup	Repeated searches in sorted indexed IDs	Binary search; O(log n).
3 Sorting	Many elements, predictable time	Merge sort O(n log n), extra array.
4 Tree order	Children before their parent	Post-order DFS.
5 Tree order	Visit nearest levels first	BFS with a queue.
6 Routing	Non-negative weights, useful safe h	A* or Dijkstra with assumptions stated.

EXPLAIN THE METHOD

A strong justification links the algorithm to the particular input and identifies at least one limitation.

LINK THE TRACE TO THE EXAM

Always show the intermediate state before stating a final answer. One correct final value without a clear method may not demonstrate the required reasoning.

2.3.1(A-F) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Answer in black pen. Indicative answers are on the next page.

BLACK PEN: ANSWER FIRST. CHECK THE NEXT PAGE ONLY AFTER FINISHING.

QUESTION 1 / 3 MARKS

Select a suitable traversal for (i) processing child nodes before their parent, and (ii) exploring a tree level by level, naming the data structure for (ii).

QUESTION 2 / 5 MARKS

A small unsorted list is searched once; a much larger sorted list is searched repeatedly. Recommend and justify a search for each.

QUESTION 3 / 6 MARKS

A navigation system has non-negative road costs and an admissible distance heuristic. Compare Dijkstra with A* and set out two validation cases before deployment.

2.3.1(A-F) / SELF-MARK / INDICATIVE POINTS

Mark, correct and explain

Use the teacher-written indicative points to check your reasoning. These are not official OCR mark schemes.

Q1 / 3 MARKS

- Post-order DFS for child-before-parent [1].
- BFS for level-order [1].
- BFS uses a FIFO queue [1].

Q2 / 5 MARKS

- Small unsorted list: linear search [1].
- No sorting prerequisite or cost [1].
- Large sorted list: binary search [1].
- Repeatedly halves candidates / O(log n) [1].
- Sorted prerequisite must be maintained [1].

Q3 / 6 MARKS

- Dijkstra uses lowest tentative g [1].
- A* orders by f=g+h [1].
- A useful admissible heuristic can reduce exploration [1].
- Non-negative edges required for standard Dijkstra [1].
- Test same start and goal / zero-length route [1].
- Test unreachable node or alternative equal-cost paths [1].

GREEN PEN CORRECTION

Choose one weak answer. Rewrite it using the missing technical term or trace step. Use the scenario to drive the justification, rather than a memorised description.

OCR APPENDIX 5d / SOURCE-ALIGNED CHECKLIST

What students actually need to do

This supplement makes the official reading conventions and OCR algorithm expectations explicit. It extends the 20 main learning cycles without replacing them.

87	Appendix checklist and exam conventions
88-91	Write searches / OCR zero-based arrays
92-95	Stack, queue, linked list / pointer code
96-99	Tree post-order DFS and BFS / write code
100-103	Bubble and insertion / work and code
104-107	Merge and quick sort / recursion
108-111	Dijkstra and A* / precise traces

SPECIFICATION VS GUIDANCE

Appendix 5d supplies notation for reading code, not a list of compulsory memorised code patterns. OCR subject clarification expands the practical skills: read, trace and write searches, sorts and data structure operations; calculate Dijkstra/A* routes and trace their code.

UNDERSTAND / 1 OF 4

Write searches in OCR-style pseudocode

A Level H446 uses readable pseudocode. Arrays in the OCR examples are zero-indexed; assignment uses =, equality uses == and integer division uses DIV. A clear correct algorithm matters more than memorising exact punctuation.

CORE EXAM EXPECTATIONS

- 0-based array: indexes 0 to n-1
- Use while ... endwhile and if ... then ... endif
- Binary search: best O(1), worst O(log n), sorted data
- When no match: return -1 only after low > high

EXAM CODE / KEY REFERENCE

```
array scores[4]
scores[0] = 5
scores[1] = 9
scores[2] = 12
scores[3] = 20
if scores[2] == 12 then
    print("found")
endif
```

Do not import J277 ERL rules into H446. Correct understandable A Level pseudocode is accepted; OCR’s appendix is the familiar reading style.

MODEL / 2 OF 4

Binary search: fully written example

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
function findSorted(items, n, target)
    low = 0
    high = n - 1
    while low <= high
        mid = (low + high) DIV 2
        if items[mid] == target then
            return mid
        elseif items[mid] < target then
            low = mid + 1
        else
            high = mid - 1
        endif
    endwhile
    return -1
endfunction
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. Given [2,5,8,11,14,17,20], target 5

2. low=0, high=6: mid=3, value=11 -> high=2

3. low=0, high=2: mid=1, value=5 -> return 1

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

State how OCR-style pseudocode assigns a value, compares equality and indexes an array.

QUESTION 2 / 5 MARKS

Trace the mid values and updated bounds when searching for 17 in [2,5,8,11,14,17,20].

QUESTION 3 / 6 MARKS

Write a linear search function for an array items of length n. Return the first matching index, or -1.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: assignment = [1]; comparison == [1]; indices start at 0 [1].

Q2 / 5 MARKS

5 marks: low=0/high=6 [1]; mid=3 [1], value=11 [1]; low=4 [1]; mid=5 and value=17, return 5 [1].

Q3 / 6 MARKS

6 marks: loop from index 0 [1] through n-1 [1]; compare items[i] to target [1]; return matching index immediately [1]; return -1 after loop [1]; correct function/termination structure [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
function findLinear(items, n, target)
    for i = 0 to n - 1
        if items[i] == target then
            return i
        endif
    next i
    return -1
endfunction
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Implement stack, queue and linked-list operations

OCR expects learners to read, trace and write code that adds and removes values. Show pointer changes and error cases, not just the LIFO/FIFO definitions.

CORE EXAM EXPECTATIONS

- For a stack, top points to the next free array slot
- Queue front = next to remove; rear = next free slot
- Check full/empty before changing pointers
- Linked-list insertion: new.next first, then previous.next

EXAM CODE / KEY REFERENCE

```
// Stack of capacity 4, initially top = 0
array stack[4]
// Push 9: stack[top] = 9; top = top + 1
// Pop: top = top - 1; return stack[top]
// Empty when top == 0; full when top == 4
```

A pointer update is not a value deletion: explain which node becomes reachable and which reference changes. State queue pointer conventions.

MODEL / 2 OF 4

Safe operations with explicit pointers

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
function push(value)
    if top == capacity then return false endif
    stack[top] = value
    top = top + 1
    return true
endfunction
function pop()
    if top == 0 then return "EMPTY" endif
    top = top - 1
    return stack[top]
endfunction
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. top=0 -> push(4): stack[0]=4, top=1
2. push(7): stack[1]=7, top=2
3. pop(): top=1, returns 7; 4 remains
4. List insertion after 4: new.next=oldNext; node4.next=new

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

Explain what top represents and identify one stack overflow and underflow condition.

QUESTION 2 / 5 MARKS

Trace push(4), push(7), pop(), push(2) with top initially 0. State the return and final logical stack.

QUESTION 3 / 6 MARKS

Describe an array circular queue with front, rear and count; give safe enqueue/dequeue pointer updates and error checks.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: next free slot [1]; overflow top==capacity [1]; underflow top==0 [1].

Q2 / 5 MARKS

5 marks: push4 gives top1 [1], push7 gives top2 [1], pop returns7 [1], top falls to1 [1], push2 leaves [4,2] [1].

Q3 / 6 MARKS

6 marks: reject enqueue when count==capacity [1]; write q[rear] [1]; rear=(rear+1) MOD capacity [1]; reject dequeue when count==0 [1]; read q[front], then front=(front+1) MOD capacity [1]; increment/decrement count appropriately [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
// Singly linked insertion after current
newNode.next = current.next
current.next = newNode
// Deleting after current
if current.next != NULL then
    current.next = current.next.next
endif
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Write post-order DFS and queue-based BFS

The required OCR tree traversal is specifically depth-first POST-ORDER, not generic preorder. BFS uses a queue to visit each level before its children.

CORE EXAM EXPECTATIONS

- Post-order: left, right, root
- BFS: use FIFO queue
- Each node in a tree is processed once: O(n) time
- For recursive DFS, recursion depth depends on tree height

EXAM CODE / KEY REFERENCE

```
// Tree: root 8; children 3 and 12
// 3 has children 1 and 6
// 12 has children 10 and 14
// Post-order: 1,6,3,10,14,12,8
// Breadth-first: 8,3,12,1,6,10,14
```

Do not say “DFS” alone when the question demands post-order; explicit left/right/root order earns the explanation.

MODEL / 2 OF 4

Two complete traversal outlines

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
procedure post(node)
    if node != NULL then
        post(node.left)
        post(node.right)
        print(node.value)
    endif
endprocedure

procedure breadth(root)
    q = new Queue()
    if root != NULL then q.enqueue(root) endif
    while NOT q.isEmpty()
        node = q.dequeue()
        print(node.value)
        for child in node.children
            q.enqueue(child)
        next child
    endwhile
endprocedure
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. BFS queue after visiting 8: [3,12]
2. After visiting 3: [12,1,6]
3. After visiting 12: [1,6,10,14]
4. Post-order does not print a parent until both children finish.

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

Explain why a FIFO queue produces breadth-first rather than depth-first traversal.

QUESTION 2 / 5 MARKS

Produce both orders for tree A with children B,C; B has children D,E, and C has child F.

QUESTION 3 / 6 MARKS

Write recursive post-order pseudocode and explain its termination and time complexity.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: enqueue discovered children [1]; remove oldest pending node [1]; therefore visit current level before newly enqueued descendants [1].

Q2 / 5 MARKS

5 marks: post-order D,E,B,F,C,A [3]; BFS A,B,C,D,E,F [2].

Q3 / 6 MARKS

6 marks: null/base guard [1]; left recursive call [1]; right recursive call [1]; root output after children [1]; terminates at null leaves [1]; O(n) because each node processed once [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
procedure postOrder(node)
    if node != NULL then
        postOrder(node.left)
        postOrder(node.right)
        print(node.value)
    endif
endprocedure
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Read and write bubble and insertion sort

Exam tasks may show incomplete code or a list after each pass. State whether the bubble implementation has an early-exit check before quoting its best case.

CORE EXAM EXPECTATIONS

- Bubble: compare adjacent values and swap if left > right
- One completed bubble pass fixes the largest remaining value
- Insertion: save key, shift larger items right
- Both worst-case $O(n^2)$; best $O(n)$ for insertion and optimised bubble

EXAM CODE / KEY REFERENCE

```
// One bubble pass over [5,1,4,2]
[5,1,4,2] -> [1,5,4,2]
           -> [1,4,5,2]
           -> [1,4,2,5]
// Largest item 5 now fixed at the end
```

A missing temporary variable can overwrite a value. Distinguish passes, comparisons and swaps.

MODEL / 2 OF 4

Insertion sort with 0-based array and key

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
procedure insertionSort(a, n)
    for i = 1 to n - 1
        key = a[i]
        j = i - 1
        while j >= 0 AND a[j] > key
            a[j+1] = a[j]
            j = j - 1
        endwhile
        a[j+1] = key
    next i
endprocedure
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. [5,2,4,1]: insert 2 -> [2,5,4,1]
2. Insert 4 -> [2,4,5,1]
3. Insert 1 -> [1,2,4,5]
4. Only items greater than the key shift.

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

State the adjacent swap rule in bubble sort and how an early-exit version can terminate.

QUESTION 2 / 5 MARKS

Show array states after each insertion of [4,2,3,1] in insertion sort.

QUESTION 3 / 6 MARKS

Write the critical inner bubble-sort loop with a swap and explain its worst-case time complexity.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: compare adjacent items [1]; swap if left>right [1]; end when a complete pass makes no swaps [1].

Q2 / 5 MARKS

5 marks: after key2 [2,4,3,1] [1]; key3 [2,3,4,1] [2]; key1 [1,2,3,4] [2].

Q3 / 6 MARKS

6 marks: loop across adjacent pairs [1]; compare > [1]; swap correctly with temp [2]; repeat for passes [1]; $O(n^2)$ because passes contain a linear number of comparisons [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
// A bubble pass on indices 0 .. end-1
for j = 0 to end - 1
    if a[j] > a[j+1] then
        temp = a[j]
        a[j] = a[j+1]
        a[j+1] = temp
    endif
next j
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

UNDERSTAND / 1 OF 4

Merge sort and quick sort

Show what each recursive step does, including the stopping condition. Helpers below have precise contracts; they are not built-in OCR commands.

WHAT THE ALGORITHMS DO

MERGE SORT

Split into halves, recursively sort each half, then merge the two already-sorted halves. Standard array version: O(n log n) time; O(n) auxiliary buffer.

QUICK SORT

Choose a pivot, put lower/equal/higher values into three partitions and recurse on lower and higher only. Average O(n log n); worst O(n^2) if pivots remain highly unbalanced.

HELPER CONTRACTS / ALL INDEXES ZERO-BASED

copyRange(a,start,stop)

Returns a new array from indexes start to stop-1; stop is EXCLUSIVE. Example: copyRange([7,2,6,1],0,2) -> [7,2].

mergeSorted(left,right)

Takes two sorted inputs and repeatedly takes the lower front element; copies the remaining tail when one input is exhausted.

partition3(a,pivot)

Returns (less,equal,more). Every input item appears exactly once; ALL values equal to pivot go to equal, avoiding recursive loops on duplicate pivots.

QUICK VISUAL CHECK

Merge [2,7] + [1,6] -> [1,2,6,7]. Pivot 6: [6,9,3,8,2,6] -> [3,2] | [6,6] | [9,8].

MODEL / 2 OF 4

Recursive structure (helpers defined)

Read each helper contract on the previous page. The pseudocode below is an algorithm outline using those named operations, not a verbatim OCR Appendix 5d language requirement.

SORTING ROUTINES / END-EXCLUSIVE SLICES

```
function mergeSort(a)
    if a.length <= 1 then return a endif
    mid = a.length DIV 2
    left = mergeSort(copyRange(a,0,mid))
    right = mergeSort(copyRange(a,mid,a.length))
    return mergeSorted(left,right)
endfunction

function quickSort(a)
    if a.length <= 1 then return a endif
    pivot = a[0]
    less,equal,more = partition3(a,pivot)
    return quickSort(less) + equal + quickSort(more)
endfunction
```

WORKED TRACE / INCLUDE EACH INTERMEDIATE STATE

MERGE [8,3,5,2]

[8,3] + [5,2] -> [8],[3],[5],[2] -> [3,8] + [2,5] -> [2,3,5,8].

QUICK [6,9,3,8,2]

Pivot 6 -> less [3,2], equal [6], more [9,8] -> [2,3] + [6] + [8,9].

QUICK WITH DUPLICATES

[6,9,3,8,2,6] puts both 6s into equal, so recursive sublists shrink; result [2,3,6,6,8,9].

TRY / 3 OF 4

Trace, write, explain

Attempt all three original exam-style questions in black pen. The marks are indicative; do not turn over until you have completed your answer.

BLACK PEN / ANSWER BEFORE CHECKING

QUESTION 1 / 3 MARKS

State the base case, what must be true of the inputs to a merge step, and the extra array-space cost of standard merge sort.

QUESTION 2 / 5 MARKS

Trace merge sort on [4,1,3,2], showing every split and both intermediate merges before the final result.

QUESTION 3 / 6 MARKS

For first-item-pivot quick sort, describe the partitions of [4,7,2,6,1] and explain how unbalanced pivots affect time complexity.

CHECK / 4 OF 4

Self-mark, correct, explain

Award each point only when it is supported by the student response. These are teacher-written indicative points, not an official OCR mark scheme.

GREEN PEN / INDICATIVE MARKING

Q1 / 3 MARKS

- Base case: zero/one item needs no further sorting [1].
- Merge consumes inputs that are each already in sorted order [1].
- Standard array merge sort uses O(n) auxiliary storage [1].

Q2 / 5 MARKS

- Split [4,1] and [3,2] [1]; then [4],[1],[3],[2] [1].
- Merge left [1,4] [1]; merge right [2,3] [1].
- Final merge produces [1,2,3,4] [1].

Q3 / 6 MARKS

- Choose pivot 4 [1]; lower partition [2,1] [1].
- Higher partition [7,6] [1]; recursively sort both [1].
- Persistent unbalanced pivots give O(n) levels [1].
- Summed comparisons yield O(n^2) worst case [1].

COMPLETE MERGE LOOP / HOW TO HANDLE LEFTOVERS

```
i = 0; j = 0; result = []
while i < left.length AND j < right.length
    if left[i] <= right[j] then
        result.append(left[i]); i = i + 1
    else
        result.append(right[j]); j = j + 1
    endif
endwhile
while i < left.length
    result.append(left[i]); i = i + 1
endwhile
while j < right.length
    result.append(right[j]); j = j + 1
endwhile
// result contains the full sorted merge
```

NOTE: result is a resizable list; [] and append are illustrative pseudocode.

UNDERSTAND / 1 OF 4

Work a Dijkstra and A* route trace

OCR clarification emphasises calculating routes and tracing code. Dijkstra settles lowest tentative distance; A* prioritises f=g+h. Use valid non-negative edge costs and a safe heuristic.

CORE EXAM EXPECTATIONS

- Graph: A-B=2, A-C=5, B-C=1, B-D=4, C-D=2
- Correct shortest route A-B-C-D, total cost 5
- Dijkstra: settle smallest tentative g
- A*: f=g+h; admissible h does not overestimate

EXAM CODE / KEY REFERENCE

```
h(A)=5, h(B)=3, h(C)=2, h(D)=0
// All these h values are true lower bounds
// At A: B g=2 f=5; C g=5 f=7
// After B: C g=3 f=5; D g=6 f=6
// After C: D g=5 f=5
```

The model uses an admissible and consistent heuristic, with goal termination on removal from the open set. “First found goal” is not enough.

MODEL / 2 OF 4

Dijkstra settled-set and A* priority trace

Read the entire routine first, then track the changing state. These examples are original teaching models, not reproduced examination questions.

```
// Dijkstra: distances from A
initial: A=0; B=C=D=infinity
settle A: B=2 via A; C=5 via A
settle B: C=3 via B; D=6 via B
settle C: D=5 via C
settle D: route A-B-C-D, cost=5

// A* candidate priority f = g + h
after A: B=2+3=5; C=5+2=7
after B: C=3+2=5; D=6+0=6
after C: D=5+0=5; pop goal D
```

WORKED TRACE / WRITE EACH INTERMEDIATE STATE

1. Key update: compare old C=5 against route A-B-C=2+1=3.
2. D initially 6 via B, improves to 5 via C.
3. With h as above, A* chooses B, then C, then D.
4. A* must not stop merely when a costly goal is first discovered.

TRY / 3 OF 4

Trace, write, test

BLACK PEN: attempt independently. Keep all answers on this page until you are ready to turn over and self-mark.

QUESTION 1 / 3 MARKS

Define g, h and f in A*, giving the formula for f.

QUESTION 2 / 5 MARKS

Trace Dijkstra from A to D: record the order settled and each improvement of C and D.

QUESTION 3 / 6 MARKS

Calculate A* f priorities after A, B and C with the supplied h; explain why the final result is optimal under these assumptions.

CHECK / 4 OF 4

Self-mark, correct, explain

GREEN PEN: award only the marks earned. The following are original indicative answers, not official OCR mark schemes.

Q1 / 3 MARKS

3 marks: g=known start-to-node cost [1], h=estimated remaining cost [1], f=g+h [1].

Q2 / 5 MARKS

5 marks: settle A,B,C,D [1]; B=2 [1]; C 5->3 [1]; D 6->5 [1]; path A-B-C-D cost5 [1].

Q3 / 6 MARKS

6 marks: B f=5 and C f=7 after A [2]; after B C f=5 and D f=6 [2]; after C D f=5 [1]; admissible (indeed consistent) h, non-negative edges and correct open-list handling preserve optimality [1].

EXAMPLE CODE / PSEUDOCODE OUTLINE

```
// A* open-list comparison (outline)
while openSet is not empty
    current = itemWithSmallestF(openSet)
    if current == goal then return path endif
    for each neighbour of current
        candidateG = g[current] + edgeCost
        if candidateG < g[neighbour] then
            predecessor[neighbour] = current
            g[neighbour] = candidateG
            f[neighbour] = candidateG + h[neighbour]
            addOrUpdate(openSet, neighbour)
        endif
    next neighbour
endwhile
```

Make one correction in green: fix a boundary, data-structure update, base case, or justification before moving on.

Paper 2 / Original exam practice

Preserved worked examples, questions and answers

This appendix keeps selected questions, worked examples and marking points from the earlier published Paper 2 revision guide. They remain in their original order. Questions are original RHSC practice, not official OCR past-paper questions. Complete the questions before turning to their associated marking pages.

PRESERVED TOPICS

Programming in C#: CountPasses and marking points — PDF p. 250
Original source pages 27-34

Computational methods: ValidTimes and Overlaps — PDF p. 258
Original source pages 95-102

Algorithm tracing: CountDistinctSorted and worked answers — PDF p. 266
Original source pages 112-121

Original mixed memory banks — PDF p. 276
Original source pages 176-179

WORKED EXAMPLE 2: a different trace

A temperature logger analyses readings. Predict both outputs before inspecting the working.

```
int[] temps = {18, 23, 21, 26, 19};
int hotDays = 0;
int total = 0;
foreach (int t in temps) {
    total += t;
    if (t >= 22) hotDays++;
}
Console.WriteLine($"{hotDays}, {total}");
```

The readings 23 and 26 meet the threshold; hotDays is 2. The total is 18 + 23 + 21 + 26 + 19 = 107.

How to answer an unfamiliar coding question

- Name the input, expected output and any constraints.
- Create a state table with one row per iteration.
- Highlight the exact condition that controls each branch.
- Check boundaries: empty input, first/last element and inclusive inequalities.
- Give the precise output before adding your explanation.

Common mistake

A loop variable can change independently of an accumulator. Record both rather than treating each printed value as the value of the loop counter.

2.2.1 · EXAM PRACTICE

SIS: trace and construct

Complete the questions without looking at the marking points.

Q1 · 5 marks Trace the value of score after each iteration and state the final output.

```
int score = 1;
for (int i = 1; i <= 4; i++) {
    if (i % 2 == 0) score += i;
    else score *= 2;
}
Console.WriteLine(score);
```

Q2 · 6 marks An age-checking method should accept ages from 18 to 65 inclusive. Identify the error, write a corrected condition and give four boundary tests (with expected results).

```
static bool ValidAge(int age) {
    return age < 18 || age > 65;
}
```

2.2.1 · EXAM PRACTICE

Writing algorithms in C#

Include a method signature, required initialisation, a valid loop and an explicit result.

Q3 · 8 marks Write static int SumEven(int[] values) to return the sum of even numbers in a non-null array. For {2,3,4}, the result must be 6. Do not modify the array.

Q4 · 6 marks Trace count, then state the time and auxiliary-space complexities if the upper limit 3 is generalised to n.

```
int count = 0;
for (int row = 1; row <= 3; row++)
    for (int col = 1; col <= row; col++)
        count++;
```

2.2.1 · EXAM PRACTICE

Selection with validation

Invalid values must be excluded before their result is considered.

Q5 · 8 marks Write static int CountPasses(int[] scores). Only scores from 0 to 100 inclusive are valid; count those valid scores that are at least 50. Input {49,50,70,-1,101} should return 2. Assume scores is non-null.

2.2.1 · CHECK AND IMPROVE

SIS: marking points

Compare with the mark scheme only after you have attempted all five questions.

Q1 · 5 marks

- score after i=1 is 2 [1]; after i=2 is 4 [1].
- After i=3 it is 8 [1]; after i=4 it is 12 [1].
- Printed output 12 [1].

Q2 · 6 marks

- The original condition returns true for values outside the allowed range [2].
- Correct condition: age >= 18 && age <= 65 [2].
- Boundary tests: 17 false and 18 true [1]; 65 true and 66 false [1].

Q3 · 8 marks

- Signature [1], accumulator initialisation [1], loop through array [2].
- Even-value test using remainder [2], add selected value [1], return sum [1].

```
static int SumEven(int[] values) {
    int total = 0;
    foreach (int v in values)
        if (v % 2 == 0) total += v;
    return total;
}
```

Q4 · 6 marks

- Inner-loop repetitions are 1, 2 and 3 [2]; therefore count = 6 [1].
- For variable n, total iterations 1+2+...+n = n(n+1)/2, hence $O(n^2)$ [2].
- Only loop counters and count are stored, so auxiliary space O(1) [1].

2.2.1 · CHECK AND IMPROVE

SIS: marking points continued

Equivalent valid C# solutions earn credit; method names and variable names may differ.

Q5 · 8 marks

- Method signature returning int [1] and accumulator initialisation [1].
- Correct iteration over scores [2].
- Reject values below 0 or above 100 [2].
- Increment for valid scores at least 50 [1], return the count [1].

```
static int CountPasses(int[] scores) {
    int count = 0;
    foreach (int n in scores) {
        if (n < 0 || n > 100) continue;
        if (n >= 50) count++;
    }
    return count;
}
```

Memory bank · Code First

SIS: sequence / iteration / selection. TRACE: track the state, respect the condition, advance one step, check the output, explain boundaries. TRACE is a memory cue, not OCR terminology.

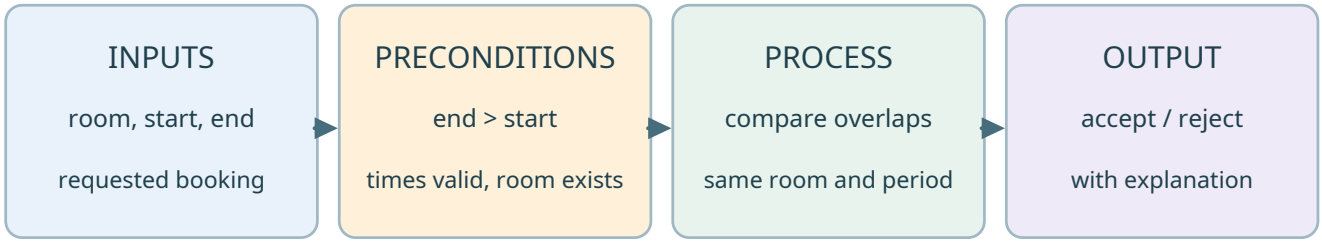
Define the computational problem

Recognise inputs, outputs, preconditions and boundaries before choosing an algorithm.

REMEMBER / THE RECIPE CARD

A precise computing problem needs **ingredients** (inputs), **rules** (preconditions), **steps** (algorithm) and a **result** (output). A recipe is only an analogy: a program must define its conditions and behaviour unambiguously.

FROM VAGUE TASK TO TESTABLE CONTRACT



EXAMPLE BOUNDARY: Half-open times [start, end):
09:00-10:00 and 10:00-11:00 do not overlap.

Make the acceptance rule unambiguous before writing code.

WORKED EXAMPLE 1 / SCHOOL ROOM BOOKING

Input: a room and proposed start/end times. Existing booking in that room is 09:00-10:00. New request is 10:00-11:00. Use **half-open intervals** [start,end), so touching endpoints do not overlap. The request is valid if start < end; it can be accepted if there is no conflicting existing booking.

General overlap test: **newStart < oldEnd AND oldStart < newEnd** (only when both bookings use the same room). For 10:00-11:00 and 09:00-10:00, the first comparison is false, so no overlap.

WORKED EXAMPLE 2 / DIFFERENT SYSTEM

An online library must approve a loan only when the **BookID** exists, the book is available, and the borrower is eligible. Identify the inputs, at least two preconditions and the output. Explain what to do if the BookID does not exist.

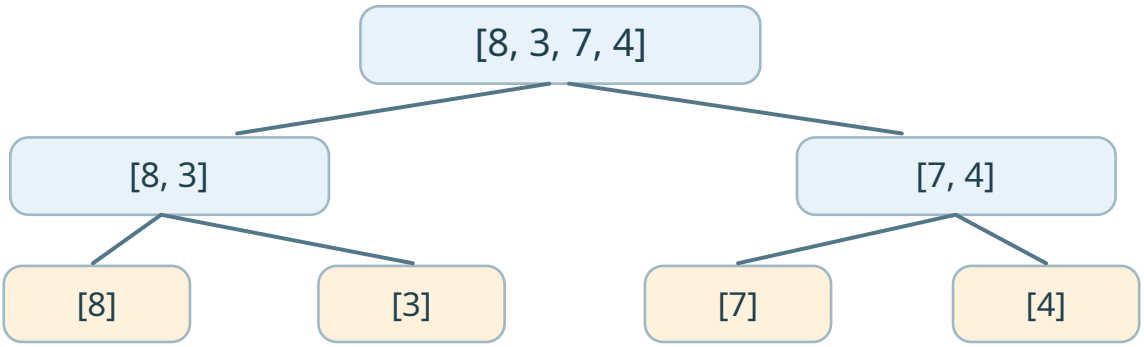
Divide and conquer

Solving smaller versions of the same problem and combining the answers.

REMEMBER / SPLIT, SOLVE, REUNITE

Divide a list into smaller instances, solve each and combine. **Merge sort** really is divide and conquer; dividing work among unrelated functions is decomposition, but it is not automatically divide and conquer.

DIVIDE -> SOLVE -> COMBINE



Combine: [3,8] and [4,7]

[3, 4, 7, 8]

Each half is sorted before they are merged.

WORKED EXAMPLE 1 / TRACE A DIFFERENT MERGE SORT

Sort **[7, 2, 6, 1]**. Split into [7,2] and [6,1]. Split again into singletons. Merge to [2,7] and [1,6]. Combine the sorted halves by comparing their front values: **[1,2,6,7]**.

WORKED EXAMPLE 2 / ANOTHER INPUT

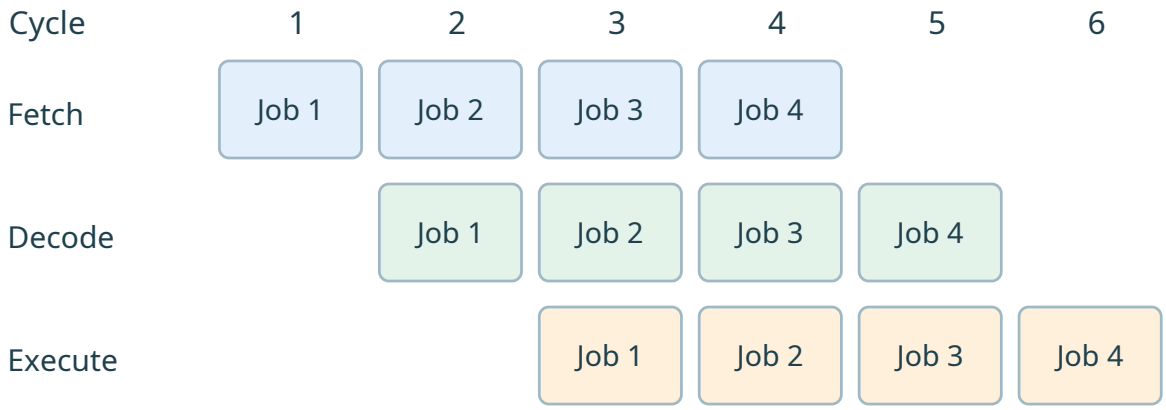
Trace **[9,5,8,3]** using the same approach. Include each two-element sorted run before the final merge.

Check: split [9,5] and [8,3], then [9], [5], [8], [3]. Merge to [5,9] and [3,8]. Finally merge to **[3,5,8,9]**.

Performance modelling and pipelining

Predict the benefit using stated assumptions, not an unsupported speed claim.

PIPELINING: OVERLAP THREE STAGES ACROSS SIX CYCLES



Ideal: four jobs, three equal stages = 3 + 4 - 1 = 6 cycles.

Without overlap: 4 x 3 = 12 stage-cycles.

WORKED EXAMPLE 1: Four jobs require Fetch, Decode and Execute stages, each taking one cycle. Without overlapping jobs: 4 x 3 = **12 cycles**. In an ideal balanced three-stage pipeline, total cycles = stages + jobs - 1 = 3 + 4 - 1 = **6 cycles**. This illustrates higher throughput, not necessarily shorter latency for one job.

WORKED EXAMPLE 2 / NEW WORKLOAD

Five jobs each pass through four equal one-cycle stages. Predict the ideal pipelined and sequential times, and calculate the saving. What happens if a stage stalls?

Check: Sequential = 5 x 4 = 20 cycles. Ideal pipeline = 4 + 5 - 1 = 8 cycles; saving = 12 cycles. A stalled stage or shared resource can delay following jobs, so actual time may be longer.

NOW BE PRECISE

A performance model simplifies reality using assumptions. Throughput is completed jobs per unit time; latency is the time a single job takes. Real constraints include bottlenecks, data dependencies, communication and pipeline stalls.

Design reusable solutions

Modularity supports testing and reuse, but does not automatically solve invalid inputs.

```
static bool Overlaps(int aStart, int aEnd,
                     int bStart, int bEnd) {
  return aStart < bEnd && bStart < aEnd;
}
static bool ValidTimes(int start, int end) {
  return start < end;
}
```

WORKED EXAMPLE 1: Overlaps(9,11,10,11) returns false, because 9 < 11 is true but 10 < 10 is false. ValidTimes(10,10) returns false. These functions can be tested independently and reused in a larger booking system.

WORKED EXAMPLE 2 / DIFFERENT PARAMETERS

Use the Overlaps function to decide whether 09:30-10:30 clashes with 10:00-11:00. The conditions are 9.5 < 11 and 10 < 10.5: both true, so the times overlap. In real code, represent times consistently as integer minutes or a suitable time type.

Common mistake: Never choose outputs or data types without defining the units. An integer minute count, for example 570 for 09:30, avoids ambiguity about decimal times.

Exam practice / problem recognition

Q1 [6 marks] A booking system must prevent two overlapping bookings in the same room. State two inputs, two preconditions and two outputs or outcomes that together define the computational problem.

Q2 [6 marks] Explain the difference between decomposition and divide and conquer using merge sort, and trace the two-element sorted runs when sorting [6,2,9,1].

Exam practice / performance modelling

Q3 [6 marks] Six independent jobs each pass through three equal one-cycle stages. Calculate sequential and ideal pipelined completion times and the ideal saving. Explain one assumption behind the prediction.

Q4 [6 marks] A school system estimates that 80% of a process can run in parallel while 20% must remain sequential. Explain why adding many processors cannot eliminate the sequential time, and discuss two further real-world constraints on speedup.

Exam practice / reusable components

Q5 [6 marks] Write C# method headers and return expressions for ValidTimes(start,end) and Overlaps(aStart,aEnd,bStart,bEnd). State the outcome for a proposed 10:00-11:00 booking compared with existing 09:00-10:00, assuming integer minutes.

Check your answers: computational methods

Check the calculations and the method conditions before self-marking.

Q1 · 6 marks

1. Give a room identifier as an input.
2. Give requested start/end times (or the current booking list) as inputs.
3. Require a valid/known room.
4. Require start before end and consistent time units.
5. Output a booking accepted/rejected decision.
6. Output an explanatory reason or a successfully recorded booking.

Q2 · 6 marks

1. Decomposition divides a task into manageable parts.
2. Divide and conquer breaks a problem into smaller instances of the same type.
3. It solves those subproblems and combines their results.
4. Split the array into **[6, 2]** and **[9, 1]**.
5. The sorted first two-element run is **[2, 6]**.
6. The sorted second run is **[1, 9]**.

Q3 · 6 marks

1. Sequential work is **6 × 3** stage-cycles.
2. Sequential total = **18 cycles**.
3. Ideal pipelining uses **3 + 6 − 1**.
4. Ideal total = **8 cycles**.
5. Ideal saving = **10 cycles**.
6. State an assumption: equal stage times, no stalls or independent jobs.

Q4 · 6 marks

1. Identify the non-parallel fraction as 20% of the work.
2. Explain that this portion still takes time regardless of extra processors.
3. Consequently speedup has a finite limit (ideal maximum **5×**, ignoring overhead).
4. Name one real-world constraint, such as communication or synchronisation overhead.
5. Explain how this constraint limits the speedup.
6. Identify and explain a second different constraint, such as memory bandwidth or load imbalance.

Q5 · 6 marks

1. ValidTimes takes integer start/end arguments and returns bool.
2. It returns the condition **start < end**.
3. Overlaps takes four integer times and returns bool.
4. It returns **aStart < bEnd && bStart < aEnd**.
5. Two bookings touching at 10:00 do not overlap under half-open intervals.
6. Therefore the proposed 10:00–11:00 booking is accepted against 09:00–10:00.

Q5 · 6 marks Compare worst-case time and auxiliary space for an in-place bubble sort and a conventional merge sort, then justify when either might be a reasonable choice.

Model answers and marking points

Attempt the questions before checking your work. Equivalent technically accurate answers may also receive credit.

Q1 · 4 marks

Constant lookup O(1) [1]; full pass O(n) [1]; two complete nested loops $O(n^2)$ [1]; repeated halving O(log n) [1].

Q2 · 6 marks

Both repeatedly halve the search range [1] and take O(log n) worst-case time [2]. Standard iterative search uses O(1) auxiliary memory [1]; a straightforward recursive implementation has O(log n) call-stack usage [1]; both require sorted input for the standard algorithm [1].

Q3 · 6 marks

Start 1 [1]; next 1 no increment [1]; new 3 increases to 2 [1]; new 5 increases to 3 [1]; repeat 5 keeps 3 [1]; new 9 increases to 4 [1]. Result 4.

Q4 · 8 marks

Counter detects three runs and reports 3 [2], but there are only two distinct values [1]. Repair 1: sort then scan [1], taking typical O(n log n) time [1], with space dependent on sorting method [1]. Repair 2: insert into a hash set [1], expected O(n) time but O(n) extra space [1].

Q5 · 6 marks

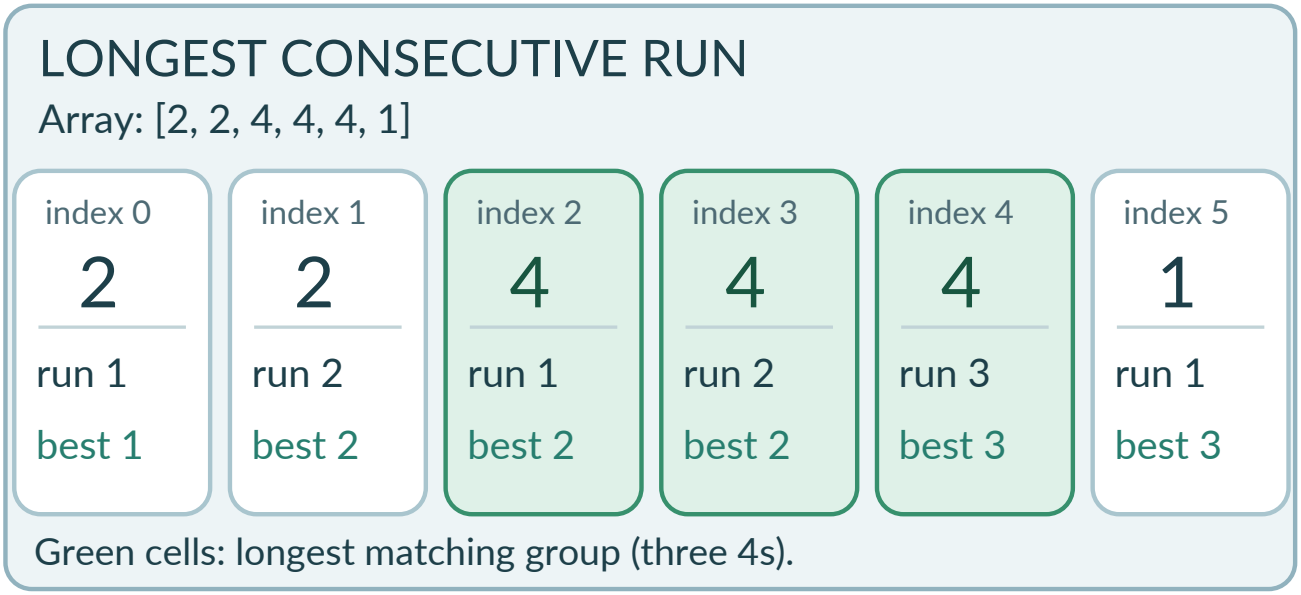
Bubble sort $O(n^2)$ worst case [1], typically O(1) extra space [1]; merge sort O(n log n) worst case [1], conventionally O(n) additional memory [1]; merge generally more suitable at large n [1], bubble can be simple for very small teaching examples or constrained extra memory [1].

Unfamiliar Algorithms: Trace, Test, Explain

In an unfamiliar algorithm, the challenge is to make the method visible: input, precondition, state, result, invariant, complexity and tests.

WORKED EXAMPLE 1 · Longest run of equal values

```
static int LongestRun(int[] a) {
    if (a.Length == 0) return 0;
    int current = 1, best = 1;
    for (int i = 1; i < a.Length; i++) {
        if (a[i] == a[i - 1]) current++;
        else current = 1;
        if (current > best) best = current;
    }
    return best;
}
```



For {2,2,4,4,4,1}, the longest adjacent run is three 4s. Invariant: current tracks the length of the run ending at index i; best tracks the largest run seen in the processed prefix.

Worst-case time O(n); auxiliary space O(1). Test empty, one element, all equal and alternating values.

WORKED EXAMPLE 2: a genuinely different algorithm

Find the first index where values increase compared with the previous element. Do not assume the array is sorted.

```
static int FirstRise(int[] a) {
    for (int i = 1; i < a.Length; i++) {
        if (a[i] > a[i - 1]) return i;
    }
    return -1;
}
// Input: {5, 4, 4, 7, 2}
```

At i=1, 4 is not greater than 5; at i=2, 4 is not greater than 4; at i=3, 7 > 4, so return 3. If there is no increase, return -1.

A reliable unfamiliar-algorithm checklist

- Input and output: what exactly is being returned?
- Preconditions: sorted input, non-null arrays, positive weights?
- State: list variables that change each step.
- Invariant: what remains true after each iteration?
- Correctness: show why the output answers the question.
- Efficiency and tests: time, additional memory and edge cases.

Remember this

TRACE: Test the assumption, Record each change, Apply the condition, Check the final result, Explain why it works. The phrase is a memory aid; use algorithmic terminology in written answers.

2.3.1 · EXAM PRACTICE

Trace and reason

The questions are independent of the WORKED EXAMPLE 1 and WORKED EXAMPLE 2 examples.

Q1 · 6 marks Trace current and best for the input {4,4,2,2,2,5}, and state the returned value. Use the LongestRun algorithm from this chapter.

Q2 · 5 marks For the FirstRise algorithm, trace {9,9,7,10,8} and state the returned index. Explain why the loop begins at index 1 and when -1 is returned.

2.3.1 · EXAM PRACTICE

Design from a specification

Ensure the precondition and edge cases are handled.

Q3 · 8 marks Write static int CountDistinctSorted(int[] a) for a non-null sorted integer array. Return 0 for an empty array, otherwise count distinct adjacent values. For {1,1,3,3,8}, return 3.

Q4 · 6 marks For LongestRun, justify the worst-case time and auxiliary-space complexity, state a loop invariant, and name two edge-case tests with their expected results.

2.3.1 · EXAM PRACTICE

Debug an unfamiliar search

The following method attempts binary search on an ascending sorted array. Find the defect and explain a failing case.

Q5 · 8 marks Identify the faulty update, repair it, show how searching for 2 in {3} can fail to progress in the original code, and state a precondition and worst-case time complexity.

```
static int Find(int[] a, int target) {
    int low = 0, high = a.Length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid;  // inspect this line
    }
    return -1;
}
```

2.3.1 · CHECK AND IMPROVE

Unfamiliar algorithms: marking points

Use complete algorithm states and avoid claiming a runtime from the test input alone.

Q1 · 6 marks

- Index 0: current=1,best=1 [1]; index 1: current=2,best=2 [1].
- Index 2: current=1,best=2 [1]; index 3: current=2,best=2 [1].
- Index 4: current=3,best=3 [1]; index 5: current=1,best=3, returning 3 [1].

Q2 · 5 marks

- Compare i=1 (9 vs 9) and i=2 (7 vs 9): neither rises [1].
- At i=3, 10 > 7 [1], so returns 3 [1].
- i starts at 1 to allow a[i-1] safely [1].
- -1 is returned when there is no rising adjacent pair [1].

Q3 · 8 marks

- Correct signature/return type [1]; empty-array check returning 0 [1].
- Initial distinct count 1 for nonempty input [1].
- Loop i from 1 to length-1 [2]; compare a[i] with a[i-1] [2].
- Increment when different and return the final count [1].

```
static int CountDistinctSorted(int[] a) {
    if (a.Length == 0) return 0;
    int count = 1;
    for (int i = 1; i < a.Length; i++)
        if (a[i] != a[i - 1]) count++;
    return count;
}
```

2.3.1 · CHECK AND IMPROVE

Unfamiliar algorithms: continued

The strongest answers link each line of code to a requirement in the problem.

Q4 · 6 marks

- Each element is processed once, hence O(n) time [1]; work per iteration is constant [1].
- Only current, best and a loop index are stored, hence O(1) auxiliary space [1].
- Invariant: current is the run ending at the latest index and best is the maximum within the prefix [1].
- Two correctly specified edge tests with outputs, e.g. [] → 0, [5] → 1, [2,2] → 2 [2].

Q5 · 8 marks

- The branch for a[mid] > target fails to reduce high [1].
- Correct it to high = mid - 1 [2].
- For a={3}, target=2: low=high=mid=0, else high=0 repeats indefinitely [2].
- The input must be sorted ascending [1].
- Each iteration discards about half of the search interval, so worst-case O(log n) [2].

Memory bank

Precondition: a condition the input must satisfy. Invariant: a property maintained throughout iteration. Complexity: how time or memory grows with input size. Test: an input with a known expected result.

H446 MEMORY BANK

Read the hook, then cover the precise wording and try to explain the concept without the analogy.

Thinking / 2.1.1 / Bouncer checking guest list

NOW BE PRECISE

Abstraction keeps only fields relevant to one job.

Careful: A different purpose may require different fields.

SIS / 2.2.1 / Sequence, Iteration, Selection

NOW BE PRECISE

Order, repeat, decide (OCR: branching).

Careful: SIS is a memory aid, not official OCR shorthand.

Recursion / 2.2.1 / Nesting dolls

NOW BE PRECISE

Calls go deeper until base case; results unwind back up.

Careful: Every recursive call requires a termination path.

Search / 2.3.1 / Higher/lower game

NOW BE PRECISE

Binary search halves the remaining *sorted* search region.

Careful: Return index not just value; unordered data breaks it.

MEMORY BANK / continued

Sort / 2.3.1 / Tidying playing cards

NOW BE PRECISE

Insertion sort builds a sorted prefix; bubble compares neighbours.

Careful: Quicksort performance depends on pivot and partitioning.

Methods / 2.2.2 / Maze with breadcrumbs

NOW BE PRECISE

Try, test, undo, retry = backtracking.

Careful: A heuristic does not guarantee the best route.

Big O / 2.3.1 / Customers at checkouts

NOW BE PRECISE

Think about how work grows when n grows.

Careful: Not exact execution time in seconds.

Paths / 2.3.1 / Price tags

NOW BE PRECISE

Dijkstra settles the smallest tentative distance; A* uses g+h.

Careful: Dijkstra needs non-negative weights; h can affect A* search order.

REVISION ROUTINE

1. Recall the hook without looking. 2. Explain in precise H446 language. 3. Trace or draw an example. 4. Attempt a fresh exam question.

MEMORY BANK

Cover the precise meaning and explain each memory hook aloud. The analogy helps you remember; the technical explanation earns the marks.

FOUR MOVES	Keep / prepare / order / decide
EXAM PRECISION	Think abstractly, ahead, procedurally and logically.
CACHE	Fast but can be stale
EXAM PRECISION	A cache saves computation but needs refresh or invalidation.
BOOKING	Validate / check / save / confirm
EXAM PRECISION	Subprocedures must respect dependencies and failed operations.
SIS	Sequence / iteration / selection
EXAM PRECISION	Constructs occur together; OCR also uses branching.
RETURN VALUE	Capture the result
EXAM PRECISION	A returned value changes a caller variable only if assigned or otherwise used.
BY REF	An alias, not a copy
EXAM PRECISION	A ref parameter can update the original caller variable.
MODEL / REALITY	Fit to purpose
EXAM PRECISION	Abstractions preserve relevant details, not every feature.

Memory bank · additional recall

Cover the precise wording, recall the memory hook, then explain the computer-science meaning aloud.

Maze: undo and retry	Backtracking Explore choices, undo failed routes and systematically try alternatives.
Detective with a chart	Data mining Find patterns; correlation alone does not demonstrate causation.
Car wash	Pipelining Overlap stages on different items to improve throughput, allowing for stalls.
Extra luggage	Auxiliary space Memory used in addition to the original input data.
Bigger inputs	Big O Compare asymptotic growth, not exact running time.
Sorted shelf	DistinctCount Adjacent comparisons count distinct values correctly when equal values are together.